

ヴァナ・ディールを描く

ファイナルファンタジー XI グラフィックス技術とリソースエクスプローラ構築

日本語版・作業原稿・第 0.1 版

本書の読み方

本書は、ファイナルファンタジー XI の描画技法とグラフィックスアーキテクチャを、読者が独立したリソースエクスプローラ兼ビジュアライザとして再構築できる深さで説明する。技術の説明、証拠の評価、C++実装、デスクトップアプリケーション統合を一つの累積工程として扱う。

編集上の範囲。 本書が扱うのは、ゲームのグラフィックス技術とその技術的背景のみである。

実装トラック。各章は一つの累積的な C++デスクトップアプリケーションを完成へ近づける。最終成果物は、インストール検出、リソースカタログ、バイナリ検査、対応 DAT デコード、モデル・ゾーン・キャラクター・エフェクト表示、テキスト・オーディオ閲覧、書き出し、セッション復元、回帰検証を備える。掲載コードは既出のモジュール契約へ接続し、各マイルストーンは実際にその機能を実装する。

目次

第 I 部 背景、方法、アプリケーション計画

第1章 ファイナルファンタジー XI と描画技術の時代背景

実装成果：C++クライアント・マイルストーン。 project skeleton、source-tree 規約、asset-to-pixel architecture、clean-room 実装境界を作る。

第2章 プラットフォーム、表示環境、技術的制約

実装成果：C++クライアント・マイルストーン。 platform、installation、resource map、display、client configuration の抽象化を定義し、同一 path を仮定しない。

第3章 リアルタイムフレームの構築

実装成果：C++クライアント・マイルストーン。 application loop、順序付き frame scheduler、追跡可能な非同期 asset-request pipeline を実装する。

第4章 調査方法と証拠台帳

実装成果：C++クライアント・マイルストーン。 各 parsed field と reconstructed behavior を source byte へ結び付ける diagnostics と evidence-register logger を作る。

第5章 視覚的証拠を過剰に断定せずに読む

実装成果：C++クライアント・マイルストーン。 byte diff、round-trip、capture comparison、experiment control を client test harness へ追加する。

第6章 アプリケーションアーキテクチャと実行可能なロードマップ

実装成果：アプリケーションコンテキスト、順序付き起動・終了、型付きサービス状態、および個別サブシステムが失敗しても機能するシェルを実装する。

第II部 バイナリリソースと描画基盤

第7章 デバイス、リソース、フレームライフサイクル

実装成果：C++クライアント・マイルストーン。 安全な DAT lookup と binary input、evidence-scoped な復号・難読化解除・展開 dispatcher、graphics device、resource creation、presentation、recovery を実装する。

第8章 インストール検出とリソースマップ

実装成果：安全なインストール検出を実装し、単一の真偽値ではなく完全な検証レポートを返す。

第9章 リソースカタログとブラウザ

実装成果：増分更新可能な検索カタログと、フィルタや再走査を越えて選択を保持するブラウザモデルを実装する。

第10章 バイナリインスペクタ、チャントツリー、フィールド由来情報

実装成果：後続すべてのデコーダが共有する、生データ／変換後バイナリインスペクタ、検証済みチャントツリー、ソース範囲ナビゲーションを実装する。

第11章 変換、座標空間、カメラ

実装成果：C++クライアント・マイルストーン。境界検査と endian 対応を持つ reader、typed numeric decoder、vector、matrix、transform、projection、検査可能 camera を作る。

第12章 頂点データ、頂点宣言、インデックスバッファ

実装成果：C++クライアント・マイルストーン。検証済み geometry record を typed header と flag set へ parse し、vertex layout/buffer と index buffer wrapper を作る。

第13章 テクスチャ、サーフェス、パレット、ミップマップ

実装成果：C++クライアント・マイルストーン。検証済み texture record を surface、palette、mip level へ decode し、source metadata を保持して upload/cache する。

第14章 テクスチャワークスペース

実装成果：遅延ミップデコード、チャンネル検査、由来情報表示、デバイス消失に耐える GPU アップロードを備えた対話型テクスチャワークスペースを実装する。

第15章 モデルワークスペース

実装成果：回転ナビゲーション、描画グループ選択、診断オーバーレイ、マテリアル検査、ソース移動を備えたモデルワークスペースを実装する。

第16章 Render State：深度、カリング、フォグ、塗りつぶし

実装成果：C++クライアント・マイルストーン。unknown bit を捨てない evidence-scoped flag decoder から explicit render-state descriptor を構築する。

第 17 章 Alpha Test、Alpha Blending、描画順序

実装成果：C++クライアント・マイルストーン。 alpha 関連 field を decode し、意図した depth/order 規則を持つ opaque、cutout、blended queue を作る。

第 18 章 Texture Stage と固定機能 Combiner

実装成果：C++クライアント・マイルストーン。 material-stage record を log/reproduce 可能な texture-stage と combiner description へ parse する。

第 19 章 Vertex Shader とプログラマブル技法

実装成果：C++クライアント・マイルストーン。 検証済み shader record、declaration、constant を capability check と明示的 fallback 付きで load する。

第 III 部 シーン構築と対話型ワークスペース

第 20 章 ワールドシーン：ゾーン、チャンク、描画グループ

実装成果：C++クライアント・マイルストーン。 zone resource graph を parse し、検証済み chunk、entity、reference、draw group から world scene を構築する。

第 21 章 ゾーンロードとシーン組み立て

実装成果：依存関係を考慮したゾーンロードを実装し、描画と検査に適した不変シーンスナップショットを公開する。

第 22 章 オブジェクトエクスプローラ、選択、プロパティ編集

実装成果：安定 ID、強調表示、可逆上書き、ソース移動を備え、ツリー、ビューポート、プロパティ検査を同期させる。

第 23 章 コリジョンジオメトリと移動診断

実装成果：明示的な信頼度ラベルを備えたコリジョン検査、加速、レイクエリ、任意接地、視覚差分オーバーレイを実装する。

第 24 章 可視性：視錐台カリング、距離、オクルージョン

実装成果：C++クライアント・マイルストーン。 supported visibility field を decode し、unknown flag を保持して frustum、distance、optional occlusion test を実装する。

第 25 章 地形、地面、大規模な環境ジオメトリ

実装成果：C++クライアント・マイルストーン。 terrain sector を parse し、reusable ground geometry、material、bounds、大規模 environment submission を作る。

第 26 章 静的オブジェクト、小物、建築ディテール

実装成果：C++クライアント・マイルストーン。 model と placement record を parse し、shared geometry と per-instance state で prop と architecture を instantiate する。

第 27 章 キャラクター：組み立て、スキニング、装備

実装成果：C++クライアント・マイルストーン。 検証済み character component を decode し、equipment、skeleton binding、skinned geometry を assemble する。

第 28 章 キャラクター組み立てワークスペース

実装成果：スロット解決、互換性検査、非同期置換、由来情報、プリセットを備えたキャラクター組み立てワークスペースを実装する。

第 29 章 アニメーション、ポーズ評価、モーションデータ

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 30 章 アニメーションブラウザと再生コントローラ

実装成果：本章のスケルタル評価器を駆動し、すべての代替処理を公開するアニメーションカタログとトランスポートコントローラを実装する。

第 31 章 ラइटニングモデル、頂点カラー、マテリアル応答

実装成果：C++クライアント・マイルストーン。 decoded material、light、vertex color、texture-stage input を検査可能な shading configuration へ変換する。

第 IV 部 大気、エフェクト、コンテンツツール

第 32 章 空、天体、時刻表現

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 33 章 フォグ、遠景ヘイズ、色調整

実装成果：C++クライアント・マイルストーン。 supported fog parameter を parse し、distance haze と scene-color control を runtime で検査可能にする。

第 34 章 水、反射、表面の動き

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 35 章 天候：雨、雪、風、環境パーティクル

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 36 章 環境制御と比較ワークスペース

実装成果：デコード状態と再構成方針を分離したまま編集、比較、キャプチャ、報告できる環境ワークスペースを実装する。

第 37 章 パーティクルシステム、軌跡、魔法エフェクト

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 38 章 エフェクトブラウザとシミュレーションワークスペース

実装成果：一つのタイムラインで命令、シミュレーション、描画、証拠ビューを駆動する決定論的エフェクトワークスペースを実装する。

第 39 章 透明表現、ソート、マルチパスエフェクト

実装成果：C++クライアント・マイルストーン。 supported pass record を transparent queue、target change、sorting policy、multi-pass composition へ変換する。

第 40 章 影、デカール、投影表現

実装成果：C++クライアント・マイルストーン。 supported projected-visual record を交換可能な shadow、decal、projector interface の背後で decode する。

第 41 章 インターフェース描画と 2D 合成

実装成果：C++クライアント・マイルストーン。 supported interface record を parse し、evidence-scoped 2D layer を compose する orthographic renderer を作る。

第42章 文字、アイコン、フォント、ウィンドウレイヤー

実装成果：C++クライアント・マイルストーン。 supported font、glyph、icon、window record を decode し、text layout、atlas、clipping、layering を実装する。

第43章 テキスト、ダイアログ、メッセージリソースエクスプローラ

実装成果：レイアウト差し替え可能で、トークン対応デコード、検索、由来情報、損失なし診断書き出しを備えたテキストエクスプローラを実装する。

第44章 オーディオカタログ、デコード、再生

実装成果：安全なヘッダ解析、非同期デコード、再生制御、ループ、未対応コーデック報告を備えた検索可能なオーディオワークスペースを実装する。

第V部 デスクトップアプリケーション、表示、性能

第45章 解像度、アスペクト比、表示スケーリング

実装成果：C++クライアント・マイルストーン。 decoded presentation setting、internal rendering dimension、output resolution、viewport、aspect ratio、UI scaling を分離する。

第46章 デスクトップシェル、ドッキング、ワークスペース寿命

実装成果：すべてのエクスプローラとビューアが使う複数文書シェル、およびワークスペースファクトリ／寿命契約を実装する。

第47章 コマンド、メニュー、設定、ツールウィンドウ

実装成果：一つの方針でメニュー、ショートカット、ダイアログ、ツールウィンドウを駆動する型付きコマンドと版付き設定を実装する。

第48章 入力、カメラ、ピッキング、ナビゲーションモード

実装成果：設定可能な入力、四つのナビゲーションモード、信頼できるポインタキャプチャ、描画ビューポートと一致する選択レイを実装する。

第49章 状態変更、バッチ処理、リソース再利用

実装成果：C++クライアント・マイルストーン。 source identity と必要な order を失わず state cache、draw sort、batching、resource reuse を追加する。

第 50 章 CPU と GPU の処理分担

実装成果：C++クライアント・マイルストーン。 parse/decode/upload timing と CPU-GPU frame telemetry で asset-to-frame 全経路を計測する。

第 51 章 バックグラウンドジョブ、キャンセル、UI 受け渡し

実装成果：カタログ化、デコード、ストリーミング、オーディオ、スクリーンショット、書き出しが共有するジョブシステムを実装する。

第 52 章 メモリ予算、ストリーミング、ロード挙動

実装成果：C++クライアント・マイルストーン。 dependency-aware streaming、cache、eviction、missing/corrupt/unsupported record の recoverable handling を作る。

第 53 章 リソースキャッシュと明示的寿命

実装成果：安定ハンドル、ロード統合、予算、診断、デバイス消失回復を備えた階層 CPU/GPU キャッシュを実装する。

第 54 章 パフォーマンス上のトレードオフと旧世代ハードウェア設計

実装成果：C++クライアント・マイルストーン。 file decoding、resource preparation、rendering、presentation を別々に benchmark し configurable budget を適用する。

第 55 章 スクリーンショット、レポート、データ書き出し

実装成果：マニフェストと明示変換方針を持つトランザクション型スクリーンショット、表、テキスト、ジオメトリ、レポート書き出しを実装する。

第 56 章 セッション、プリセット、再現可能なワークスペース

実装成果：実行時専用状態を保存せず調査を再現できる版付きセッションと用途別プリセットを実装する。

第 VI 部 ケーススタディ、検証、提供

第 57 章 不透明な環境描画を読み解く

実装成果：C++クライアント・マイルストーン。 source byte、parsed record、resource、state、depth behavior、draw order を verify する opaque environment case study を作る。

第 58 章 キャラクター描画を読み解く

実装成果：C++クライアント・マイルストーン。 decoded component、binding、animation、material、equipment、draw decision を公開する character case study を作る。

第 59 章 透明エフェクトを読み解く

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 60 章 証拠からフレームを再構成する

実装成果：C++クライアント・マイルストーン。 file evidence を parsed record、resource、state transition、intermediate target へ結ぶ frame-inspection mode を作る。

第 61 章 確認済みの構造、未解決事項、競合するモデル

実装成果：C++クライアント・マイルストーン。 provenance、confidence、contradiction、reproducible next test を持つ architecture/header/flag register を維持する。

第 62 章 ヴァナ・ディールの映像技術から得られる教訓

実装成果：C++クライアント・マイルストーン。 検証済み DAT-reading/rendering system を testable visual client へ統合し、regression scene と extension point を文書化する。

第 63 章 テスト、ファジング、回帰ハードニング

実装成果：対話操作なしで全デコーダとアプリ境界を検査する階層テストランナとファズ入口を実装する。

第 64 章 パッケージング、文書化、安全な拡張

実装成果：ツールチェーン非依存のリリース配置、マニフェスト、初回起動、診断バンドル、拡張登録、最終受け入れチェックリストを作成する。

付録 A-J：API、Render State、Combiner、数学、証拠台帳、用語集、年表、参考文献、索引。

第 I 部 背景、方法、アプリケーション計画

第 7 章 デバイス、リソース、フレームライフサイクル

実装成果：C++クライアント・マイルストーン。 安全な DAT lookup と binary input、evidence-

scoped な復号・難読化解除・展開 dispatcher、graphics device、resource creation、presentation、recovery を実装する。

第 8 章 インストール検出とリソースマップ

実装成果：安全なインストール検出を実装し、単一の真偽値ではなく完全な検証レポートを返す。

第 9 章 リソースカタログとブラウザ

実装成果：増分更新可能な検索カタログと、フィルタや再走査を越えて選択を保持するブラウザモデルを実装する。

第 10 章 バイナリインスペクタ、チャンクツリー、フィールド由来情報

実装成果：後続すべてのデコーダが共有する、生データ／変換後バイナリインスペクタ、検証済みチャンクツリー、ソース範囲ナビゲーションを実装する。

第 11 章 変換、座標空間、カメラ

実装成果：C++クライアント・マイルストーン。境界検査と endian 対応を持つ reader、typed numeric decoder、vector、matrix、transform、projection、検査可能 camera を作る。

第 12 章 頂点データ、頂点宣言、インデックスバッファ

実装成果：C++クライアント・マイルストーン。検証済み geometry record を typed header と flag set へ parse し、vertex layout/buffer と index buffer wrapper を作る。

第 13 章 テクスチャ、サーフェス、パレット、ミップマップ

実装成果：C++クライアント・マイルストーン。検証済み texture record を surface、palette、mip level へ decode し、source metadata を保持して upload/cache する。

第 14 章 テクスチャワークスペース

実装成果：遅延ミップデコード、チャンネル検査、由来情報表示、デバイス消失に耐える GPU アップロードを備えた対話型テクスチャワークスペースを実装する。

第 15 章 モデルワークスペース

実装成果：回転ナビゲーション、描画グループ選択、診断オーバーレイ、マテリアル検査、ソース移動を備えたモデルワークスペースを実装する。

第 16 章 Render State：深度、カリング、フォグ、塗りつぶし

実装成果：C++クライアント・マイルストーン。 unknown bit を捨てない evidence-scoped flag decoder から explicit render-state descriptor を構築する。

第 17 章 Alpha Test、Alpha Blending、描画順序

実装成果：C++クライアント・マイルストーン。 alpha 関連 field を decode し、意図した depth/order 規則を持つ opaque、cutout、blended queue を作る。

第 18 章 Texture Stage と固定機能 Combiner

実装成果：C++クライアント・マイルストーン。 material-stage record を log/reproduce 可能な texture-stage と combiner description へ parse する。

第 19 章 Vertex Shader とプログラマブル技法

実装成果：C++クライアント・マイルストーン。 検証済み shader record、declaration、constant を capability check と明示的 fallback 付きで load する。

第 II 部 バイナリリソースと描画基盤

第 20 章 ワールドシーン：ゾーン、チャンク、描画グループ

実装成果：C++クライアント・マイルストーン。 zone resource graph を parse し、検証済み chunk、entity、reference、draw group から world scene を構築する。

第 21 章 ゾーンロードとシーン組み立て

実装成果：依存関係を考慮したゾーンロードを実装し、描画と検査に適した不変シーンスナップショットを公開する。

第 22 章 オブジェクトエクスプローラ、選択、プロパティ編集

実装成果：安定 ID、強調表示、可逆上書き、ソース移動を備え、ツリー、ビューポート、プロパティ検査を同期させる。

第 23 章 コリジョンジオメトリと移動診断

実装成果：明示的な信頼度ラベルを備えたコリジョン検査、加速、レイクエリ、任意接地、視覚差分オーバーレイを実装する。

第 24 章 可視性：視錐台カリング、距離、オクルージョン

実装成果：C++クライアント・マイルストーン。 supported visibility field を decode し、unknown flag を保持して frustum、distance、optional occlusion test を実装する。

第 25 章 地形、地面、大規模な環境ジオメトリ

実装成果：C++クライアント・マイルストーン。 terrain sector を parse し、reusable ground geometry、material、bounds、大規模 environment submission を作る。

第 26 章 静的オブジェクト、小物、建築ディテール

実装成果：C++クライアント・マイルストーン。 model と placement record を parse し、shared geometry と per-instance state で prop と architecture を instantiate する。

第 27 章 キャラクター：組み立て、スキニング、装備

実装成果：C++クライアント・マイルストーン。 検証済み character component を decode し、equipment、skeleton binding、skinned geometry を assemble する。

第 28 章 キャラクター組み立てワークスペース

実装成果：スロット解決、互換性検査、非同期置換、由来情報、プリセットを備えたキャラクター組み立てワークスペースを実装する。

第 29 章 アニメーション、ポーズ評価、モーションデータ

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 30 章 アニメーションブラウザと再生コントローラ

実装成果：本章のスケルタル評価器を駆動し、すべての代替処理を公開するアニメーションカタログとトランスポートコントローラを実装する。

第 31 章 ライティングモデル、頂点カラー、マテリアル応答

実装成果：C++クライアント・マイルストーン。 decoded material、light、vertex color、texture-stage input を検査可能な shading configuration へ変換する。

第 III 部 シーン構築と対話型ワークスペース

第 32 章 空、天体、時刻表現

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 33 章 フォグ、遠景ヘイズ、色調整

実装成果：C++クライアント・マイルストーン。 supported fog parameter を parse し、distance haze と scene-color control を runtime で検査可能にする。

第 34 章 水、反射、表面の動き

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 35 章 天候：雨、雪、風、環境パーティクル

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 36 章 環境制御と比較ワークスペース

実装成果：デコード状態と再構成方針を分離したまま編集、比較、キャプチャ、報告できる環境ワークスペースを実装する。

第 37 章 パーティクルシステム、軌跡、魔法エフェクト

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 38 章 エフェクトブラウザとシミュレーションワークスペース

実装成果：一つのタイムラインで命令、シミュレーション、描画、証拠ビューを駆動する決定論的エフェクトワークスペースを実装する。

第 39 章 透明表現、ソート、マルチパスエフェクト

実装成果：C++クライアント・マイルストーン。 supported pass record を transparent queue、target change、sorting policy、multi-pass composition へ変換する。

第 40 章 影、デカル、投影表現

実装成果：C++クライアント・マイルストーン。 supported projected-visual record を交換可能な shadow、decals、projector interface の背後で decode する。

第41章 インターフェース描画と2D合成

実装成果：C++クライアント・マイルストーン。 supported interface record を parse し、evidence-scoped 2D layer を compose する orthographic renderer を作る。

第42章 文字、アイコン、フォント、ウィンドウレイヤー

実装成果：C++クライアント・マイルストーン。 supported font、glyph、icon、window record を decode し、text layout、atlas、clipping、layering を実装する。

第43章 テキスト、ダイアログ、メッセージリソースエクスプローラ

実装成果：レイアウト差し替え可能で、トークン対応デコード、検索、由来情報、損失なし診断書き出しを備えたテキストエクスプローラを実装する。

第44章 オーディオカタログ、デコード、再生

実装成果：安全なヘッダ解析、非同期デコード、再生制御、ループ、未対応コーデック報告を備えた検索可能なオーディオワークスペースを実装する。

第IV部 大気、エフェクト、コンテンツツール

第45章 解像度、アスペクト比、表示スケーリング

実装成果：C++クライアント・マイルストーン。 decoded presentation setting、internal rendering dimension、output resolution、viewport、aspect ratio、UI scaling を分離する。

第46章 デスクトップシェル、ドッキング、ワークスペース寿命

実装成果：すべてのエクスプローラとビューアが使う複数文書シェル、およびワークスペースファクトリ／寿命契約を実装する。

第47章 コマンド、メニュー、設定、ツールウィンドウ

実装成果：一つの方針でメニュー、ショートカット、ダイアログ、ツールウィンドウを駆動する型付きコマンドと版付き設定を実装する。

第48章 入力、カメラ、ピッキング、ナビゲーションモード

実装成果：設定可能な入力、四つのナビゲーションモード、信頼できるポインタキャプチャ、描画ビューポートと一致する選択レイを実装する。

第 49 章 状態変更、バッチ処理、リソース再利用

実装成果：C++クライアント・マイルストーン。 source identity と必要な order を失わず state cache、draw sort、batching、resource reuse を追加する。

第 50 章 CPU と GPU の処理分担

実装成果：C++クライアント・マイルストーン。 parse/decode/upload timing と CPU-GPU frame telemetry で asset-to-frame 全経路を計測する。

第 51 章 バックグラウンドジョブ、キャンセル、UI 受け渡し

実装成果：カタログ化、デコード、ストリーミング、オーディオ、スクリーンショット、書き出しが共有するジョブシステムを実装する。

第 52 章 メモリ予算、ストリーミング、ロード挙動

実装成果：C++クライアント・マイルストーン。 dependency-aware streaming、cache、eviction、missing/corrupt/unsupported record の recoverable handling を作る。

第 53 章 リソースキャッシュと明示的寿命

実装成果：安定ハンドル、ロード統合、予算、診断、デバイス消失回復を備えた階層 CPU/GPU キャッシュを実装する。

第 54 章 パフォーマンス上のトレードオフと旧世代ハードウェア設計

実装成果：C++クライアント・マイルストーン。 file decoding、resource preparation、rendering、presentation を別々に benchmark し configurable budget を適用する。

第 55 章 スクリーンショット、レポート、データ書き出し

実装成果：マニフェストと明示変換方針を持つトランザクション型スクリーンショット、表、テキスト、ジオメトリ、レポート書き出しを実装する。

第 56 章 セッション、プリセット、再現可能なワークスペース

実装成果：実行時専用状態を保存せず調査を再現できる版付きセッションと用途別プリセットを実装する。

第 V 部 デスクトップアプリケーション、表示、性能

第 57 章 不透明な環境描画を読み解く

実装成果：C++クライアント・マイルストーン。 source byte、parsed record、resource、state、depth behavior、draw order を verify する opaque environment case study を作る。

第 58 章 キャラクター描画を読み解く

実装成果：C++クライアント・マイルストーン。 decoded component、binding、animation、material、equipment、draw decision を公開する character case study を作る。

第 59 章 透明エフェクトを読み解く

実装成果：本章の統合アプリケーションチェックポイントを完成させる。

第 60 章 証拠からフレームを再構成する

実装成果：C++クライアント・マイルストーン。 file evidence を parsed record、resource、state transition、intermediate target へ結ぶ frame-inspection mode を作る。

第 61 章 確認済みの構造、未解決事項、競合するモデル

実装成果：C++クライアント・マイルストーン。 provenance、confidence、contradiction、reproducible next test を持つ architecture/header/flag register を維持する。

第 62 章 ヴァナ・ディールの映像技術から得られる教訓

実装成果：C++クライアント・マイルストーン。 検証済み DAT-reading/rendering system を testable visual client へ統合し、regression scene と extension point を文書化する。

第 63 章 テスト、ファジング、回帰ハードニング

実装成果：対話操作なしで全デコーダとアプリ境界を検査する階層テストランナとファズ入口を実装する。

第 64 章 パッケージング、文書化、安全な拡張

実装成果：ツールチェーン非依存のリリース配置、マニフェスト、初回起動、診断バンドル、拡張登録、最終受け入れチェックリストを作成する。

第 VI 部 ケーススタディ、検証、提供

付録 A-J：API、Render State、Combiner、数学、証拠台帳、用語集、年表、参考文献、索引。

第I部 背景、方法、アプリケーション計画

付録A-J：API、Render State、Combiner、数学、証拠台帳、用語集、年表、参考文献、索引。

第I部 背景と方法

第1章 ファイナルファンタジーXIと描画技術の時代背景

予定するデータ経路: 歴史的な asset pipeline、storage 制約、および instructional code と original client についての claim を分離する clean-room 境界。

C++クライアント・マイルストーン: project skeleton、source-tree 規約、asset-to-pixel architecture、clean-room 実装境界を作る。

第2章 プラットフォーム、表示環境、技術的制約

予定するデータ経路: client/build identity、installation layout、resource identifier、lookup context、display target、platform ごとの差異。

C++クライアント・マイルストーン: platform、installation、resource map、display、client configuration の抽象化を定義し、同一 path を仮定しない。

第3章 リアルタイムフレームの構築

予定するデータ経路: resource identifier から file lookup、byte 取得、検証済み変換、parse、upload、draw submission、presentation までの全経路。

C++クライアント・マイルストーン: application loop、順序付き frame scheduler、追跡可能な非同期 asset-request pipeline を実装する。

第4章 調査方法と証拠台帳

予定するデータ経路: raw byte range、hash、offset、decoded artifact、header、flag、capture、trace、競合解釈の evidence record。

C++クライアント・マイルストーン: 各 parsed field と reconstructed behavior を source byte へ結び付ける diagnostics と evidence-register logger を作る。

第5章 視覚的証拠を過剰に断定せずに読む

予定するデータ経路: encryption、obfuscation、compression、encoding、corruption、unknown field を過剰主張せず区別する controlled test。

C++クライアント・マイルストーン: byte diff、round-trip、capture comparison、experiment control を client test harness へ追加する。

第II部 バイナリリソース、グラフィックスAPI、描画の基礎

第7章 デバイス、リソース、フレームライフサイクル

予定するデータ経路: resource table と DAT lookup、安全な path、境界検査付き file I/O、検証済みの復号または難読化解除、展開、validation gate、device resource、frame lifecycle。

C++クライアント・マイルストーン: 安全な DAT lookup と binary input、evidence-scoped な復号・難読化解除・展開 dispatcher、graphics device、resource creation、presentation、recovery を実装する。

第11章 変換、座標空間、カメラ

予定するデータ経路: byte order、scalar/vector encoding、coordinate 規約、packed value、transform、projection、camera。

C++クライアント・マイルストーン: 境界検査と endian 対応を持つ reader、typed numeric decoder、vector、matrix、transform、projection、検査可能 camera を作る。

第12章 頂点データ、頂点宣言、インデックスバッファ

予定するデータ経路: DAT record header、tag、size、count、offset、alignment、bitfield、unknown flag、vertex layout、index stream、ownership。

C++クライアント・マイルストーン: 検証済み geometry record を typed header と flag set へ parse し、vertex layout/buffer と index buffer wrapper を作る。

第13章 テクスチャ、サーフェス、パレット、ミップマップ

予定するデータ経路: texture record header と payload、dimension、format、palette、mip chain、候補となる layout 変換、validation、upload、cache。

C++クライアント・マイルストーン: 検証済み texture record を surface、palette、mip level へ decode し、source metadata を保持して upload/cache する。

第16章 Render State : 深度、カリング、フォグ、塗りつぶし

予定するデータ経路: decoded state field と flag を depth、culling、fog、fill、rasterization behavior へ対応付け、unknown bit を保持する方法。

C++クライアント・マイルストーン: unknown bit を捨てない evidence-scoped flag decoder から explicit render-state descriptor を構築する。

第17章 Alpha Test、Alpha Blending、描画順序

予定するデータ経路: alpha-test/blending field、comparison function、blend factor、pass metadata、depth behavior、draw order への影響。

C++クライアント・マイルストーン: alpha 関連 field を decode し、意図した depth/order 規則を持つ opaque、cutout、blended queue を作る。

第 18 章 Texture Stage と固定機能 Combiner

予定するデータ経路: material header、texture reference、stage count、operation field、coordinate selector、constant、fixed-function combiner state。

C++クライアント・マイルストーン: material-stage record を log/reproduce 可能な texture-stage と combiner description へ parse する。

第 19 章 Vertex Shader とプログラマブル技法

予定するデータ経路: shader program payload、declaration、constant、capability flag、validation、documented fallback。

C++クライアント・マイルストーン: 検証済み shader record、declaration、constant を capability check と明示的 fallback 付きで load する。

第 III 部 シーン構築と対話型ワークスペース

第 20 章 ワールドシーン：ゾーン、チャンク、描画グループ

予定するデータ経路: zone header、root record、chunk directory、object table、reference、draw group、未解決 record type。

C++クライアント・マイルストーン: zone resource graph を parse し、検証済み chunk、entity、reference、draw group から world scene を構築する。

第 24 章 可視性：視錐台カリング、距離、オクルージョン

予定するデータ経路: bounds/visibility record、distance または LOD field、culling flag、occlusion 仮説、controlled observation。

C++クライアント・マイルストーン: supported visibility field を decode し、unknown flag を保持して frustum、distance、optional occlusion test を実装する。

第 25 章 地形、地面、大規模な環境ジオメトリ

予定するデータ経路: terrain header、grid、sector、vertex/index range、surface reference、placement data、terrain 固有 flag。

C++クライアント・マイルストーン: terrain sector を parse し、reusable ground geometry、material、bounds、大規模 environment submission を作る。

第 26 章 静的オブジェクト、小物、建築ディテール

予定するデータ経路: model/mesh-group header、placement record、resource reference、transform、variant、instance flag、architecture assembly。

C++クライアント・マイルストーン: model と placement record を parse し、shared geometry と per-instance state で prop と architecture を instantiate する。

第 27 章 キャラクター：組み立て、スキニング、装備

予定するデータ経路: character component table、equipment reference、skeleton/palette binding、weight data、part flag、assembly rule。

C++クライアント・マイルストーン: 検証済み character component を decode し、equipment、skeleton binding、skinned geometry を assemble する。

第 29 章 アニメーション、ポーズ評価、モーションデータ

予定するデータ経路：スケルタルモーションに加え、正規化されたエフェクト曲線、スケジュール上の発生期間、粒子ごとの寿命、セクション依存のアニメーション命令を扱う。

C++クライアント到達点：スケジュール時刻、放出時刻、粒子年齢を混同せず、スケルタルクリップと再利用可能なエフェクト曲線を評価する。

第 31 章 ライティングモデル、頂点カラー、マテリアル応答

予定するデータ経路: material/light record、vertex color、parameter、texture reference、response flag、再現可能な shading result。

C++クライアント・マイルストーン: decoded material、light、vertex color、texture-stage input を検査可能な shading configuration へ変換する。

第 IV 部 環境表現と特殊描画

第 32 章 空、天体、時刻表現

予定するデータ経路：環境レコード、カメラ追従の空ジオメトリ、天体リソース、通常の配置表に存在しないジェネレーター起動型の空レイヤーを扱う。

C++クライアント到達点：復号済みのスカイドームを最初に描き、裏付けのあるエフェクトレコードから天体と雲のレイヤーを起動する。

第 33 章 フォグ、遠景ヘイズ、色調整

予定するデータ経路: environment/fog record、distance range、color、interpolation field、zone/weather reference、unknown value。

C++クライアント・マイルストーン: supported fog parameter を parse し、distance haze と scene-color control を runtime で検査可能にする。

第 34 章 水、反射、表面の動き

予定するデータ経路: 配置された水面、ジェネレーター起動型の表面、UV 移動、頂点アルファ、ブレンドと深度状態、反射らしい色と実際の反射処理の境界を扱う。

C++クライアント到達点: 配置型とエフェクト型の水面を通常の透明描画経路で処理し、確認済みの UV アニメーションだけを実装する。

第 35 章 天候: 雨、雪、風、環境パーティクル

予定するデータ経路: 天候 ID、環境外観、降水ジェネレーター、放出範囲、寿命、風応答、ゾーン固有の起動条件を扱う。

C++クライアント到達点: 天候状態、環境補間、粒子放出を別々の系として実装し、明示的なリソース参照で接続する。

第 37 章 パーティクルシステム、軌跡、魔法エフェクト

予定するデータ経路: セクション化されたジェネレーター、命令フレーミング、リソース、付着、放出、寿命、移動、拡縮、色、曲線、UV アニメーション、子ジェネレーター、ブレンド/深度分類、未解明命令を扱う。

C++クライアント到達点: 未知命令を保存し、実装した挙動ごとに根拠を提示できる境界検査付きパーサーと決定論的エフェクトランタイムを構築する。

第 39 章 透明表現、ソート、マルチパスエフェクト

予定するデータ経路: pass metadata、transparent ordering field、render-target reference、sorting key、composition rule、unknown control bit。

C++クライアント・マイルストーン: supported pass record を transparent queue、target change、sorting policy、multi-pass composition へ変換する。

第 40 章 影、デカルル、投影表現

予定するデータ経路: shadow/decal/projector record、geometry/texture reference、placement field、

state flag、競合する rendering model。

C++クライアント・マイルストーン: supported projected-visual record を交換可能な shadow、decal、projector interface の背後で decode する。

第V部 表示とパフォーマンス

第41章 インターフェース描画と2D合成

予定するデータ経路: interface resource container、sprite/layout record、coordinate rectangle、clipping data、layer reference、window flag。

C++クライアント・マイルストーン: supported interface record を parse し、evidence-scoped 2D layer を compose する orthographic renderer を作る。

第42章 文字、アイコン、フォント、ウィンドウレイヤー

予定するデータ経路: text/icon table、character encoding、font/glyph metric、atlas reference、layout field、clipping、window-layer record。

C++クライアント・マイルストーン: supported font、glyph、icon、window record を decode し、text layout、atlas、clipping、layering を実装する。

第45章 解像度、アスペクト比、表示スケーリング

予定するデータ経路: display configuration/presentation record、internal/output dimension、aspect ratio、viewport、safe area、UI scaling。

C++クライアント・マイルストーン: decoded presentation setting、internal rendering dimension、output resolution、viewport、aspect ratio、UI scaling を分離する。

第49章 状態変更、バッチ処理、リソース再利用

予定するデータ経路: stable resource identity、state key、deduplication、batching、cache reuse、flag 依存の visual order 保持。

C++クライアント・マイルストーン: source identity と必要な order を失わず state cache、draw sort、batching、resource reuse を追加する。

第50章 CPU と GPU の処理分担

予定するデータ経路: lookup、file I/O、transformation、parsing、upload、CPU preparation、submission、synchronization、GPU work 全体の instrumentation。

C++クライアント・マイルストーン: parse/decode/upload timing と CPU-GPU frame telemetry で asset-to-frame 全経路を計測する。

第 52 章 メモリ予算、ストリーミング、ロード挙動

予定するデータ経路: resource map/dependency、asynchronous request、memory budget、cache、eviction、zone transition、missing data、parse failure。

C++クライアント・マイルストーン: dependency-aware streaming、cache、eviction、missing/corrupt/unsupported record の recoverable handling を作る。

第 54 章 パフォーマンス上のトレードオフと旧世代ハードウェア設計

予定するデータ経路: capability limit、memory/bandwidth budget、transformation/parsing cost、submission cost、fill rate、repeatable tradeoff test。

C++クライアント・マイルストーン: file decoding、resource preparation、rendering、presentation を別々に benchmark し configurable budget を適用する。

第 VI 部 ケーススタディと総合分析

第 57 章 不透明な環境描画を読み解く

予定するデータ経路: 一つの environment resource を identifier/DAT lookup から変換、header、flag、geometry、material state、final draw まで追跡する。

C++クライアント・マイルストーン: source byte、parsed record、resource、state、depth behavior、draw order を verify する opaque environment case study を作る。

第 58 章 キャラクター描画を読み解く

予定するデータ経路: character resource を component lookup、header、flag、skeleton/motion record、assembly、skinning、material、pixel まで追跡する。

C++クライアント・マイルストーン: decoded component、binding、animation、material、equipment、draw decision を公開する character case study を作る。

第 59 章 透明エフェクトを読み解く

予定するデータ経路: 一つのエフェクトを、ジェネレーターの生バイトからセクション依存命令、ソース、放出、シミュレーション、描画順、ブレンド／深度状態、最終画素まで追跡する。

C++クライアント到達点: 対応済みの寄与をすべて再生・説明し、欠落と推測も公開する時系列エフェクト資料を作る。

第 60 章 証拠からフレームを再構成する

予定するデータ経路: source path/byte offset を parsed field、live resource、pass、draw、state change、target へ結び付ける frame reconstruction。

C++クライアント・マイルストーン: file evidence を parsed record、resource、state transition、intermediate target へ結ぶ frame-inspection mode を作る。

第 61 章 確認済みの構造、未解決事項、競合するモデル

予定するデータ経路: confirmed structure、strong inference、speculation、alternative、unknown bit を分離する versioned architecture/field dictionary。

C++クライアント・マイルストーン: provenance、confidence、contradiction、reproducible next test を持つ architecture/header/flag register を維持する。

第 62 章 ヴァナ・ディールの映像技術から得られる教訓

予定するデータ経路: validated resource を開き、unsupported data を安全に報告し、regression scene を render し、open question を記録する統合 client。

C++クライアント・マイルストーン: 検証済み DAT-reading/rendering system を testable visual client へ統合し、regression scene と extension point を文書化する。

付録

A. グラフィックス API リファレンス

B. Render State リファレンス

C. Texture Stage・Combiner リファレンス

D. 行列と座標空間の入門

E. ブレンド、深度、アルファ計算式

F. 証拠台帳テンプレート

G. 用語集

H. 技術的発見の年表

I. 参考文献と一次資料

J. 索引

この完全目次は、本書の作業用地図である。章題は対象範囲を示し、エンジン固有の結論は証拠が裏付ける場合にのみ追加する。

第Ⅰ部 — 方法と背景

この部の目的は、完全な実装図を断定することではない。後続の証拠を正確に読むための視点を与えることである。

第I部 背景、方法、アプリケーション計画

第1章 ファイナルファンタジーXIと描画技術の時代背景

ファイナルファンタジーXIは、リアルタイムグラフィックスが移行期にあった時代の作品である。この時代のレンダラは、固定機能パイプラインと、慎重に選ばれたプログラマブル技法を組み合わせつつ、メモリ、fill rate、CPU 時間、状態変更に関する厳しい予算の中で動作することが多かった。これは後年のレンダラを単純に縮小したものではない。当時利用できた資源とインターフェースで、一貫した永続世界を成立させるという別種の設計課題であった。

本章の範囲。以下は描画技術の時代全体についての説明である。ここに挙げる各技法をファイナルファンタジーXIが使用したことを、それだけで確定するものではない。

C++クライアント・マイルストーン。 project skeleton、source-tree 規約、asset-to-pixel architecture、clean-room 実装境界を作る。

```
// Cumulative implementation excerpt: one clean asset-to-pixel boundary.
struct ClientApp {
    AssetRepository assets;
    Renderer renderer;
    EvidenceLog evidence;

    int run() {
        while (renderer.pumpWindow()) {
            Scene scene = assets.buildScene(evidence);
            renderer.draw(scene, evidence);
        }
        return 0;
    }
};

int main() { return ClientApp{}.run(); }
```

1.1 リアルタイムで連続する世界

オンラインRPGの世界では、レンダラは複数の役割を同時に担う。読み取りやすい景観を作り、キャラクターと装備をアニメーションさせ、変化する天候と時刻の雰囲気を保ち、移動や戦闘の最中にも判読できるインターフェースを表示しなければならない。これらの目的は同じフレーム予算を奪い合う。遠景にはフォグが必要かもしれない、魔法エフェクトには透明表現が必要かもしれない、キャラクターには複数のマテリアルが見えるかもしれない。そしてすべてを安定した順序で合成する必要がある。

したがって、シーンの視覚的な性格は個々の asset だけでなく、何を優先するかによって決まる。ジオメトリを抑えても、色、フォグ、アニメーション、テクスチャ処理、レイヤーによって広がりを感じさせることができる。技術分析では、画像のどの部分がある役割を担っているように見えるかを問い、その機構を証拠で確かめるべきである。

1.2 固定機能の考え方とプログラマブルな可能性

当時のグラフィックス用語には、transform、vertex data、texture、render state、深度規則、blending、texture combiner が含まれる。固定機能パスでは、頂点を変換し、ライティングを適用し、テクスチャをサンプリングし、色を組み合わせ、深度を判定し、結果を書き込むという定められた流れでこれらを扱える。プログラマブルな頂点処理は頂点側で行える処理の幅を広げたが、固定機能パスの有用性をなくしたわけではない。

このため、視覚的な見た目だけで機能を分類してはならない。色の勾配は vertex color、texture stage、フォグ、あるいはそれらの組み合わせから生じうる。動く表面は、変換された texture coordinate、animated texture、ジオメトリの運動、複数パスのいずれでも表現できる。後の API 章では、これらの可能性を区別する方法を扱う。

1.3 画像を形作る制約

この時代には、いくつかの制約が特に強く影響した。texture memory は、コンパクトな再利用と慎重な解像度選択を促す。限られた fill rate では、大きなブレンド領域や反復する全画面処理が高価になる。CPU からの draw 送出のコストは、似た描画をまとめる batch 処理を有利にする。深度と描画順序も重要である。不透明面、切り抜きテクスチャ、半透明エフェクトは、すべてを同じようには扱えないためである。

これらは設計上の圧力であり、特定の実装の証明ではない。エンジンが不透明処理と半透明処理を分け、マテリアル状態を再利用し、遠景を簡略化し、追加ジオメトリよりテクスチャによる詳細表現を選ぶ理由を考える助けにはなる。ただし、そのような結論はゲーム固有の証拠が裏付ける場合にのみ、確認済みと記述する。

1.4 プラットフォームの文脈が重要な理由

プラットフォームは仕様表以上の意味を持つ。利用できるグラフィックス API、表示挙動、メモリ構成、開発者が前提としたデバッグ方法を定める。シーンや capture を比較するときは、調査対象のプラットフォームと build を記録することが重要である。ある環境で見た挙動は、レンダラ全体の設計ではなく、設定、互換性、またはプラットフォーム固有の経路を反映している可能性がある。

そのため本書では、技術的な主張を行う際に、観測の文脈を明記する。すなわち、プラットフォーム、シーン、カメラ条件、証拠の種類、検討した代替説明である。この方法により、不確実性を偽の確実性に変えることなく、後から修正できる。

1.5 調査を導く問い

最初に扱う問いは、意図的に具体的なものとする。距離によってどの機能が見え方を変えるか。天候、時刻、カメラ角度の変化で何がかわるか。どの draw が不透明、切り抜き、ブレンドに当たるか。どの texture と状態群が繰り返し使われるか。単一パスに依存する結果と、合成を必要とする結果は何か。その答えが、歴史的背景から再現可能な技術分析への橋渡しとなる。

証拠基準。見慣れた技法は発見ではなく仮説である。証拠台帳には、観測したこと、残る競合説明、それらを解決する観測を記録する。

1.6 実装規約

実装トラックでは、後続章で `std::span` や `std::endian` などの機能を用いるため、C++20 を言語要件とする。これは compiler、editor、IDE、build system の指定ではない。本書が扱う graphics API は Windows の型と library を通じて公開されるため、描画 checkpoint の対象 platform は Windows である。

code には二つの形式がある。complete checkpoint listing は、必要な宣言、entry point、error path を含み、小さな実行ファイルとして compile できる。cumulative implementation excerpt は、その章で client へ追加する関数を示し、先行 checkpoint で導入済みの interface、または module contract で明示した interface だけに依存する。excerpt を単独 program として提示するものではない。

適合する toolchain では、指定された C++ translation unit を C++20 mode で compile し、Direct3D 8 の symbol を提供する platform import library とともに一つの実行ファイルへ link する。command の具体的な書式は toolchain ごとに異なるため、本書では固定しない。選択した toolchain は、標準 C++ library、Windows 宣言、Direct3D 8 宣言、および対応する import library を提供しなければならない。

1.7 累積 module の規約

client は、特定の directory layout ではなく責務で分割する。core は error、identifier、byte range、checked arithmetic、vector、matrix を所有する。evidence は provenance と confidence record を所有する。dat は bounded file input、chunk walk、transformation dispatch、typed record view を所有する。assets は decode 済み CPU resource と cache を所有する。scene は camera、instance、animation、visibility、draw description を所有する。platform は window と event pump を所有する。graphics は device と GPU resource を所有する。client は schedule と統合を所有する。

読者は file 名を変更しても、translation unit を統合してもよい。ただし依存方向は守る。evidence と DAT parser は graphics へ依存しない。decode 済み asset は source identity を保持する。graphics は検証済み description を受け取り、file byte を再解釈しない。未対応 record は、回復可能な diagnostic result として保持する。

1.8 checkpoint の完了条件

各 checkpoint には四つの gate がある。textbook 内だけに存在する未宣言 symbol を残さず compile できること。不正 input が range 外 access ではなく bounded error になること。文書化した fixture から期待する構造結果が得られること。対応済み field の source byte range を evidence log から追跡できることである。見た目の一致は追加 test であり、これらの gate の代用ではない。

最初の実行ファイルは window を開き、graphics device を作成し、frame を clear して present し、一時的な device loss から復帰する。二番目は、明示的に選んだ DAT file を読み、未対応 payload を解釈せず、retail client で確認された chunk 境界を walk する。以後は、検証済み asset family を一つずつ追加する。この順序なら、format 全体の対応前から有用な custom visual client を構築できる。

1.9 段階ごとの gate

Gate A — foundation（第 1～5 章）。前提：core と evidence の責務を実装した C++20 code。合格条件：client shell が実行でき、byte 比較が deterministic で、すべての experiment が input を記録する。

Gate B — byte から GPU まで（第 6～13 章）。前提：Gate A と第 7 章の二つの complete checkpoint。合格条件：一種類の検証済み record family が、bounded input、chunk walk、対応済み transformation、typed parsing、upload、state application を通り、visible diagnostic draw になる。未対応 record は名前付き failure として残る。

Gate C — scene reconstruction（第 14～24 章）。前提：Gate B。合格条件：一つの小規模 fixture scene が reference を解決し、static geometry、camera visibility、environment state、および少なくとも一つの character または animated object を構築する。renderer は raw file byte を読まない。

Gate D — ordered composition（第 25～30 章）。前提：Gate C。合格条件：deterministic effect simulation、transparent ordering、render-target transition、2D composition を pause でき、source offset と render descriptor を inspect できる。

Gate E — resilient client（第 31～40 章）。前提：Gate A～D。合格条件：cache、streaming、budget、case-study report、regression scene が一つの実行ファイルで動作する。parser revision を、過去の結果を消さずに evidence、structure、image で比較できる。

第 2 章 プラットフォーム、表示環境、技術的制約

ファイナルファンタジー XI の展開には、PlayStation 2 版、Windows 版、Xbox 360 版が含まれる。出版元の記録は、比較調査にとって重要な終点も示している。PlayStation 2 版と Xbox 360 版のサービスは 2016 年 3 月 31 日に終了し、Windows 版は継続した。本章ではこのプラットフォーム史を事実上の基礎として用いる。ただし、すべての版が同一のレンダラ、asset 経路、表示設定を用いたとは仮定しない。

一次資料に関する注記。本章のプラットフォーム史は、スクウェア・エニックスの公式「WE ARE VANA'DIEL」年表と公式移行告知に基づく。版ごとの詳細な挙動には、個別の証拠記録が必要である。

C++クライアント・マイルストーン。 platform、installation、resource map、display、client configuration の抽象化を定義し、同一 path を仮定しない。

```
// Cumulative implementation excerpt: platform and installation are data, not globals.
struct ClientConfig {
    std::filesystem::path installRoot;
    std::vector<std::filesystem::path> romVolumes;
    std::uint32_t backBufferWidth = 1280;
    std::uint32_t backBufferHeight = 720;
    bool windowed = true;
};

std::filesystem::path resolveResource(
    const ClientConfig& c, std::size_t volume, std::filesystem::path relative) {
    if (volume >= c.romVolumes.size()) throw std::out_of_range("ROM volume");
    return c.installRoot / c.romVolumes[volume] / relative;
}
```

2.1 一つの世界と複数の技術的文脈

複数プラットフォームの作品では、世界、ゲーム規則、広い視覚方針を共有しながら、画面に到達する経路が異なりうる。グラフィックスインターフェース、texture format、リソース制限、出力タイミング、表示オプション、入力上の前提、後年の互換性対応によって差が生じることがある。したがって、同じシーンが二つのプラットフォームに存在することは、コンテンツの連続性を示しても、実装の同一性を示すものではない。

グラフィックス調査における実用的な単位は、単なるゲーム名ではない。特定の client、build、platform、configuration、scene である。これらの詳細を欠く capture や観測は、プラットフォーム由来の挙動とエンジン全般の挙動を無意識に混ぜてしまうため、比較が難しい。

2.2 表示環境も設計の一部である

レンダラは抽象的なスクリーンショットではなく、表示環境のための画像を生成する。画面サイズ、アスペクト比、安全領域、色変換、filtering、scaling はすべて、プレイヤーが読めるものに影響する。インターフェースパネル、細い輪郭、フォグの境界は、想定された表示環境を前提に選ばれることがある。現代の capture が異様に鮮明、ぼやけている、横長、または切り取られて見える場合、まず presentation path が変わったかを問うべきである。

これは、すべての見た目の差が表示上の artifact だという意味ではない。むしろ、pixel を証拠として扱う前に出力条件を記録するという規律を定めるものである。証拠台帳には、resolution、aspect

ratio、window または full-screen mode、scaling layer、capture method、camera の位置を含めるべきである。

2.3 制約は欠点ではなく予算である

リアルタイム描画はいくつもの予算によって左右される。memory は、どれだけの geometry、texture data、animation data、一時作業領域を保持できるかを制限する。bandwidth はリソースを移動できる速さを制限する。CPU 時間はシーン準備と draw 送出を制限する。graphics processing は geometry 処理、texture sampling、blending、pixel 出力を制限する。設計は、どこに詳細を置くと最も視覚価値が高いかを選ぶことで成立する。

この見方は、旧世代ハードウェアを単一のボトルネックとして扱うより有用である。あるシーンは geometry が軽くて overdraw が高価かもしれず、小さな texture を使っていても状態変更が多いかもしれず、memory には収まっていても準備に時間がかかるかもしれない。後のパフォーマンス章では、これらの分類を分けて扱い、単に効率的に見えるからという理由で機能の存在を断定しない。

2.4 ワールド描画への含意

大規模なオンライン世界では、距離の扱いが視覚言語になる。フォグは遠景と空をつなぎ、詳細の削減は近くのキャラクターのためにフレーム予算を残し、繰り返し使われるマテリアルはリソースを節約しながらゾーンの統一感を作り、レイヤー化された効果は画面のすべての pixel を同じ複雑さにせずに天候や魔法を伝えられる。これらは時代全体に見られる可能性であり、ファイナルファンタジー XI の特定の subsystem を確認した説明ではない。

同じ注意はプラットフォーム比較にも当てはまる。ある環境で effect が欠ける、変化する、または見えにくい場合には、設定の差、asset variant、rendering path、capture の制限、scene condition など、複数の説明が残る。有用な比較では、一度に一つの変数だけを変え、結果を記録する。

2.5 後続章のための capture protocol

今後の case study は、短い記録から始めるべきである。platform と client build、zone と座標または landmark、time と weather、判明している場合は camera position と field of view、display mode、

検証する正確な視覚的問いである。さらに、direct capture、observed gameplay、static image comparison、API trace、data inspection、implementation analysis を区別する。

この protocol は、否定的な結果にも価値を与える。文書化された条件で主張された effect を再現できないとしても、それだけで直ちに反証になるとは限らない。その主張が成立する条件を狭めることになる。再現可能性は、魅力的な説明と信頼できる技術解説を結ぶ橋である。

作業上の規則。最初に platform を記し、次に観測を述べ、その後で説明を評価する。プラットフォーム全体についての一般化で、特定の描画挙動に対する証拠を置き換えてはならない。

第3章 リアルタイムフレームの構築

フレームは一つの操作ではない。エンジンがシーンに何が存在するかを判断し、描画に必要なデータを準備し、グラフィックスシステムへ処理を送出し、得られた画像をプレイヤーが見る一枚に合成するまでの連鎖である。正確な順序はエンジンごとに異なり、platform や scene によっても変わってくる。本章は分析のための作業モデルであり、ファイナルファンタジー XI の内部図を確認済みとして示すものではない。

作業モデル。本章の段階は分析上の分類である。確認済みのフレーム順序には、client と観測された描画挙動からの証拠が必要である。

C++クライアント・マイルストーン。 application loop、順序付き frame scheduler、追跡可能な非同期 asset-request pipeline を実装する。

```
// Cumulative implementation excerpt: ordered frame work plus asynchronous loading.
class AssetPipeline {
    std::mutex mutex_;
    std::queue<DecodedAsset> ready_;
public:
    void request(ResourceId id) {
        std::thread([this, id] {
            auto decoded = decodeAsset(readResource(id));
            std::lock_guard lock(mutex_);
            ready_.push(std::move(decoded));
        }).detach();
    }
    void publishReady(Scene& scene) {
        std::lock_guard lock(mutex_);
        while (!ready_.empty()) { scene.install(std::move(ready_.front())); ready_.pop(); }
    }
};

void frame(ClientApp& app) {
    app.assets.publishReady(app.scene);
    app.scene.update();
    app.renderer.beginFrame(); app.renderer.draw(app.scene); app.renderer.present();
}
```

3.1 シーン状態から始める

すべてのフレームはシーン状態から始まる。プレイヤーと camera の位置、animation time、zone data、可視の entity、weather、時刻に関する parameter、interface state である。このデータの一部は毎フレーム変わり、一部は resource の load や event が起きた時だけ変わる。エンジンはこの混在した状態を、描画に適した小さな判断の集合へ変換しなければならない。

特に camera は重要である。世界そのものではなく、世界の見え方を定めるためである。camera の背後にある建物、viewing volume の外にある character、visibility rule によって隠される遠方 object は、そのフレームで draw を必要としない場合がある。後の章では camera transform、visibility test、フォグのような芸術的な距離表現を区別する。

3.2 作業を準備し選別する

draw が起きる前に、renderer は利用可能な resource を必要とする。vertex data、index data、texture、transform、material parameter、それらを解釈するための state である。エンジンは以前に準備した作業を再利用し、小さな部分だけを更新し、または新しい resource 集合を準備できる。また、候補となる object のうち、実際に送出する価値があるものを選ぶこともできる。

この選別はしばしば visibility management と呼ばれるが、単一の culling test より広い概念である。distance threshold、object category、zone partition、character priority、effect や interface element の特別規則を含みうる。見える結果だけでは、どの規則が使われたかは分からない。分析者はまず観測した変化に名前を付け、その後で候補となる説明を検証すべきである。

3.3 ジオメトリとマテリアル状態を送出する

draw submission は、何を描き、どのように描くかを記述する。少なくとも geometry を transform、material、graphics state の集合へ結び付ける。geometry は位置などの per-vertex value を与える。material は texture binding と組み合わせ規則を与える。state は depth test、culling、fog、alpha test、blending などの条件を定める。

効率的な renderer は似た draw の間で不要な変更を避けようとするが、効率だけが目的ではない。正しい画像には特定の順序が必要な場合がある。たとえば depth を書き込む surface と、すでに frame

bufferにある画像と blend しなければならない半透明 effect は異なる。重要なのは draw call の数だけではなく、意図した layer を現すためにどの state change が必要かである。

3.4 不透明、切り抜き、透明表現

有用な概念上の区分は、不透明処理、切り抜き処理、blend 処理である。不透明 pixel は通常、信頼できる depth coverage を与える。切り抜き処理は alpha threshold を用い、pixel を受け入れるか破棄するかを決める。これは葉や模様のある縁のような形に有用である。blend 処理は source color と、すでに frame buffer にある destination を組み合わせ、半透明、glow、その他の合成結果を作る。

これらの分類は描画挙動を示すものであり、asset file の名前を示すものではない。一つの object が複数の分類を必要とすることも、scene が特殊な effect のためにそれらを交互に処理することもある。本書では、透明に見えるすべての content が同じ path を通ると仮定せず、capture と state の証拠から実際の grouping を判断する。

3.5 表示を合成する

world と effect の処理の後に、renderer は interface layer、text、cursor、fade、その他の presentation element を加えることがある。そして完成した画像を表示経路へ present する。この最終段階には、world renderer の外側で buffering、synchronization、scaling、color conversion が含まれる場合がある。したがってスクリーンショットは、孤立した scene pass ではなく、完全な presentation chain の観測である。

後の case study では、フレームを証拠の timeline として再構成する。draw の前に何が存在するか、後に何が変わるか、どの state と resource が active か、完成画像に何が残るかを追う。目的は pass に名前を付けることだけではない。その順序が観測された pixel をなぜ生むかを説明することである。

3.6 本書で用いるフレームマップ

以後の章では、六つの問いを繰り返し用いる。どの scene data が存在するか。どの作業が選別されるか。どの geometry が送出されるか。どの material と state rule が active か。layer はどのように合成されるか。結果はどのように present されるか。この地図によって、後の主張を位置付けながら、ゲームの実際の順序が一般モデルと異なることを示す証拠にも対応できる。

証拠基準。提案された pass order は、再現可能な挙動、直接の API activity、implementation analysis、または複数の独立した裏付けに結びつくまで推測である。

3.7 実行可能な checkpoint

すべての subsystem を統合してから client を実行しようとしてはならない。checkpoint 0 は空 frame を present する。checkpoint 1 は検証済み chunk 境界を出力する。checkpoint 2 は、source record と state を inspect できる diagnostic geometry を描画する。checkpoint 3 では、その geometry を一種類の検証済み texture または mesh family で置き換える。checkpoint 4 では、検証済み reference から小規模 scene を構築する。animation、weather、effect、interface composition、streaming、optimization は、先行する image と evidence record を再現できてから追加する。

各 checkpoint では、最後に合格した実行ファイルと fixture hash を保存する。新しい parser が過去の結果を変えた場合、reference image を変更する前に evidence record を比較する。これにより、本書は無関係な sample の集合ではなく、incremental implementation path になる。

第 4 章 調査方法と証拠台帳

技術教本が信頼を得るためには、結論に至る道筋が見える形にする必要がある。この主題では、レンダラについての主張だけでなく、それを支える観測、観測した環境、残る代替説明、証拠に見合う確信度も記録することを意味する。証拠台帳は、その作業のための本書の共有記憶である。

編集上の規則。魅力的な説明だけでは十分ではない。ゲーム固有の技術的主張はすべて、証拠と確信度を示す記録へたどれる必要がある。

C++クライアント・マイルストーン。 各 parsed field と reconstructed behavior を source byte へ結び付ける diagnostics と evidence-register logger を作る。

```
// Cumulative implementation excerpt: every decoded field retains its byte provenance.
struct EvidenceEntry {
    ResourceId resource;
    std::uint64_t offset;
    std::uint32_t size;
    std::string field;
    std::string interpretation;
    Confidence confidence;
};

class EvidenceLog {
    std::vector<EvidenceEntry> entries_;
public:
    template<class T>
```

```

void field(ResourceId r, std::uint64_t off, std::string name,
           const T& value, Confidence c) {
    entries_.push_back({r, off, sizeof(T), std::move(name),
                       toDiagnosticString(value), c});
}
std::span<const EvidenceEntry> entries() const { return entries_; }
};

```

4.1 三つの確信度ラベル

確認済みは、再現可能な観測、一次資料、implementation analysis、または論点を曖昧さなく確立する他の資料によって、主張が直接裏付けられることを意味する。有力な推論は、複数の観測と確立された描画技法が、代替説明より一つの説明を強く支持する一方で、決定的な trace がまだないことを意味する。推測は、もっともらしい、または検証に有用な説明であるが、未証明であることを意味する。

これらのラベルは、著者の熱意ではなく主張を表す。有用な推測モデルは実験を導くことがある。確認済みの詳細は、狭く目立たないこともある。重要なのは、証拠がどこまで届くかを読者に示すことである。

4.2 証拠の種類

最も強い分析は、複数の証拠を組み合わせることが多い。直接の視覚観測は画面に何が出るかを示す。統制された比較は、記録された一つの条件を変えたとき何が変わるかを示す。API activity は resource、state change、draw submission を示しうる。data inspection は構造と関係を示しうる。implementation analysis は観測された挙動を code path へ結び付けうる。当時の documentation は、関係する platform や API の能力と用語を確立できる。

どの種類の証拠も、常に単独で十分とは限らない。capture は結果を示せても、機構を常に示すわけではない。code path が存在しても、調査中の scene で実行されたことを証明するとは限らない。file structure は意図を示唆しても、render state を証明しない。台帳には各資料の強さと限界の両方を記録する。

4.3 最低限の主張記録

各記録には、短い主張、安定した識別子、platform・client build・scene・capture condition、証拠の種類、観測の簡潔な説明、検討した代替説明、確信度ラベル、確信度を上げ下げする次の行動を含める。source file、capture、trace は、別の研究者が見つけられるだけ正確に名前を記すべきである。

たとえば、記録は surface が『water』であるとだけ書くべきではない。surface が動くか、camera によって見え方が変わるか、depth を書くか、blend するか、どの条件を固定したかを観測として記す。結論は不確実なままでも、観測は再利用できる。

4.4 統制された比較

統制された比較では、できる限り他の条件を安定させながら、一つの関連変数を変える。有用な変数には distance、camera angle、time、weather、model state、display mode、platform がある。記録には、何を変え、何を変えなかったかを記す。複数の変数が同時に動く場合も結果は興味深いかもしれないが、説明力は低くなる。

否定的な証拠も台帳に属する。提案された条件で目に見える変化が起きない場合、その結果は単純な説明を退けたり、未記録の変数が重要であることを示したりする。再現に失敗した記録は、文書化されていない成功より価値が高いことが多い。

4.5 観測と解釈を分ける

観測は証拠に近い言葉を使う。『閾値を越えると縁が消える』『camera を回すと色が変わる』『二つの draw が異なる blend state を使う』といった表現である。解釈は原因を提案する。distance culling、view-dependent texture coordinate、別の material pass などである。これらの文を分けることで、後からの修正が容易になる。

この区別は、見慣れた effect で特に重要である。揺らぐ surface が既知の技法に似ていても、似ていることは実装を特定しない。実験または trace が区別できるまで、証拠台帳は競合するモデルを残すべきである。

4.6 公開時の基準

章の最終的な文章は、台帳が支える確信度だけを述べるべきである。確認済みの発見は直接書ける。有力な推論は推論の根拠を示す。推測は目立つようにラベルを付け、検証可能な問いと組にする。証拠が衝突するとき、その不一致は隠すべき欠点ではなく、発見の一部である。

台帳テンプレート。主張、証拠、文脈、代替説明、確信度、次の検証。どれか一つでも欠けていれば、その記述は技術的結論にする準備ができていない。

第5章 視覚的証拠を過剰に断定せずに読む

完成した画像は不可欠な証拠であるが、レンダラをそのまま見通せる窓ではない。スクリーンショットには、geometry、texture、state rule、camera condition、display processing、timingの結果が含まれる。検証すべき関係を示すことはできても、見た目だけで単一の実装を通常は特定できない。

読み取りの規則。描画技法に名前を付ける前に、見える変化を記述する。

C++クライアント・マイルストーン。 byte diff、round-trip、capture comparison、experiment control を client test harness へ追加する。

```
// Cumulative implementation excerpt: byte diff and round-trip verification.
struct ByteDifference { std::size_t offset; std::byte expected, actual; };

std::vector<ByteDifference> diffBytes(
    std::span<const std::byte> a, std::span<const std::byte> b) {
    std::vector<ByteDifference> out;
    const auto n = std::min(a.size(), b.size());
    for (std::size_t i = 0; i < n; ++i)
        if (a[i] != b[i]) out.push_back({i, a[i], b[i]});
    for (std::size_t i = n; i < std::max(a.size(), b.size()); ++i)
        out.push_back({i, i < a.size() ? a[i] : std::byte{},
                       i < b.size() ? b[i] : std::byte{}});
    return out;
}

void requireRoundTrip(const DecodedAsset& asset) {
    const auto rebuilt = encodeAsset(asset);
    const auto differences = diffBytes(asset.sourceBytes, rebuilt);
    if (!differences.empty()) throw ValidationError("round-trip mismatch");
}
```

5.1 観測できることから始める

有用な観測は具体的で比較可能である。距離で縁が柔らくなる、ある surface が別の surface の背後で消える、camera を回すと色が変わる、particle が geometry を通して見える、weather condition で effect が変わる、といった記述である。これらは、機構を決めつけずに pixel と条件を述べる。

次の段階は候補となる機構を列挙することである。遠景の fade には fog、alpha gradient、texture change、distance selection、またはそれらの組み合わせが関わりうる。動く模様には animated texture coordinate、texture replacement、geometry motion、compositing が関わりうる。この一覧によって、最初に思い付いた説明が根拠のない結論になることを防ぐ。

5.2 変数を分ける比較を使う

比較は、一度に一つの関連条件だけを変えると最も強くなる。camera distance を変える間は location と display mode を安定させ、time または weather を変える間は camera position を安定させ、二つの documented client environment を比べる間は scene を安定させる。固定できなかった条件は、可能な限界として記録する。

一枚の画像より系列が重要である。測定した複数の distance で object が変化するなら、そのパターンは threshold、段階的な interpolation、camera-dependent response を示すかもしれない。結論は証拠に釣り合うべきである。系列は内部原因を確定する前に、まずパターンを確立できる。

5.3 presentation artifact に注意する

capture と display の段階は誤解を招く証拠を作りうる。scaling は縁を柔らかくし、compression は band や noise を作り、異なる color-management path は見かけの明るさを変え、frame timing は一時的な effect を誤解を招く瞬間に捉え、cropping は視覚的境界を説明する文脈を隠しうる。記録では可能な限り元の capture を残し、後の編集や conversion を示すべきである。

この注意は視覚的証拠を弱くするものではない。利用可能にするものである。文書化された条件で繰り返せる観測は、内部実装が不明でも価値がある。

5.4 不在を慎重に扱う

見える effect がないことは、system がないことの自動的な証拠ではない。effect は distance、direction、time、weather、state、platform、configuration、resource availability、timing window に依存している可能性がある。否定的な結果は、試験条件を明示し、もっともらしい条件を体系的に変えたときに情報量を持つ。

5.5 画像から技術的主張へ

責任ある技術的主張には三層がある。観測、代替説明、結論である。まず画像が示すことを述べる。次に、それらと両立する説明を挙げる。最後に、確信度に支えられた結論と、代替説明を決めるために必要な追加観測を述べる。この構造により、後続章は過剰に断定せず精密になれる。

公開時の基準。スクリーンショットは主張を図解するものであり、その背後にある証拠記録の代わりにはならない。

第6章 アプリケーションアーキテクチャと実行可能なロードマップ

本章では、調査計画を製品計画へ変換する。累積成果物は、インストールを検出し、リソースをカタログ化し、境界検査付きのバイナリ表示を開き、対応レコードをデコードし、モデルとゾーンを描画し、未対応データも検査可能なまま保持するデスクトップ型リソースエクスプローラである。

実装成果。

アプリケーションコンテキスト、順序付き起動・終了、型付きサービス状態、および個別サブシステムが失敗しても機能するシェルを実装する。

6.1 責務と所有

アプリケーションは、インストール検出、リソースカタログ、デコーダ登録、ワークスペース管理、グラフィックスランタイム、診断ストアという六つの長寿命サービスを所有する。ビューは安定した識別子を借用し、ソースファイルやGPUオブジェクトを直接所有しない。

6.2 実装手順

起動時には設定を読み、インストールを検証し、カタログを構築または復元し、デバイスを作成し、ワークスペースファクトリを登録してイベントループへ入る。終了時にはジョブをキャンセルし、ワークスペースを閉じ、GPUリソースを解放し、設定を保存してからウィンドウ層を破棄する。

6.3 障害と回復

サービス障害は型付きエラーとして報告し、それ以前に作成したサービスを安全に破棄できる状態を保つ。インストール未設定、カタログ期限切れ、デコーダ拒否、一時的なデバイス利用不能でもシェル自体は操作可能でなければならない。

6.4 検証ゲート

受け入れフィクスチャは空の設定ディレクトリから起動し、アイドル状態のシェルへ到達し、フィクスチャ用インストールを接続し、一つのリソースワークスペースを開き、未完了ジョブや残存リソースなしで終了する。

6.5 累積 C++実装

C++クライアントマイルストーン。アプリケーションコンテキスト、順序付き起動・終了、型付きサービス状態、および個別サブシステムが失敗しても機能するシェルを実装する。

```
// Cumulative implementation listing
enum class ServiceState { Missing, Ready, Degraded, Failed };
struct ServiceStatus { ServiceState state; std::string message; };

class ExplorerApplication {
    Settings settings_;
    Diagnostics diagnostics_;
    InstallationService installations_;
    ResourceCatalog catalog_;
    DecoderRegistry decoders_;
    GraphicsRuntime graphics_;
    WorkspaceManager workspaces_;
public:
    int run() {
        settings_.load();
        installations_.configure(settings_, diagnostics_);
        catalog_.open(installations_, diagnostics_);
        graphics_.create(settings_.display(), diagnostics_);
        workspaces_.registerBuiltins(decoders_, graphics_);
        const int result = workspaces_.eventLoop();
        workspaces_.cancelAndCloseAll();
        graphics_.shutdown();
        settings_.save();
        return result;
    }
};
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第II部 バイナリリソースと描画基盤

第7章 デバイス、リソース、フレームライフサイクル

本章は resource lifecycle の両端、すなわち storage から取得した信頼できる byte と、rendering device を通じて送出される graphics resource を結ぶ。clean-room client は、DAT byte range から検証済みの変換と parse を経て device-owned texture、buffer、surface、draw へ進む間も evidence を保存しなければならない。

API 側では、Direct3D 8 は device を中心とする interface を通して描画を表現する。application は IDirect3D8::CreateDevice によって IDirect3DDevice8 を取得し、その device を用いて resource を作成し、state を設定し、draw call を発行し、完成した back buffer を present する。これは API reference としての文脈であり、特定のファイナルファンタジー XI client が任意の call sequence を使うことを証明するものではない。

API reference に関する注記。Microsoft の Direct3D 8 documentation では、IDirect3DDevice8 は DrawPrimitive に基づく描画、resource creation、device state、presentation に使う interface として示されている。

C++クライアント・マイルストーン。 安全な DAT lookup と binary input、evidence-scoped な復号・難読化解除・展開 dispatcher、graphics device、resource creation、presentation、recovery を実装する。

```
// Cumulative implementation excerpt: bounded input, scoped transforms, device lifecycle.
DecodedAsset loadDat(ResourceId id, GraphicsDevice& graphics, EvidenceLog& log) {
    const auto path = resourceTable().resolve(id);
    const std::vector<std::byte> file = readWholeFileBounded(path, 64_MiB);
    ChunkHeader header = parseChunkHeader(file, log);
    auto payload = checkedSubspan(file, 16, header.size - 16);
    std::vector<std::byte> clear = transformDispatcher().decode(
        header.type, header.version, payload, log);
    DecodedAsset asset = parseValidatedPayload(header, clear, log);
    asset.gpuResource = graphics.createResource(asset);
    return asset;
}

void renderFrame(GraphicsDevice& g, const Scene& scene) {
    if (!g.beginFrame()) { g.recover(); return; }
    g.draw(scene); g.endFrame(); g.present();
}
```

7.1 復号より先に、信頼できる byte view を作る

変換方法を提案する前に、client には信頼できる byte view が必要である。したがって data path の最初の program は、DAT resource を復号、展開、解釈しない。選択した file を binary mode で開き、明示した範囲を読み、各 byte を absolute source offset とともに表示する。この意図的に小さな tool が、後続するすべての parser と変換を検証する reference になる。

信頼できる byte view は四つの予備的な問いに答える。意図した file を開いたか。正確にどの byte range を読んだか。offset をどの基数で表したか。独立した byte-for-byte view でも同じ位置に同じ値が見えるか。これらへ再現可能に答えるまでは、parser の失敗と、file 選択の誤り、truncated read、表示の誤り、offset mismatch を区別できない。

証拠の境界。一致する hex dump が確認するのは byte acquisition と presentation である。それだけで header を特定したり、encryption を証明したり、key を明らかにしたり、読める tag の意味を確定したりすることはできない。

7.2 境界検査を持つ C++20 reader

Listing 6.1 は最初の簡潔な実装である。unsigned byte buffer を使い、要求範囲を file size に対して検査し、short read を成功と区別し、64 KiB の preview limit を明示する。formatter は input byte ごとに一つの表示位置を保ち、保守的な ASCII gutter を使う。multibyte text は byte alignment を見えにくくするため、character-set decoding は別の view に置く。

Listing 6.1 境界検査付き raw-byte acquisition と hexadecimal dump。

```
#include <algorithm>
#include <cctype>
#include <cstdint>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <vector>

using ByteBuffer = std::vector<std::uint8_t>;

ByteBuffer read_range(const std::filesystem::path& path,
                     std::uint64_t offset,
                     std::size_t requested) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open input file");
    }

    input.seekg(0, std::ios::end);
    const auto end = input.tellg();
```

```

    if (end < 0) {
        throw std::runtime_error("cannot determine file size");
    }

    const auto file_size = static_cast<std::uint64_t>(end);
    if (offset > file_size) {
        throw std::runtime_error("offset lies beyond end of file");
    }

    const auto available = file_size - offset;
    const auto count = static_cast<std::size_t>(
        std::min<std::uint64_t>(available, requested));
    ByteBuffer bytes(count);

    input.seekg(static_cast<std::streamoff>(offset), std::ios::beg);
    input.read(reinterpret_cast<char*>(bytes.data()),
        static_cast<std::streamsize>(bytes.size()));
    if (input.gcount() != static_cast<std::streamsize>(bytes.size())) {
        throw std::runtime_error("short or failed read");
    }
    return bytes;
}

void dump_hex(const ByteBuffer& bytes, std::uint64_t base_offset) {
    constexpr std::size_t width = 16;
    std::cout << "Offset(h) 00 01 02 03 04 05 06 07 "
        "08 09 0A 0B 0C 0D 0E 0F  ASCII\n";

    for (std::size_t row = 0; row < bytes.size(); row += width) {
        std::cout << std::hex << std::uppercase << std::setfill('0')
            << std::setw(8) << (base_offset + row) << " ";

        for (std::size_t column = 0; column < width; ++column) {
            if (row + column < bytes.size()) {
                std::cout << std::setw(2)
                    << static_cast<unsigned>(bytes[row + column])
                    << ' ';
            } else {
                std::cout << " ";
            }
            if (column == 7) std::cout << ' ';
        }

        std::cout << ' ';
        for (std::size_t column = 0;
            column < width && row + column < bytes.size(); ++column) {
            const unsigned char value = bytes[row + column];
            std::cout << (std::isprint(value)
                ? static_cast<char>(value) : '.');
        }
        std::cout << '\n';
    }
}

int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "usage: dat_bytes <file>\n";
        return 2;
    }

    try {
        constexpr std::size_t preview_size = 64 * 1024;
        const auto bytes = read_range(argv[1], 0, preview_size);
        std::cout << "Read " << std::dec << bytes.size()
            << " bytes (preview limit " << preview_size << ")\n";
        dump_hex(bytes, 0);
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 1;
    }
}

```

この初稿の reader は file path だけを受け取る。後の改訂では、明示的な starting offset と byte count を受け取り、完全な file size を記録し、file 全体または選択 range の hash を計算するべきである。これにより、filename、installation、client build が異なっても二つの観測を比較できる。

7.3 見た目ではなく byte を比較する

検証では absolute offset 上の値を比較する。file 先頭、多数の zero を含む領域、さまざまな値を含む領域、最後の partial row を含め、互いに離れた複数 range を選ぶとよい。custom reader と独立した reference は byte 単位で一致しなければならない。visible text column だけの比較では不十分である。character decoding が違ってても underlying byte は同じことがあるためである。

offset notation は明示する。decimal 表示の連続 row は 0、16、32、48 である。hexadecimal 表示では同じ row が 0x00、0x10、0x20、0x30 になる。どちらも正しいが、heading が hexadecimal と示す一方で row が decimal なら、存在しない不一致を作る。本書では、prose 中の hexadecimal offset には 0x prefix を、dump には Offset(h)を使う。

7.4 最初の観測から確立できること

調べた一つの DAT file の先頭には、ASCII gutter で syst および coll と見える printable byte sequence があり、その後に zero-filled region と強く反復する region が続く。確認済みの観測は、その file での byte value と location に限られる。sequence は candidate tag であり、確認済み type name ではない。zero は candidate padding または field であり、どちらの証明でもない。反復 pattern は structured numeric data、packed value、transformed data、または別の何かでありうる。

読める sequence、長い zero run、反復値が同居することから、見えている range 全体が一様に encrypted であるという model は、最初の仮説として弱い。これは推論であり、encrypted region が存在しないことの証明ではない。DAT container は plain header と transformed payload を混在させる可能性があり、resource family ごとに処理が異なる可能性もある。分析では、DAT extension を持つ全 file へ一つの性質を割り当てず、境界を定めた region を分類する。

7.5 変換する前に分類する

decryption という語は、定義された encryption algorithm と必要な key material を用いて暗号化を逆変換する場合に限るべきである。その前に、plain structured data、character encoding、compression、byte または bit permutation、XOR に似た obfuscation、platform 固有の numeric representation、texture tiling、corruption を検討する。これらは組み合わせられることがあり、順序も重要である。たとえば payload は deobfuscation 後に decompression され、周囲の record header は直接読めるかもしれない。

candidate transformation は、input byte range から output byte range への明示的な function として表す。evidence record には source file、source offset、input length、algorithm version、parameter または key material、output length、validation result を記す。original buffer は immutable のまま残す。これにより結果を再現し、競合する変換を比較し、証拠を失わずに誤った解釈を取り消せる。

7.6 提案した decode の validation gate

output が以前より random でなく見えるという理由だけで変換を採用してはならない。result は独立した invariant を満たすべきである。record size が available range 内に収まり、offset が valid boundary を指し、count が containing record を overflow せず、alignment が複数 sample で一貫し、known field が plausible value を取り、期待される reference が解決し、re-encoding または reversible operation の再適用で original byte を正確に再現することを確認する。

初期段階で最も強い証拠は、複数 file での反復性から得られる。同じ proposed header layout、size rule、flag behavior、transformation order が、特例なしに独立して選んだ多数の record を parse できれば確信度は上がる。一つの選別 sample だけで成功する rule は仮説のままである。unknown field と未説明 flag bit は、parser を成功して見せるために clear せず、verbatim に保存する。

暫定結論。raw reader は reproducible input、absolute offset、failure check を与えるため、decryption 研究の基礎になる。ただし reader 自体は decryptor ではない。次の implementation layer が byte を変換するのは、transformation type と validation contract を明示した後だけである。

7.6.1 retail client で確認された chunk step

retail client は、chunk offset +4 にある 32-bit information word を読み、7 bit 右 shift し、0x7FFFF で mask し、16 倍して次の chunk へ進む。したがって size field は 19 bit である。残りの bit には 7-bit type、is_shadow、is_extracted、3-bit version、is_virtual が保持される。この layout は 32 bit をすべて使用するため、size を 20 bit へ広げると is_shadow と重なる。

Listing 6.2 は complete checkpoint program である。指定した一つの file を取得し、確認済み information word だけを decode し、zero advance または range 外 advance を拒否し、すべての boundary を出力する。step で得た range を自動的に payload と呼ばず、共通 payload header size も仮定しない。

Listing 6.2 bounded DAT chunk walk を行う complete C++20 checkpoint.

```
#include <bit>
#include <cstdint>
#include <cstdint>
#include <cstring>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <span>
#include <stdexcept>
#include <vector>

using Bytes = std::vector<std::byte>;

std::uint32_t read_u32_le(std::span<const std::byte> bytes,
                        std::size_t offset) {
    if (offset > bytes.size() || bytes.size() - offset < 4)
        throw std::runtime_error("truncated 32-bit value");
    std::uint32_t value{};
    std::memcpy(&value, bytes.data() + offset, sizeof value);
    if constexpr (std::endian::native == std::endian::big) {
        value = ((value & 0x000000ffu) << 24) |
                ((value & 0x0000ff00u) << 8) |
                ((value & 0x00ff0000u) >> 8) |
                ((value & 0xff000000u) >> 24);
    }
    return value;
}

struct ChunkInfo {
    std::uint8_t type;
    std::uint32_t next_blocks;
    bool is_shadow;
    bool is_extracted;
    std::uint8_t version;
    bool is_virtual;
};

ChunkInfo decode_info(std::uint32_t word) {
    return {
        static_cast<std::uint8_t>(word & 0x7fu),
        (word >> 7) & 0x7ffffu,
        ((word >> 26) & 1u) != 0,
        ((word >> 27) & 1u) != 0,
        static_cast<std::uint8_t>((word >> 28) & 0x7u),
    };
}
```



```

        ((word >> 31) & 1u) != 0
    };
}

Bytes read_file(const std::filesystem::path& path,
               std::uintmax_t maximum = 256u * 1024u * 1024u) {
    const auto size = std::filesystem::file_size(path);
    if (size > maximum) throw std::runtime_error("file exceeds safety limit");
    Bytes bytes(static_cast<std::size_t>(size));
    std::ifstream input(path, std::ios::binary);
    if (!input) throw std::runtime_error("cannot open input file");
    input.read(reinterpret_cast<char*>(bytes.data()),
              static_cast<std::streamsize>(bytes.size()));
    if (!input) throw std::runtime_error("short read");
    return bytes;
}

void walk_chunks(std::span<const std::byte> bytes) {
    std::size_t offset = 0;
    while (offset < bytes.size()) {
        if (bytes.size() - offset < 8)
            throw std::runtime_error("truncated chunk prefix");

        const std::uint32_t word = read_u32_le(bytes, offset + 4);
        const ChunkInfo info = decode_info(word);
        const std::size_t next = static_cast<std::size_t>(info.next_blocks) << 4;
        if (next == 0) throw std::runtime_error("zero-length chunk");
        if (next > bytes.size() - offset)
            throw std::runtime_error("chunk extends beyond file");

        std::cout << "offset=0x" << std::hex << offset
                  << " type=0x" << unsigned(info.type)
                  << " bytes=0x" << next
                  << " shadow=" << info.is_shadow
                  << " extracted=" << info.is_extracted
                  << " version=" << std::dec << unsigned(info.version)
                  << " virtual=" << info.is_virtual << '\n';
        offset += next;
    }
}

int main(int argc, char** argv) {
    try {
        if (argc != 2) {
            std::cerr << "usage: dat_chunks <input.dat>\n";
            return 2;
        }
        const Bytes bytes = read_file(argv[1]);
        walk_chunks(bytes);
        return 0;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 1;
    }
}

```

checkpoint test. 最後の chunk を含む複数の出力 offset を、独立した byte view と比較する。すべての step が 16-byte alignment を満たし、file 内に収まり、選択 range の終端へ正確に到達した場合だけ合格とする。失敗した場合、その resource は offset zero からこの walk を用いない、file が truncated である、または外側 container の別 rule を先に確立する必要がある。

7.6.2 推測できない transformation registry

累積 client は、transformation を registry 経由で実行する。初期 registry が対応するのは identity operation だけである。decryption、deobfuscation、decompression、layout transform は、その input range、parameter、output invariant、および round-trip または cross-file validation を文書化してから登録する。未知 identifier は UnsupportedTransform となり、evidence log に残る。

Listing 6.3 明示的な unsupported path を持つ transformation dispatch。

```
enum class TransformId : std::uint32_t { Plain = 0 };

class UnsupportedTransform : public std::runtime_error {
public:
    explicit UnsupportedTransform(std::uint32_t id)
        : std::runtime_error("unsupported payload transform " +
                             std::to_string(id)) {}
};

using TransformFunction = std::vector<std::byte> (*)(
    std::span<const std::byte> source,
    std::span<const std::byte> parameters);

class TransformRegistry {
    std::unordered_map<std::uint32_t, TransformFunction> functions_;
public:
    TransformRegistry() {
        functions_.emplace(static_cast<std::uint32_t>(TransformId::Plain),
            [](std::span<const std::byte> source,
               std::span<const std::byte>) {
                return std::vector<std::byte>(source.begin(), source.end());
            });
    }

    void register_verified(std::uint32_t id, TransformFunction function) {
        if (!function || functions_.contains(id))
            throw std::logic_error("invalid transform registration");
        functions_.emplace(id, function);
    }

    std::vector<std::byte> decode(
        std::uint32_t id, std::span<const std::byte> source,
        std::span<const std::byte> parameters = {}) const {
        const auto found = functions_.find(id);
        if (found == functions_.end()) throw UnsupportedTransform(id);
        return found->second(source, parameters);
    }
};
```

7.7 描画文脈としての device

device は、application がグラフィックスシステムに処理を記述する地点である。render target、depth-stencil surface、swap-chain behavior、capability information、draw に使われる active state を保持または公開する。device は scene そのものではない。application が resource を bind し、device を通じて geometry を送出して初めて scene data が可視になる。

調査では、device activity が順序付きの記録を与えるため有用である。resource creation、state change、texture binding、stream selection、draw call を timeline に置ける。ただし、call name だけでは terrain、character、effect、interface のどれに属するかは分からないため、scene context も必要である。

7.8 resource と所有関係

Direct3D 8 の resource には texture、vertex buffer、index buffer、surface、render target、depth-stencil surface がある。device は CreateTexture、CreateVertexBuffer、CreateIndexBuffer、CreateRenderTarget、CreateDepthStencilSurface のような作成 method を提供する。resource description には dimension、format、usage、memory pool などの property が記録される。

resource はグラフィックス上の役割を持つ data であり、必ずしも一つの asset ではない。一つの texture は複数の material で再利用でき、一つの vertex buffer は複数の draw range を含められ、surface は目に見える画像ではなく中間 destination になりうる。trace を読むときは、視覚的な名前を与える前に resource identifier と binding history を記録する。

7.9 慣例的な Direct3D 8 フレーム

慣例的な API-level frame では、resource を準備または更新し、必要なら target を clear し、BeginScene を呼び、state と binding を設定し、DrawPrimitive または DrawIndexedPrimitive を発行し、EndScene を呼び、最後に Present を呼ぶ。BeginScene と EndScene は API で scene boundary を定め、Present は device が所有する次の back buffer を presentation path へ渡す。

この順序は有用な reference model であって、すべての internal pass についての記述ではない。engine は scene boundary の前に resource 処理を行い、複数の target を使い、state を再利用し、一枚の画像からは分からない形で処理を遅延させることがある。client と build における実際の順序を確立できるのは観測された trace だけである。

7.9.1 complete graphics checkpoint

Listing 6.4 は、最初の complete rendering executable である。native window を作成し、Direct3D 8 interface と device を取得し、back buffer を clear し、BeginScene と EndScene で frame を囲み、present し、cooperative-level の reset path を処理する。asset についての仮定は含まない。

この listing を C++20 として compile し、Direct3D 8 entry point を実装する platform import library と link する。操作の toolchain 固有の書式は固定しない。合格条件は、960×540 の応答可能な window に濃い青色の client area が表示されること、および window を閉じたとき device と API interface が解放されることである。

Listing 6.4 window、device、frame、presentation、recovery の complete checkpoint.

```
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <d3d8.h>
#include <stdexcept>

LRESULT CALLBACK window_proc(HWND window, UINT message,
                              WPARAM wParam, LPARAM lParam) {
    if (message == WM_DESTROY) {
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProcW(window, message, wParam, lParam);
}

struct GraphicsCheckpoint {
    HWND window{};
    IDirect3D8* api{};
    IDirect3DDevice8* device{};
    D3DPRESENT_PARAMETERS presentation{};

    ~GraphicsCheckpoint() {
        if (device) device->Release();
        if (api) api->Release();
        if (window) DestroyWindow(window);
    }

    void create(HINSTANCE instance) {
        WNDCLASSW wc{};
        wc.hInstance = instance;
        wc.lpfnWndProc = window_proc;
        wc.lpszClassName = L"RenderingVanadielCheckpoint";
        wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
        if (!RegisterClassW(&wc) && GetLastError() != ERROR_CLASS_ALREADY_EXISTS)
            throw std::runtime_error("RegisterClassW failed");

        window = CreateWindowExW(0, wc.lpszClassName,
            L"Rendering Vana'diel - graphics checkpoint",
            WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,
            960, 540, nullptr, nullptr, instance, nullptr);
        if (!window) throw std::runtime_error("CreateWindowExW failed");

        api = Direct3DCreate8(D3D_SDK_VERSION);
        if (!api) throw std::runtime_error("Direct3DCreate8 failed");

        presentation.Windowed = TRUE;
        presentation.SwapEffect = D3DSWAPEFFECT_DISCARD;
        presentation.BackBufferFormat = D3DFMT_UNKNOWN;
        presentation.EnableAutoDepthStencil = TRUE;
        presentation.AutoDepthStencilFormat = D3DFMT_D16;
        presentation.hDeviceWindow = window;

        HRESULT result = api->CreateDevice(D3DADAPTER_DEFAULT,
            D3DDEVTYPE_HAL, window, D3DCREATE_HARDWARE_VERTEXPROCESSING,
            &presentation, &device);
        if (FAILED(result)) {
            result = api->CreateDevice(D3DADAPTER_DEFAULT, D3DDEVTYPE_HAL,
                window, D3DCREATE_SOFTWARE_VERTEXPROCESSING,
                &presentation, &device);
        }
    }
};
```

```

    }
    if (FAILED(result)) throw std::runtime_error("CreateDevice failed");
    ShowWindow(window, SW_SHOW);
}

bool recover_if_needed() {
    const HRESULT state = device->TestCooperativeLevel();
    if (state == D3DERR_DEVICELOST) return false;
    if (state == D3DERR_DEVICENOTRESET)
        return SUCCEEDED(device->Reset(&presentation));
    return SUCCEEDED(state);
}

void render() {
    if (!recover_if_needed()) return;
    device->Clear(0, nullptr, D3DCLEAR_TARGET | D3DCLEAR_ZBUFFER,
        D3DCOLOR_XRGB(24, 40, 64), 1.0f, 0);
    if (SUCCEEDED(device->BeginScene())) device->EndScene();
    device->Present(nullptr, nullptr, nullptr, nullptr);
}

};

int main() {
    try {
        GraphicsCheckpoint app;
        app.create(GetModuleHandleW(nullptr));
        MSG message{};
        while (message.message != WM_QUIT) {
            if (PeekMessageW(&message, nullptr, 0, 0, PM_REMOVE)) {
                TranslateMessage(&message);
                DispatchMessageW(&message);
            } else {
                app.render();
            }
        }
        return static_cast<int>(message.wParam);
    } catch (...) {
        return 1;
    }
}

```

7.10 lifetime、update、reuse

resource の lifetime は多くの frame にまたがりうる。static geometry や頻繁に使う texture は一度作成され、繰り返し bind されることがある。dynamic data は animation や effect の変化に合わせて lock、update、unlock されうる。この違いは重要である。重そうに見える視覚結果が、現在の frame で重い処理を行うのではなく、前もって準備した data に依存している場合がある。

証拠台帳では creation、update、binding、draw use を分けるべきである。これにより、texture の creation を調査中 scene で sample された証拠とみなしたり、buffer update をその全内容が render された証拠とみなしたりする誤りを防げる。

7.11 device trace に記録すること

少なくとも、判明している device creation parameter、target と depth surface、resource descriptor、state-setting call、texture と stream binding、draw call とその count、scene

boundary、presentation call を記録する。各 event を第 2 章で定めた capture condition へ結び付ける。そうすれば後の章は、もっともらしいが根拠のない順序を語るのではなく、証拠から pass を再構成できる。

ゲーム固有の境界。Direct3D 8 API は device を通じて何を表現できるかを定めるが、client 固有の証拠なしにファイナルファンタジー XI がどの選択をしたかは示さない。

7.6.3 VTABLE と FTABLE による file identifier の解決

client 側の resource identifier は filesystem path ではない。installation table は、まず ROM volume を選び、directory と filename を一つの 16-bit 値に encode する。directory = packed / 0x80、file = packed % 0x80 である。Listing 6.5 は installed volume を順に調べ、table entry 欠落、truncated table、target file 欠落を別々の結果として扱う。

Listing 6.5 toolchain 非依存の resource-ID 解決。

```
struct ResolvedResource {
    ResourceId id;
    std::uint8_t volume;
    std::uint16_t packedLocation;
    std::filesystem::path path;
};

std::optional<ResolvedResource> resolveResourceId(
    const std::filesystem::path& root, ResourceId id) {
    for (std::uint8_t volume = 1; volume < 20; ++volume) {
        const std::string suffix = volume == 1 ? "" : std::to_string(volume);
        const auto tableRoot = volume == 1 ? root : root / ("ROM" + suffix);
        const auto vtable = tableRoot / ("VTABLE" + suffix + ".DAT");
        const auto ftable = tableRoot / ("FTABLE" + suffix + ".DAT");
        if (!exists(vtable) || !exists(ftable)) continue;

        const auto selectedVolume = readU8At(vtable, id.value);
        if (!selectedVolume || *selectedVolume != volume) continue;
        const auto packed = readU16LEAt(ftable, checkedMul(id.value, 2));
        if (!packed) throw ParseError("truncated FTABLE entry");

        const auto directory = *packed / 0x80u;
        const auto file = *packed % 0x80u;
        const auto rom = volume == 1 ? "ROM" : "ROM" + suffix;
        const auto path = root / rom / std::to_string(directory) /
            (std::to_string(file) + ".DAT");
        if (!exists(path)) throw ParseError("table entry names a missing DAT");
        return ResolvedResource{id, volume, *packed, path};
    }
    return std::nullopt;
}
```

7.6.4 decode の前に recognize する

common walker は、すべての type 値を decoder へ渡してはならない。handler は、bounded record が supported、structurally invalid、または現在の client では unknown かを先に判定する。unknown

record は回復可能なまま残し、invalid record は該当 resource を停止し、supported record だけを decode する。

Listing 6.6 recognition と decoding を分離した registry。

```
enum class Recognition { Invalid, Unsupported, Supported };

struct ChunkView {
    std::size_t offset;
    ChunkInfo info;
    std::span<const std::byte> raw;
    std::span<const std::byte> data;
};

struct ChunkHandler {
    std::uint8_t type;
    Recognition (*recognize)(const ChunkView&, EvidenceLog&);
    AssetResult (*decode)(const ChunkView&, EvidenceLog&);
};

class ChunkRegistry {
    std::unordered_map<std::uint8_t, ChunkHandler> handlers_;
public:
    void add(ChunkHandler handler) {
        if (!handlers_.emplace(handler.type, handler).second)
            throw std::logic_error("duplicate chunk handler");
    }

    Recognition inspect(const ChunkView& chunk, EvidenceLog& log) const {
        const auto found = handlers_.find(chunk.info.type);
        if (found == handlers_.end()) return Recognition::Unsupported;
        return found->second.recognize(chunk, log);
    }

    AssetResult decode(const ChunkView& chunk, EvidenceLog& log) const {
        const auto found = handlers_.find(chunk.info.type);
        if (found == handlers_.end()) return UnsupportedChunk{chunk.info.type};
        if (found->second.recognize(chunk, log) != Recognition::Supported)
            return FailedChunk{chunk.offset, "invalid supported record"};
        return found->second.decode(chunk, log);
    }
};
```

7.6.5 検証済み transformation とその境界

次の object-map transformation は、16-byte outer header の後にある chunk data へ作用する。0x1B 未満の version は変更しない。後期 version では rolling key で bounded run を条件付き反転し、その後各 object identifier の 16 byte を XOR する。256-byte key table は algorithm に隠さず、別 version の evidence data として扱い、検証済み retail build と hash を記録する。

confidence boundary。object-map algorithm と version gate は調査した map family について confirmed である。同じ container を用いるという理由だけで texture、skeleton、geometry、animation へ適用するのは speculation である。

Listing 6.7 key provenance を明示した bounded object-map transformation。

```
// The key table is evidence data, not an algorithmic constant invented here.
using KeyTable = std::span<const std::uint8_t, 256>;
```

```

void decodeObjectMap(std::span<std::byte> data, KeyTable keyTable) {
    if (data.size() < 8 || u8(data[3]) < 0x1b) return;
    const std::size_t declared = readU24LE(data, 0);
    const std::size_t limit = std::min(declared, data.size());
    std::uint32_t key = keyTable[u8(data[7]) ^ 0xffu];
    std::uint32_t counter = 0;

    for (std::size_t pos = 8; pos < limit; ) {
        const std::size_t run = ((key >> 4) & 7u) + 16u;
        if ((key & 1u) && run < limit - pos)
            for (std::size_t i = 0; i < run; ++i) data[pos + i] ^= std::byte{0xff};
        key += ++counter;
        pos = checkedAdd(pos, run);
    }

    constexpr std::size_t header = 32, nodeStride = 92, nameBytes = 16;
    const std::size_t declaredNodes = readU24LE(data, 4);
    const std::size_t availableNodes = data.size() > header
        ? (data.size() - header) / nodeStride : 0;
    for (std::size_t n = 0; n < std::min(declaredNodes, availableNodes); ++n)
        for (std::size_t i = 0; i < nameBytes; ++i)
            data[header + n * nodeStride + i] ^= std::byte{0x55};
}

```

第8章 インストール検出とリソースマップ

リソースツールは単一の固定パスを前提にせずデータを発見しなければならない。本章では、明示的なインストールルート、ROM ボリューム検出、版の識別、そしてカタログやビューアがファイルを開く前の読み取り専用検証を定義する。

実装成果。

安全なインストール検出を実装し、単一の真偽値ではなく完全な検証レポートを返す。

8.1 責務と所有

InstallationRecord は、選択されたルート、検出したボリューム、テーブルパス、識別証拠、検証上の問題を保持する。パスはデータであり、プロセス全体の定数ではない。

8.2 実装手順

利用者がルートを選択または確認する。検出処理は期待されるボリューム名だけを列挙し、正規化パスを求め、すべてがルート配下にあることを確認し、必須テーブルを読み取り専用で開き、任意ボリュームは別に記録する。

8.3 障害と回復

任意ボリュームの欠落は対応範囲を狭めるだけだが、主要テーブルの欠落は識別子解決を禁止する。いずれもシェルを停止させてはならない。シンボリックリンク、パストラバーサル、重複ボリューム、読み取り不能ファイルは明示的に報告する。

8.4 検証ゲート

テストでは最小構成、任意ボリュームの欠落、大文字小文字だけが異なる重複、ルート外へ出るパス、レコード幅と整合しない長さのテーブルを扱う。

8.5 累積 C++実装

C++クライアントマイルストーン。安全なインストール検出を実装し、単一の真偽値ではなく完全な検証レポートを返す。

```
// Cumulative implementation listing
InstallationRecord discoverInstallation(const std::filesystem::path& root) {
    InstallationRecord out{canonicalChecked(root), {}, {}, {}};
    for (unsigned i = 0; i <= 19; ++i) {
        const std::string name = i == 0 ? "ROM" : "ROM" + std::to_string(i);
        const auto candidate = out.root / name;
        if (!std::filesystem::exists(candidate)) continue;
        const auto volume = canonicalChecked(candidate);
        if (!isDescendant(out.root, volume)) {
            out.issues.push_back({Severity::Error, candidate, "volume escapes root"});
            continue;
        }
        out.volumes.push_back({i, volume});
    }
    out.vtable = validateTable(out.root / "VTABLE.DAT", 2, out.issues);
    out.ftable = validateTable(out.root / "FTABLE.DAT", 2, out.issues);
    return out;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第9章 リソースカタログとブラウザ

カタログは、多数の匿名パスと数値識別子を検索可能で再現可能な一覧へ変換する。すべてのリソース種別が既知であるとは仮定せず、分類結果には信頼度とデコーダ対応状況を別々に保持する。

実装成果。

増分更新可能な検索カタログと、フィルタや再走査を越えて選択を保持するブラウザモデルを実装する。

9.1 責務と所有

各 `CatalogEntry` は、リソース ID、解決済みボリュームとパス、バイト長、ハッシュ、認識候補、対応状態、走査診断を保持する。UI 行はエントリ ID を参照し、再走査で生ポインタ選択が無効にならないようにする。

9.2 実装手順

走査は識別子を解決し、認識に必要な境界付き先頭部だけを読み、より深いメタデータ抽出は表示中または明示要求されたエントリに限って予約する。フィルタは識別子範囲、ボリューム、サイズ、認識結果、対応状況、文字列検索を組み合わせる。

9.3 障害と回復

読み取り不能ファイルもエラー付きカタログ項目として残す。キャンセルされた走査は完了バッチだけを確定し、カタログを部分状態として記録し、初期化途中の項目を公開しない。

9.4 検証ゲート

決定論的フィクスチャカタログは、ワーカ数に関係なく同じソート済み識別子、ハッシュ、認識候補、診断件数を生成しなければならない。

9.5 累積 C++実装

C++クライアントマイルストーン。増分更新可能な検索カタログと、フィルタや再走査を越えて選択を保持するブラウザモデルを実装する。

```
// Cumulative implementation listing
void ResourceCatalog::scan(const InstallationRecord& install,
                           StopToken stop, CatalogSink& sink) {
    std::vector<CatalogEntry> batch;
    for (ResourceId id : enumerateResourceIds(install)) {
        if (stop.stopRequested()) break;
        CatalogEntry e;
        e.id = id;
        e.path = resolveResourceId(install, id);
        try {
            PrefixFile file(e.path, 4096);
            e.byteLength = file.length();
            e.prefixHash = hash(file.prefix());
            e.candidates = recognizers_.classify(file.prefix());
        } catch (const std::exception& ex) {
            e.issues.push_back({Severity::Error, ex.what()});
        }
        batch.push_back(std::move(e));
        if (batch.size() == 256) { sink.commit(batch); batch.clear(); }
    }
    if (!batch.empty()) sink.commit(batch);
}
```

```
    sink.finish(stop.stopRequested() ? ScanState::Partial : ScanState::Complete);  
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 10 章 バイナリインスペクタ、チャンクツリー、フィールド由来情報

バイナリインスペクタは 16 進ダンプと型付きデコーダの橋渡しである。生データと変換後データ、チャンク境界、解釈済みフィールド、フラグ、未知ビット、各値を支える正確なソース範囲を表示する。

実装成果。

後続すべてのデコーダが共有する、生データ／変換後バイナリインスペクタ、検証済みチャンクツリー、ソース範囲ナビゲーションを実装する。

10.1 責務と所有

InspectorNode は安定パス、ラベル、ソース範囲、任意の変換後範囲、型付き値、信頼度、子ノードを持つ。生バイトペインとツリー選択は UI オブジェクトのアドレスではなく範囲を介して連携する。

10.2 実装手順

リソースを開くと、境界付きページをマップまたは読み込み、認識を実行し、検証済みチャンクだけを歩き、対応デコーダにフィールドノードを追加させる。ノード選択は対応バイトを強調し、バイト選択はそれを含む最小ノードを探す。

10.3 障害と回復

不正サイズは終端エラーノードを生成し、該当枝だけを停止する。未知レコードは不透明な子ノードとして残す。変換ビューは元バイトを上書きしたり隠したりしてはならない。

10.4 検証ゲート

テストでは双方向の範囲選択、切り詰め時の挙動、未知ビット保持、無関係な兄弟追加後も安定するノードパスを検証する。

10.5 累積 C++実装

C++クライアントマイルストーン。後続すべてのデコーダが共有する、生データ／変換後バイナリインスペクタ、検証済みチャンクツリー、ソース範囲ナビゲーションを実装する。

```
// Cumulative implementation listing
InspectorNode inspectChunks(ByteView raw, const DecoderRegistry& decoders) {
    InspectorNode root{"/", "resource", {0, raw.size()}, {}, {}, Confidence::Observed};
    std::size_t offset = 0;
    while (offset < raw.size()) {
        const ChunkHeader h = readChunkHeader(raw, offset);
        if (!h.valid || h.byteSize == 0 || h.byteSize > raw.size() - offset) {
            root.children.push_back(errorNode(offset, raw.size() - offset,
                                              "invalid chunk boundary"));
            break;
        }
        InspectorNode node = chunkNode(h, offset);
        if (const Decoder* d = decoders.find(h.type, raw.subview(offset, h.byteSize)))
            d->describe(raw.subview(offset, h.byteSize), node);
        else
            node.children.push_back(opaqueNode(offset + h.headerSize,
                                              h.byteSize - h.headerSize));
        root.children.push_back(std::move(node));
        offset += h.byteSize;
    }
    return root;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 11 章 変換、座標空間、カメラ

レンダラは object を一つの普遍的な座標系で描くわけではない。mesh を local に作成し、world へ配置し、camera から見て、screen へ投影し、最終的に pixel へ対応させるために、data を名前の付いた space へ移動させる。この space は、単純な位置の誤りに見える問題を正確に問うための言葉を与える。

範囲。本章は標準的な transform の用語を説明する。ファイナルファンタジー XI の正確な matrix convention や camera implementation を確定するものではない。

C++クライアント・マイルストーン。 境界検査と endian 対応を持つ reader、typed numeric decoder、vector、matrix、transform、projection、検査可能 camera を作る。

```
// Cumulative implementation excerpt: bounded little-endian reader and inspectable camera.
class ByteReader {
    std::span<const std::byte> bytes_; std::size_t pos_ = 0;
public:
    explicit ByteReader(std::span<const std::byte> b) : bytes_(b) {}
    template<class T> T little() {
        static_assert(std::is_trivially_copyable_v<T>);
        if (pos_ + sizeof(T) > bytes_.size()) throw ParseError("read past end");
        T v; std::memcpy(&v, bytes_.data() + pos_, sizeof(T)); pos_ += sizeof(T);
    }
}
```

```

        if constexpr (std::endian::native == std::endian::big) v = byteSwap(v);
        return v;
    }
};

struct Camera {
    Vec3 eye, target, up{0, 1, 0}; float fovY, aspect, nearZ, farZ;
    Mat4 view() const { return lookAtLH(eye, target, up); }
    Mat4 projection() const { return perspectiveFovLH(fovY, aspect, nearZ, farZ); }
};

```

11.1 local、world、view、projection space

local space は geometry を自身の origin に対して記述する。world transform は geometry を scene に置く。view transform は scene を camera に対して表す。projection transform は camera-relative view を perspective division と rasterization に適した形へ変換する。この積は world-view-projection transform と呼ばれることが多いが、engine は計算を複数の方法で組織できる。

これらの space を分けることは重要である。camera が固定でも、回転する character は local-to-world transform を変えうる。camera pan は object position を変えずに view transform を変える。field of view の変更は projection に影響し、world coordinate が変わらなくても遠方 object が動いたり拡大縮小したりして見える。

11.2 camera は視点だけでなく transform である

camera は orientation と position を定めるが、どの scene relationship が見えるかも定める。near clip plane と far clip plane は描画対象となる depth range を制限する。projection は perspective behavior と aspect relationship を定める。これらは見かけの scale、screen-space motion、object が view volume から出る地点に影響する。

capture を分析するときは、判明している範囲で camera position、direction、subject までの distance、field-of-view condition を記録する。camera と projection の効果を除外するまで、見かけの大きさの変化を geometry LOD や texture replacement の証拠として扱ってはならない。

11.3 hierarchy と animation

character や関節を持つ object には通常、複数の transform が必要である。bone または node は parent の transform を継承し、自身の local transform を加える。この hierarchy を評価すると、rigid

part を置いたり skinned vertex を変形したりする最終 pose が得られる。equipment は同じ geometry を共有せずに同じ hierarchy に従える。

したがって見える motion には複数の源がある。object placement、node animation、skeletal deformation、vertex processing、texture-coordinate motion、camera motion である。信頼できる説明は、どの space が変わるかと、代替説明を区別する証拠を示す。

11.4 texture coordinate と非幾何学的な motion

texture coordinate は vertex に関連付けられた値で、renderer に画像の sample 方法を伝える。object position とは独立に transform できる。texture coordinate を scroll、rotate、その他の方法で変えると、geometry が静的なまま surface 上に motion を作れる。water、energy、装飾 material で一般的な分析候補であるが、effect ごとに証明が必要である。

11.5 transform の checklist

調査する各 draw について問う。local origin は何か。world に何が置くか。どの camera view が適用されるか。どの projection が適用されるか。hierarchy は関わるか。texture coordinate には独自の transform があるか。これらの答えは、『何かが動く』という曖昧な主張を、何が変化するかという検証可能な説明へ変える。

証拠基準。matrix 名、value、API call は直接の証拠である。スクリーンショットで見える rotation や scroll は、複数の transform による説明を持ちうる観測である。

第 12 章 頂点データ、頂点宣言、インデックスバッファ

geometry は構造化された vertex data として rendering device へ届き、必要に応じて index buffer と組み合わされる。vertex は position だけではない。normal、diffuse color、texture coordinate、blend information、active な rendering path が必要とするその他の値を持てる。declaration または flexible vertex format は、その layout の読み方を device に伝える。

範囲。本章は Direct3D 時代の geometry 概念を説明する。直接の証拠なしに、ファイナルファンタジー XI の model が使う layout を特定するものではない。

C++クライアント・マイルストーン。 検証済み geometry record を typed header と flag set へ parse し、vertex layout/buffer と index buffer wrapper を作る。

```
// Cumulative implementation excerpt: validated geometry and owned D3D8 buffers.
Geometry parseGeometry(ByteReader& r, std::size_t recordEnd) {
    Geometry g;
    g.vertexCount = r.little<std::uint32_t>();
    g.indexCount = r.little<std::uint32_t>();
    g.layout = decodeVertexLayout(r.little<std::uint32_t>());
    const auto vertexBytes = checkedMultiply(g.vertexCount, g.layout.stride);
    g.vertices = r.bytesWithin(vertexBytes, recordEnd);
    g.indices = r.arrayWithin<std::uint16_t>(g.indexCount, recordEnd);
    validateIndices(g.indices, g.vertexCount);
    return g;
}

GpuGeometry upload(IDirect3DDevice8& d, const Geometry& g) {
    return {createVertexBuffer(d, g.vertices, g.layout.fvf),
            createIndexBuffer(d, g.indices), g.vertexCount, g.indexCount};
}
```

12.1 vertex stream と layout

vertex buffer は vertex record の列を格納する。stride は一つの record から次までの byte 数を示す。layout は既知の offset にある field へ意味を与える。position、normal、color、一つ以上の texture-coordinate set、プログラマブル vertex path が使う data などである。Direct3D 8 では flexible vertex format が一般的な固定機能 layout を記述し、vertex shader はより明示的な declaration mechanism を提供する。

同じ mesh position でも、layout や state が異なれば違う結果を生む。normal は lighting を可能にし、vertex color は material を tint し、二番目の texture-coordinate set は追加の texture stage を支える。buffer address だけを示す trace は、stride、declaration、draw range が分かるまで不完全である。

12.2 index buffer と reuse

index buffer は vertex を番号で参照し、stream 内で共有 vertex を重複させずに triangle を再利用できる。これは storage efficiency を高め、接続された mesh を明確に表現できる。index count、primitive type、base または start value は、draw がどの geometry subset を使うかを定める。

indexing だけでは object boundary は分からない。一つの buffer は複数の object や material group を含め、一つの object は複数の index range で draw されうる。したがって証拠では、buffer を一つの visible model として扱わず、draw を特定の range へ関連付ける。

12.3 primitive topology

device は vertex と index を point、line、triangle list、triangle strip などの primitive として解釈する。topology は draw が消費する vertex の順序と数に影響するが、それだけで visual complexity を定めない。短い draw が大きな screen area を覆うことも、大きな draw がほぼ隠れたり cull されたりすることもある。

12.4 animation と material のための data

character rendering には position、normal、texture coordinate、blend weight、blend index、その他の per-vertex attribute が必要になりうる。environment rendering はより単純な layout を使うかもしれない。vertex color や複数の coordinate set を加えるかもしれない。これらは一般的な可能性であって、本作の scene class を分類するものではない。data layout は名前を付ける前に観測または復元しなければならない。

12.5 geometry evidence を読む

有用な geometry record には、buffer size と pool、stride、layout または FVF、index format、primitive type、draw offset、count、draw 時に active な transform と texture を含める。visible result を特定する capture と組にする。これにより、共有 geometry に対する material pass と、本当に別の mesh を区別できる。

証拠基準。vertex semantic と draw range は、declaration、data inspection、trace evidence で確認する。object の見かけの形だけでは確認できない。

12.6 最初の supported geometry slice

最初の実装は既知 format 全体より意図的に狭くする。Listing 8.1 は unskinned geometry だけを受け入れ、16-bit 単位の offset と size を検証し、position と optional normal を読み、material selection と triangle-list command を処理する。weighted vertex、mirrored pass、strip、draw-state record、unknown opcode は推測せず unsupported result にする。

Listing 8.1 unskinned triangle-list geometry の evidence-scoped decode。

```
struct SupportedGeo {
    SourceRef source;
    std::string material;
    std::vector<VertexPNUV> vertices;
```



```

        std::vector<std::uint32_t> indices;
    };

SupportedGeo decodeUnskinnedGeo(SourceRef source, std::span<const std::byte> data) {
    if (data.size() < 80) throw ParseError("short geometry header");
    const std::uint16_t flags = u16le(data, 2);
    const auto drawOffset = checkedMul<std::size_t>(u32le(data, 8), 2);
    const auto drawBytes = checkedMul<std::size_t>(u16le(data, 12), 2);
    const auto weightedCounts = u32le(data, 24);
    const auto weightOffset = u32le(data, 32);
    const auto vertexOffset = checkedMul<std::size_t>(u32le(data, 40), 2);
    const auto vertexBytes = checkedMul<std::size_t>(u16le(data, 44), 2);
    if (weightedCounts || weightOffset)
        throw UnsupportedRecord("first geometry slice is unskinned only");
    requireRange(data, drawOffset, drawBytes);
    requireRange(data, vertexOffset, vertexBytes);

    const bool withoutNormals = (flags & 0x007fu) != 0;
    const std::size_t stride = withoutNormals ? 12 : 24;
    if (vertexBytes % stride) throw ParseError("non-integral vertex stream");

    SupportedGeo out{source};
    for (std::size_t p = 0; p < vertexBytes; p += stride) {
        VertexPNUV v{};
        v.position = readVec3(data, vertexOffset + p);
        v.normal = withoutNormals ? Vec3{0, 1, 0}
            : readVec3(data, vertexOffset + p + 12);
        out.vertices.push_back(v);
    }

    CommandReader commands(data.subspan(drawOffset, drawBytes));
    while (!commands.empty()) {
        switch (const std::uint16_t op = commands.u16()) {
            case 0x8000: out.material = commands.fixedString(16); break;
            case 0x0054: {
                const auto triangleCount = commands.u16();
                for (std::uint16_t t = 0; t < triangleCount; ++t) {
                    const auto corner = commands.triangleRecord(); // 3 indices + 3 UVs
                    for (int i = 0; i < 3; ++i) {
                        if (corner.index[i] >= out.vertices.size())
                            throw ParseError("triangle index outside vertex stream");
                        out.vertices[corner.index[i]].uv = corner.uv[i];
                        out.indices.push_back(corner.index[i]);
                    }
                }
                break;
            }
            case 0xffff: return out;
            default: throw UnsupportedRecord("geometry draw opcode", op);
        }
    }
    throw ParseError("geometry stream lacks terminator");
}

```

acceptance test. 最大 index は decoded vertex count 未満であり、全 command は declared draw stream 内にあり、end opcode が存在し、記録した source range を再読すると同じ position、normal、index、UV、material name になること。

第 13 章 テクスチャ、サーフェス、パレット、ミップマップ

texture は material へ sampled image data を与える。surface は texture level、render target、depth buffer、その他の device resource として使える二次元 storage を提供する。visible role は file name や dimension だけではなく、どのように bind され使用されるかで決まる。

範囲。Direct3D 8 は texture、surface、format、pool、mip level を支える。本章は、特定のファイナルファンタジー XI resource がどの format や palette path を使うかを主張しない。

C++クライアント・マイルストーン。 検証済み texture record を surface、palette、mip level へ decode し、source metadata を保持して upload/cache する。

```
// Cumulative implementation excerpt: decoded mip chain with source-aware caching.
struct TextureSurface { std::uint32_t width, height; PixelFormat format;
                        std::vector<std::byte> pixels; };
struct DecodedTexture { ResourceId source; std::uint64_t sourceOffset;
                        std::vector<TextureSurface> mips; };

DecodedTexture decodeTexture(ResourceId id, ByteReader& r, EvidenceLog& log) {
    TextureHeader h = readTextureHeader(r, log);
    DecodedTexture out{id, h.sourceOffset, {}};
    for (std::uint32_t level = 0; level < h.levelCount; ++level)
        out.mips.push_back(decodeMip(r, h, level, log));
    validateMipDimensions(out.mips);
    return out;
}

IDirect3DTexture8* cachedUpload(TextureCache& cache, GraphicsDevice& g,
                                const DecodedTexture& t) {
    return cache.getOrCreate({t.source, hashMips(t.mips)}, [&] { return g.upload(t); });
}
```

13.1 texture identity と binding

texture resource には dimension、format、level count、usage、memory pool がある。application は texture stage へ bind し、sampling と combination state がその寄与を定める。一つの texture は多くの material に使え、同じ material は時間とともに異なる texture を bind できる。したがって trace では、resource descriptor と各 draw における stage binding の両方を記録する。

visual result は addressing や filtering のような sampler-related choice にも依存する。repeat、clamp、point sampling、filtered sampling は、同じ image を大きく異なって見せうる。見える seam や blur は観測であり、その説明には関連する state が必要である。

13.2 mipmap と distance

mipmapped texture は、successively smaller version の image を格納する。sampling 時に適切な level を選ぶことで、surface が screen 上で小さくなるとき aliasing を減らし texture detail を制御できる。Direct3D 8 の texture creation では level count を指定でき、要求された場合には対応 hardware で sublevel を自動生成できる。

mipmap は distance による変化の一般的な説明だが、唯一ではない。material は LOD selection、texture binding の変更、fog、geometry change、display filtering でも変化しうる。mip chain を確認するには、遠方のスクリーンショットだけでなく resource または sampling evidence が必要である。

13.3 format と palette

format は texel が color と alpha をどのように符号化するかを定める。palettized representation は texel ごとに完全な color を持つ代わりに palette への index を格納し、表現の自由度と引き換えに compactness を得る。palette change は texture image を置き換えずに indexed data の見え方を変える。Direct3D 8 は palette-related device operation を公開するが、API capability は特定 asset での使用を証明しない。

13.4 texture surface と render surface

surface は texture の一部として sample されるか、rendering destination として使われるか、その role と device support によって決まる。off-screen render target では、一つの pass が別の pass で消費される image を作れる。この能力は reflection、distortion、composited interface element の調査に関係するが、target change と binding で実証しなければならない。

13.5 texture evidence checklist

dimension、format、level count、usage、pool、存在する場合は palette association、stage binding、addressing、filtering、resource を sample した draw を記録する。resource copy は data の証拠として保存し、最終 pixel result を説明するために binding と state record を組み合わせる。

証拠基準。texture image は利用可能な data を証明する。それだけでは、その data が現れた material、stage、sampling rule、pass を証明しない。

13.6 最初の supported texture slice

texture family には複数の representation がある。Listing 9.1 は、observed version-40 header と二つの paletted type だけを、palette entry が 32-bit BGRA の場合に受け入れる。dimension を検証し、256 palette entry を読み、pixel ごとの index を展開し、stored row order を反転する。DXT と mixed palette/DXT record は別 prefix と payload length を検証するまで unsupported とする。

Listing 9.1 bounded 32-bit-palette texture decoder.

```
struct SupportedTexture {
    std::string name;
    std::uint32_t width, height;
    std::vector<Rgba8> pixels;
};

SupportedTexture decodePalette32(std::span<const std::byte> data) {
    constexpr std::size_t headerBytes = 60;
    if (data.size() < headerBytes) throw ParseError("short texture header");
    const std::uint8_t type = u8(data[0]);
    const std::uint32_t version = u32le(data, 20);
    const std::uint32_t width = u32le(data, 24);
    const std::uint32_t height = u32le(data, 28);
    const std::uint32_t paletteBits = u32le(data, 56);
    if ((type != 0x91 && type != 0x01) || version != 40 || paletteBits != 32)
        throw UnsupportedRecord("texture is outside the first supported slice");
    if (!width || !height || width > 4096 || height > 4096)
        throw ParseError("invalid texture dimensions");

    constexpr std::size_t paletteEntries = 256;
    constexpr std::size_t paletteBytes = paletteEntries * 4;
    const std::size_t pixelCount = checkedMul<std::size_t>(width, height);
    requireRange(data, headerBytes, checkedAdd(paletteBytes, pixelCount));
    const auto palette = data.subspan(headerBytes, paletteBytes);
    const auto indices = data.subspan(headerBytes + paletteBytes, pixelCount);

    SupportedTexture out{fixedString(data.subspan(1, 16)), width, height,
        std::vector<Rgba8>(pixelCount)};
    for (std::uint32_t y = 0; y < height; ++y) {
        for (std::uint32_t x = 0; x < width; ++x) {
            const auto index = u8(indices[(height - 1 - y) * width + x]);
            const auto p = palette.subspan(index * 4, 4); // stored BGRA
            out.pixels[y * width + x] = {u8(p[2]), u8(p[1]), u8(p[0]), u8(p[3])};
        }
    }
    return out;
}
```

acceptance test. texture name、type、dimension、palette width、payload range、decoded hash を記録する。corner pixel と離れた複数 row を独立 decode と比較する。見た目がもっともらしくても row order または channel order が異なれば不合格である。

第 14 章 テクスチャワークスペース

デコーダは結果を検査できて初めて実用的になる。テクスチャワークスペースは、ソースメタデータ、パレットとミップ制御、チャンネル表示、ズーム、アルファ用市松模様、書き出しを統合し、選択画像のバイト由来情報を保持する。

実装成果。

遅延ミップデコード、チャンネル検査、由来情報表示、デバイス消失に耐える GPU アップロードを備えた対話型テクスチャワークスペースを実装する。

14.1 責務と所有

TextureDocument はデコード済み CPU サーフェスとソース記録を所有する。TextureView は表示選択だけを所有する。GPU テクスチャハンドルはグラフィックスキャッシュに置き、デバイス消失後に再作成する。

14.2 実装手順

オープン時にはまずメタデータとサムネイルを生成し、完全なミップデコードは要求時に行う。パレット、ミップ、チャンネル、背景の変更は表示層だけを無効化し、ソース文書は変更しない。

14.3 障害と回復

未対応形式でも信頼できる寸法とヘッダフィールドを表示し、プレビュー不可を明示する。デコード済みサーフェスがサイズとストライド検証に合格するまで書き出しは無効にする。

14.4 検証ゲート

フィクスチャでは不透明、二値アルファ、連続アルファ、複数ミップ、パレット端値、奇数幅、切り詰め、デバイス再作成を扱う。

14.5 累積 C++実装

C++クライアントマイルストーン。遅延ミップデコード、チャンネル検査、由来情報表示、デバイス消失に耐える GPU アップロードを備えた対話型テクスチャワークスペースを実装する。

```
// Cumulative implementation listing
void TextureWorkspace::open(ResourceId id) {
```

```

        close();
        document_ = textures_.readMetadata(id, diagnostics_);
        requestMip(0);
    }

    void TextureWorkspace::requestMip(unsigned level) {
        if (!document_ || level >= document_>mipCount) return;
        selectedMip_ = level;
        jobs_.replace(JobKey{document_>id, level}, [this, level](StopToken stop) {
            DecodedSurface s = textures_.decode(*document_, level, stop);
            ui_.post([this, level, s = std::move(s)]() mutable {
                if (!document_ || level != selectedMip_) return;
                cpuSurface_ = std::move(s);
                gpuTexture_ = graphics_.textures().upload(cpuSurface_);
                invalidate();
            });
        });
    }
}

```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 15 章 モデルワークスペース

モデルワークスペースは最初の完全な視覚検査ツールである。対応ジオメトリとマテリアルを読み、カメラを自動調整し、描画グループを公開し、境界や法線を重ね、選択描画をレコードとバイトへ結び付ける。

実装成果。

回転ナビゲーション、描画グループ選択、診断オーバーレイ、マテリアル検査、ソース移動を備えたモデルワークスペースを実装する。

15.1 責務と所有

ModelDocument はデコード済みメッシュ、マテリアル、依存関係、境界、由来情報を所有する。

ModelViewport はカメラと可視化フラグを保持する。選択には安定したメッシュ ID と描画グループ ID を使う。

15.2 実装手順

ロードは依存関係を解決し、CPU データをデコードし、信頼できる境界を計算し、不変バッファをアップロードし、文書を原子的に公開する。ビューポートは境界から近遠クリップ面と回転中心を導出する。

15.3 障害と回復

テクスチャ欠落時は目立つ診断用マテリアルを使い、不正ジオメトリはバッファ作成へ渡さない。空または極端な境界では未定義行列ではなく文書化された代替カメラを使う。

15.4 検証ゲート

テストでは描画数、インデックス境界、依存関係エラー、カメラフレーミング、選択安定性、オーバーレイ間の状態復元、決定論的スクリーンショットを検証する。

15.5 累積 C++実装

C++クライアントマイルストーン。回転ナビゲーション、描画グループ選択、診断オーバーレイ、マテリアル検査、ソース移動を備えたモデルワークスペースを実装する。

```
// Cumulative implementation listing
void ModelWorkspace::render(const ViewportSize size) {
    if (!document_) return;
    RenderFrame frame = renderer_.begin(size, camera_.matrices(size));
    for (const DrawGroup& group : document_->drawGroups) {
        DrawPacket packet = makeDrawPacket(*document_, group);
        packet.highlight = group.id == selection_.drawGroup;
        frame.submit(packet);
    }
    frame.flushOpaque();
    frame.flushTransparentBackToFront();
    if (view_.showBounds) overlays_.drawBounds(frame, document_->bounds);
    if (view_.showNormals) overlays_.drawNormals(frame, *document_);
    frame.present();
}

void ModelWorkspace::select(DrawGroupId id) {
    selection_.drawGroup = id;
    inspector_.navigate(document_->drawGroup(id).source);
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 16 章 Render State : 深度、カリング、フォグ、塗りつぶし

render state は draw の解釈を定める device-level setting である。geometry や texture とは独立に変えられるため、同じ mesh を使う二つの draw でも大きく異なる画像を作れる。Direct3D 時代の分析では、state history が asset と visible result を結ぶ欠けた要素になることが多い。

範囲。本章は state の分類と証拠の扱いを説明する。特定の material class に対してファイナルファンタジー XI がどの state value を使うかは主張しない。

C++クライアント・マイルストーン。 unknown bit を捨てない evidence-scoped flag decoder から explicit render-state descriptor を構築する。

```
// Cumulative implementation excerpt: known bits are decoded; unknown bits survive.
struct RenderStateDesc {
    bool depthTest = true, depthWrite = true, fog = false;
    CullMode cull = CullMode::CounterClockwise;
    std::uint32_t rawFlags = 0, unknownFlags = 0;
};

RenderStateDesc decodeRenderFlags(std::uint32_t raw) {
    constexpr std::uint32_t Known = 0x0000000Fu;
    RenderStateDesc s;
    s.rawFlags = raw; s.unknownFlags = raw & ~Known;
    s.depthTest = (raw & 0x1) != 0;
    s.depthWrite = (raw & 0x2) != 0;
    s.fog = (raw & 0x4) != 0;
    s.cull = (raw & 0x8) ? CullMode::None : CullMode::CounterClockwise;
    return s;
}
```

16.1 depth test と depth write

depth test は candidate pixel をその screen location にすでに保存された depth と比較する。depth write は pixel が受け入れられたとき保存値を更新する。この二つは不透明 scene における通常の occlusion の大部分を定める。また、後の transparent work では新しい depth を書かずに既存 depth に対して test する場合があります、重要である。

object が別の object の背後で消えることは depth test と整合するが、スクリーンショットだけでは state value、target setup、draw order を確定しない。trace は draw が depth test と depth write を使ったかを示し、統制された visual test は結果の挙動を示せる。

16.2 face culling

triangle winding は renderer に front-facing face を判断する基礎を与える。culling は front-facing または back-facing triangle を reject し、処理を減らし、閉じた model の内側が現れることを防ぐ。culling は visibility culling と同じではない。前者は orientation で個別 triangle を除き、後者は draw 前に object または group を省くことがある。

16.3 fog

fog は depth-related または distance-related rule に従って fragment を blend または変更する。遠方 geometry と sky color をつなぎ、contrast を下げ、遠景の視覚的負担を制限できる。その見た目は

alpha fade、lighting、texture change、post-processing に似ることがある。特定には関係する fog state、parameter、ordering が必要である。

16.4 fill と rasterization choice

fill mode は primitive を point、wireframe、filled triangle のどれとして rasterize するかを制御する。他の rasterization-related setting は clipping、depth bias、shading、polygon の接続に影響する。通常の画像では見えないことが多いが、gap、coplanar flicker、diagnostic rendering mode のような artifact を説明できる。

16.5 state group と state block

state は group として読むと最も情報量が多い。不透明 environment work から effect への変化には、depth write、culling、fog、alpha rule、texture stage、blend factor が一緒に関わることがある。Direct3D 8 は device state の集合を capture して apply する state-block operation も提供する。grouped change の存在は rendering transition の証拠であって、named effect の自動的な証拠ではない。

16.6 state evidence の記録

各 draw について depth、culling、fog、fill、clipping、rasterization に関する active value を記録し、近くの draw と結果画像を比較する。最も有用な結論は、最小限の state-supported claim で観測された差を説明する。

証拠基準。trace の state value は、その trace についての確認済みの事実である。それらを『water pass』や『shadow pass』と呼ぶ意味づけには scene correlation が必要であり、推論に留まることもある。

16.7 decoded material から再現可能な state packet へ

reconstruction renderer には complete state baseline が必要である。前 draw の culling、alpha、depth、texture transform を次 draw へ継承させない。Listing 10.1 は Direct3D 8 で explicit packet を適用する。packet には supported material field だけを入れ、例の texture-scroll rate は viewer policy として decoded evidence から分離する。

Listing 10.1 explicit fixed-function material state と分離された reconstruction policy。

```

struct MaterialState {
    bool twoSided = false;
    bool alphaTest = false;
    std::uint8_t alphaReference = 0;
    bool alphaBlend = false;
    bool depthWrite = true;
    bool textureScroll = false;
};

void applyMaterial(IDirect3DDevice8& d, const MaterialState& m, float seconds) {
    d.SetRenderState(D3DRS_CULLMODE, m.twoSided ? D3DCULL_NONE : D3DCULL_CW);
    d.SetRenderState(D3DRS_ALPHATESTENABLE, m.alphaTest);
    d.SetRenderState(D3DRS_ALPHAREF, m.alphaReference);
    d.SetRenderState(D3DRS_ALPHAFUNC, D3DCMP_GREATER);
    d.SetRenderState(D3DRS_ALPHABLENDENABLE, m.alphaBlend);
    d.SetRenderState(D3DRS_SRCBLEND, D3DBLEND_SRCALPHA);
    d.SetRenderState(D3DRS_DESTBLEND, D3DBLEND_INVSRCALPHA);
    d.SetRenderState(D3DRS_ZWRITEENABLE, m.depthWrite);
    d.SetTextureStageState(0, D3DTSS_COLOROP, D3DTOP_MODULATE2X);
    d.SetTextureStageState(0, D3DTSS_COLORARG1, D3DTA_TEXTURE);
    d.SetTextureStageState(0, D3DTSS_COLORARG2, D3DTA_DIFFUSE);

    D3DMATRIX uv = identityD3DMatrix();
    uv._31 = m.textureScroll ? std::fmod(seconds * 0.012f, 1.0f) : 0.0f;
    uv._32 = m.textureScroll ? std::fmod(seconds * 0.006f, 1.0f) : 0.0f;
    d.SetTransform(D3DTS_TEXTURE0, &uv);
    d.SetTextureStageState(0, D3DTSS_TEXTURETRANSFORMFLAGS,
        m.textureScroll ? D3DTTFF_COUNT2 : D3DTTFF_DISABLE);
}

```

reconstruction boundary. MODULATE2X、alpha comparison、一般的な source-alpha blending、例の scroll rate は、retail trace または decoded field で actual value が確立した時点で個別に promote または replace する。

第 17 章 Alpha Test、Alpha Blending、描画順序

alpha は透明表現として語られがちだが、複数の異なる描画判断を支える。alpha test は fragment を完全に discard できる。alpha blending は新しい fragment を target にすでにある color と組み合わせる。draw order は、その組み合わせの時点でどの destination color が利用できるかを定める。これらの機構は分けて分析しなければならない。

範囲。本章は標準的な描画挙動を説明する。透明に見える結果だけでは、ファイナルファンタジー XI の draw がどの alpha mechanism を使ったかは確定しない。

C++クライアント・マイルストーン。 alpha 関連 field を decode し、意図した depth/order 規則を持つ opaque、cutout、blended queue を作る。

```

// Cumulative implementation excerpt: explicit opaque, cutout, and blended queues.
struct QueuedDraw { Draw draw; float cameraDepth; std::uint64_t sourceOrder; };
struct DrawQueues { std::vector<QueuedDraw> opaque, cutout, blended; };

void enqueue(DrawQueues& q, QueuedDraw item, const AlphaDesc& a) {
    if (a.blendEnabled) q.blended.push_back(std::move(item));
    else if (a.testEnabled) q.cutout.push_back(std::move(item));
    else q.opaque.push_back(std::move(item));
}

```

```

}

void submit(DrawQueues& q, Renderer& r) {
    stableStateSort(q.opaque); stableStateSort(q.cutout);
    std::stable_sort(q.blended.begin(), q.blended.end(),
        [](auto& a, auto& b) { return a.cameraDepth > b.cameraDepth; });
    r.draw(q.opaque, DepthWrite::On);
    r.draw(q.cutout, DepthWrite::On);
    r.draw(q.blended, DepthWrite::Off);
}

```

17.1 alpha test : 受け入れるか破棄するか

alpha test では、sampled alpha value を test function に従って reference と比較する。失敗した fragment は discard され、通過した fragment は後続操作に対する通常の candidate になる。結果は partial transparency ではなく hard cutout である。このため alpha test は、depth behavior を opaque geometry に近いままにしたい patterned edge、foliage-like shape、masked detail に有用である。

ぎざぎざした、または binary に見える edge は alpha test と整合するが、filtering、resolution、compression、capture scaling も同じ見た目を作りうる。確認には関連する alpha-test state、または threshold behavior を分ける controlled test が必要である。

17.2 alpha blending : source と destination

alpha blending は current draw からの source color と、render target にすでにある destination color を組み合わせる。blend factor はそれぞれがどれだけ寄与するかを定める。conventional source-alpha blending は translucency を作れ、additive combination は light-like accumulation を作れ、他の factor は specialized result を支える。重要なのは destination が以前の作業をすでに反映していることである。

blending は destination color に依存するため、blended geometry は order に敏感である。重なる二つの translucent surface は逆順で draw すると違って見えうる。depth test は surface が近い opaque geometry を通して現れることを防ぎ、depth write は一つの transparent surface が別の surface を誤って隠さないように異なる扱いを受けることが多い。

17.3 draw order は material の一部である

texture と blend factor だけを挙げて material description は不完全である。frame における場所も重要である。opaque work は depth が hidden pixel を効率よく reject できるように配置されることが多

い。cutout はその depth structure に参加できることが多い。blended work には後の phase、sorting、grouping、または意図的な approximation が必要になることがある。engine は particle、character accessory、weather、interface layer、screen-space effect で異なる選択をできる。

17.4 一般的な visual clue とその限界

暗い background 上に accumulate する glow は additive blending と整合する。background を見せる pane は source-destination blending と整合する。firm depth edge を保つ leaf-like mask は alpha test と整合する。それぞれは state evidence と draw context が支えるまで仮説に過ぎない。似た画像は multiple pass、precomposed texture、target-based effect から生じうる。

17.5 実用的な alpha record

調査対象 draw について、alpha testing が enabled か、function と reference、blending が enabled か、source と destination factor、depth-test と depth-write value、culling、target、draw order、visible overlap condition を記録する。同じ draw を opaque および blended neighbor と比較する。この記録は『transparent texture』のような label より多くを教える。

証拠基準。blend factor と alpha-test value は確認済みの trace fact である。芸術的意図や pass name の説明には scene との correlation が必要で、有力な推論に留まることもある。

第 18 章 Texture Stage と固定機能 Combiner

programmable pixel shader が標準的な material language になる前、固定機能 pipeline は texture stage を通じて material を組み立てられた。各 stage は bound texture を sample し、configured operation と argument に従ってその value を current color と組み合わせる。一つの stage の result は次の stage の input になりうる。

範囲。本章は固定機能 stage model が表現できることを説明する。multi-texture effect が見えることは、ファイナルファンタジー XI が特定の stage sequence を使った証拠ではない。

C++クライアント・マイルストーン。 material-stage record を log/reproduce 可能な texture-stage と combiner description へ parse する。

```
// Cumulative implementation excerpt: parse, log, and apply fixed-function stages.
struct TextureStageDesc {
```

```

        std::uint32_t colorOp, colorArg1, colorArg2;
        std::uint32_t alphaOp, alphaArg1, alphaArg2;
        std::uint32_t texCoordIndex;
    };

    TextureStageDesc readStage(ByteReader& r, EvidenceLog& log) {
        const auto start = r.position();
        TextureStageDesc s{r.u32(), r.u32(), r.u32(), r.u32(), r.u32(), r.u32()};
        log.field(currentResource(), start, "textureStage", s, Confidence::Observed);
        return s;
    }

    void applyStage(IDirect3DDevice8& d, DWORD n, const TextureStageDesc& s) {
        d.SetTextureStageState(n, D3DTSS_COLOROP, s.colorOp);
        d.SetTextureStageState(n, D3DTSS_COLORARG1, s.colorArg1);
        d.SetTextureStageState(n, D3DTSS_COLORARG2, s.colorArg2);
        d.SetTextureStageState(n, D3DTSS_ALPHAOP, s.alphaOp);
        d.SetTextureStageState(n, D3DTSS_TEXCOORDINDEX, s.texCoordIndex);
    }

```

18.1 current color

stage は current color と、設定されている場合は current alpha を扱う。material evaluation の始まりでは、これらの value は diffuse vertex color または他の定義された input から来る。stage は texture sample、diffuse color、current result、complement などの argument を選び、color または alpha operation を適用する。典型的な operation には selection、modulation、addition、interpolation がある。

color と alpha の分離は重要である。material は color を multiply しながら、alpha を別の input から select したり、alpha を後の blend の制御に使ったりできる。一枚の texture に見える画像が、独立に設定された複数 operation の積であることがある。

18.2 複数の coordinate set と stage

各 texture stage は、その stage のために選択または transform された texture coordinate を使える。複数の coordinate set を持つ mesh は、base image と異なる mapping の detail、light-like、mask-like image を支えられる。同じ coordinate も pattern を scroll または project するために transform できる。これらは capability であり、使用を決めるには vertex-layout、stage-state、visual evidence を合わせる必要がある。

18.3 一般的な material pattern

modulation は base texture を vertex color または別 texture で tint できる。addition はより明るい contribution を作れる。interpolation は alpha-like factor を使って source 間を選べる。後の stage は

以前の result を拡張せず置き換えることもできる。これらの pattern は detail texture、mask、light-like overlay、animated surface を調査する仮説になるが、pattern name で recorded operation を置き換えてはならない。

18.4 stage order と disabled stage

stage order は重要である。各 stage が以前の current result を consume できるためである。disabled stage は関係する方向の固定機能 chain を終える。したがって同じ texture 集合でも、stage を並べ替えたり operation を変えたりすると異なる画像を生む。state diff では texture binding とすべての active stage の color/alpha operation を記録する。

18.5 combiner trace を読む

各 stage について bound texture、coordinate source、transform、color operation、color argument、alpha operation、alpha argument、stage-specific state を記録する。次に resulting color と alpha の小さな symbolic expression を書く。expression は provisional でもよいが、仮定を見えるようにし、後の capture が正確な予測を検証できる。

証拠基準。記録された stage operation は確認済みの API evidence である。その芸術的役割を detail map、light map、mask などと特定するには resource と scene の correlation が必要である。

第 19 章 Vertex Shader とプログラマブル技法

programmable vertex processing は rasterization 前に vertex data へ行う処理を広げる。application は固定機能の transform-and-lighting path だけに頼らず、declared input を解釈して transformed position や他の output を書く vertex shader を選べる。Direct3D 8 時代には、この柔軟性は固定機能 material と texture-stage combiner に共存していた。

範囲。本章は API と分析の入門である。shader binding、declaration、または同等の client-specific trace が確立するまで、ファイナルファンタジー XI の draw が vertex shader を使うことを主張しない。

C++クライアント・マイルストーン。 検証済み shader record、declaration、constant を capability check と明示的 fallback 付きで load する。

```
// Cumulative implementation excerpt: capability-gated shader with fixed-function fallback.  
struct ShaderProgram { DWORD handle = 0; bool programmable = false; };
```

```

ShaderProgram loadVertexProgram(IDirect3DDevice8& d,
                                const ShaderRecord& record,
                                const D3DCAPS8& caps) {
    if (caps.VertexShaderVersion >= D3DVS_VERSION(1, 1) && record.validated) {
        DWORD handle = 0;
        if (SUCCEEDED(d.CreateVertexShader(record.declaration.data(),
                                           record.bytecode.data(), &handle, 0)))
            return {handle, true};
    }
    d.SetVertexShader(record.fallbackFvf);
    return {record.fallbackFvf, false};
}

```

19.1 vertex shader が変えられるもの

vertex shader は position を transform し、per-vertex lighting value を計算または変更し、texture coordinate を生成または transform し、deformation-related calculation を適用し、triangle 全体で後に interpolation される value を準備できる。処理単位は pixel ではなく vertex である。したがって programmable pixel stage を必要とせずに、silhouette motion、lighting gradient、coordinate behavior、その他 vertex-derived aspect を変えられる。

19.2 input、declaration、constant

programmable path には input layout と constant についての合意が必要である。vertex declaration は stream data が shader input へどう対応するかを記述する。constant は matrix、color、time-like parameter、material value のように draw または frame ごとに変わる value を与える。shader program は input を output へ変える方法を定める。

証拠作業では shader identity だけでは十分でない。declaration、stream、stride、回復できる場合は constant、texture と state、primitive range、visible result を記録する。一つの shared shader は異なる constant と binding によって複数の material class を扱う。

19.3 固定機能とプログラマブル path は共存できる

engine は多くの通常 draw に fixed-function processing を使い、選ばれた transform や effect に programmable vertex processing を使うことができる。一つの frame 内で path を切り替えることもできる。この可能性は、ゲーム全体を完全に fixed-function または完全に programmable と扱わないよう警告する。分析単位は draw とその active state である。

19.4 visual clue は仮説である

deformation、通常と異なる coordinate motion、複雑な lighting gradient は shader 仮説を導ける。しかしこれらの appearance は vertex shader を一意に特定しない。同等の結果は CPU-prepared vertex、skeletal matrix、fixed-function texture transform、multiple draw、precomputed color からも得られる。shader selection を示す trace は effect の見た目より強い証拠であり、recovered code や constant は説明をさらに強くできる。

19.5 programmable-technique record

programmable path が疑われる場合、device の shader selection、declaration、stream binding、constant、draw call、近くの state change、result が現れる condition を記録する。unknown value を推測で名前付けせず unknown として保存する。後の比較では constant、matrix、input stream を変えると観測 effect が変わるかを問える。

証拠基準。bound vertex shader はその draw について確認済みである。その意味上の目的が skinning、wave motion、coordinate generation、その他の役割かどうかは、input、output、scene behavior が一致する必要がある。

第 III 部 シーン構築と対話型ワークスペース

第 20 章 ワールドシーン：ゾーン、チャンク、描画グループ

大規模なゲーム世界は通常、毎フレーム一つの区別のない mesh として扱えない。renderer には scene の管理可能な部分を選び、準備し、送出する方法が必要である。zone、chunk、sector、cell、batch、draw group などの語は有用な分析ラベルだが、互いに同義ではなく、証拠なしに engine へ当てはめてはならない。

範囲。本章は scene organization の語彙を与える。ファイナルファンタジー XI の内部 partition 名、boundary、submission group を確認するものではない。

C++クライアント・マイルストーン。 zone resource graph を parse し、検証済み chunk、entity、reference、draw group から world scene を構築する。

```
// Cumulative implementation excerpt: build a scene graph from bounded chunk references.
Scene buildZone(const DatFile& dat, EvidenceLog& log) {
    Scene scene;
    std::unordered_map<ResourceName, Mesh> meshes;
    for (const ChunkView& c : dat.chunks()) {
        if (c.type == 0x2E) {
            Mesh m = parseZoneMesh(c, log);
            meshes.emplace(m.name, std::move(m));
        }
        for (const ChunkView& c : dat.chunks()) if (c.type == 0x1C) {
            for (const Placement& p : parsePlacements(c, log)) {
                auto found = meshes.find(p.meshName);
                if (found == meshes.end()) { scene.missing.push_back(p.meshName); continue; }
                scene.instances.push_back({&found->second, placementMatrix(p), p.flags});
            }
        }
    }
    scene.meshes = std::move(meshes);
    return scene;
}
```

20.1 体験上の境界としての zone

zone は player-facing の有用な単位である。固有の landscape、object population、atmosphere、loading context を持つ名前付き area である。data package、world-space coordinate region、server concept、rendering scene、またはそれらの複数に対応することがある。この語だけでは geometry と material が描画のためにどう分けられるかは分からない。

20.2 spatial chunk と selection

renderer は zone を chunk または cell へ分け、すべての object ではなく近くまたは visible な部分を考慮できる。この分割は grid、hierarchy、authored room、terrain tile、object list、その他の構造に従いうる。camera motion で object が現れる挙動は selection boundary を示唆するが、実際の形を確立できるのは data または trace evidence だけである。

chunk は draw と同じでもない。一つの selected region は material、layer、object class ごとに多くの draw を発行できる。逆に一つの draw は複数の近接 piece から compatible geometry を結合できる。spatial organization と submission organization を分けて扱う。

20.3 draw group と material group

draw group は compatible な state、material、transform condition で render される geometry の実用的な集合である。grouping は texture、shader、stream、render state の変更を減らせるが、transparency、animation、culling、artistic ordering によって制約されることがある。material group も同様に、active binding と draw range が示すまで分析上の語である。

20.4 static と dynamic な scene content

world scene には通常、安定した structural geometry と、character、moving object、weather、interface marker のような dynamic content が含まれる。この区別は、update frequency、visibility rule、data lifetime、rendering order が異なりうるため有用である。見かけ上 static または dynamic なすべての item が一つの shared code path に従うという主張ではない。

20.5 world scene の調査方法

repeatable な location と camera route から始める。route が一度に一つの condition だけを変える間、どの object が現れ、消え、detail を変え、state を変えるかを記録する。route を resource と draw evidence に対応させる。どの buffer、texture、state、target change が繰り返されるか。次に観測が spatial partition、material grouping、distance rule、または組み合わせを支持するかを問う。

強い結果は証拠が支えることだけに名前を付ける。たとえば『cameraがこのboundaryを越えるところらのdrawと一緒に繰り返される』は、構造を早まってchunkと呼ぶより強い。後の証拠はboundaryがzone portal、material switch、無関係なeventであることを示すかもしれない。

証拠基準。spatial data、client behavior、trace evidenceが確立したときにworld-scene classificationは確認済みとなる。visual proximityだけはinternal grouping ruleではない。

20.6 world draw の provenance spine

scene nodeは生成過程を消してはならない。次のrecordはresource identifier、chunkとfield offset、placement、material、state、draw rangeを保持する。inspectorはselected pixel contributionから、それを供給したexact file rangeまで逆にたどれる。

Listing 14.1 decoded field から draw submission まで保持する provenance。

```
struct DecodedField {
    ResourceId resource;
    std::size_t chunkOffset;
    std::size_t fieldOffset;
    std::size_t byteCount;
    std::string name;
    std::string value;
    Confidence confidence;
};

struct DrawProvenance {
    ResourceId resource;
    std::size_t geometryChunk;
    std::vector<DecodedField> fields;
    std::string objectName;
    Mat4 placement;
    std::string materialName;
    MaterialState state;
    std::size_t firstIndex, indexCount;
};

DrawProvenance makeDrawProvenance(const Placement& placement,
                                   const SupportedGeo& mesh,
                                   const EvidenceLog& log) {
    return {placement.source.resource, mesh.source.chunkOffset,
            log.fieldsFor(mesh.source), placement.objectName,
            placement.world, mesh.material, placement.materialState,
            0, mesh.indices.size()};
}
```

対応UIでは、successful instanceとともに unsupported reference と unresolved object name を表示する。missing geometry は現在の decoder または resource set に関する evidence であり、placement を scene report から黙って消す許可ではない。

第 21 章 ゾーンロードとシーン組み立て

ゾーンは単一メッシュではない。本章ではリソースグラフを解決し、対応チャンクをデコードし、再利用モデルをインスタンス化し、配置を適用し、環境状態を結び、必須基盤が有効になってからシーンを公開するローダを構築する。

実装成果。

依存関係を考慮したゾーンロードを実装し、描画と検査に適した不変シーンスナップショットを公開する。

21.1 責務と所有

ZoneDocument は、不変ソース記録、デコード済みアセットハンドル、インスタンス変換、空間インデックス、環境制御、診断を分離する。描画オブジェクトは生バイトポインタを保持しない。

21.2 実装手順

ローダはまずルートと依存関係を発見し、並列に検証・デコードし、一つの決定論的スレッドでインスタンスを組み立て、境界と空間構造を作り、可視優先項目をアップロードしてから任意内容をストリームする。

21.3 障害と回復

循環、依存欠落、未対応チャンク、キャンセルされた任意処理は診断になる。必須配置または座標データの失敗はシーン公開を止めるが、装飾一個の失敗では止めない。

21.4 検証ゲート

フィクスチャシーンでは依存関係閉包、決定論的インスタンス順、変換由来情報、境界、部分内容、キャンセル、基準俯瞰画像を検証する。

21.5 累積 C++実装

C++クライアントマイルストーン。依存関係を考慮したゾーンロードを実装し、描画と検査に適した不変シーンスナップショットを公開する。

```
// Cumulative implementation listing
ZoneLoadResult ZoneLoader::load(ResourceId root, StopToken stop) {
```

```

ZoneLoadResult result;
ResourceGraph graph = discoverDependencies(root, stop, result.issues);
auto decoded = decoderPool_.decode(graph.nodes(), stop, result.issues);
if (stop.stopRequested()) return result;

ZoneBuilder builder(root);
for (ResourceId id : graph.topologicalOrder()) {
    const DecodedResource* r = decoded.find(id);
    if (!r) continue;
    builder.accept(*r, result.issues);
}
if (!builder.hasRequiredFoundation()) {
    result.issues.push_back({Severity::Error, "required zone records missing"});
    return result;
}
result.scene = builder.finishDeterministically();
return result;
}

```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 22 章 オブジェクトエクスプローラ、選択、プロパティ編集

シーンビューアは、階層、ビューポート、プロパティが同じ識別体系を共有して初めて探索ツールになる。本章では仮想化オブジェクトツリー、ピッキングと強調、ソース移動、デコードデータと分離した可逆編集を実装する。

実装成果。

安定 ID、強調表示、可逆上書き、ソース移動を備え、ツリー、ビューポート、プロパティ検査を同期させる。

22.1 責務と所有

SceneObjectId は一つのシーンスナップショット内で安定する。SelectionModel はツリー、一覧、ビューポート、プロパティパネルで共有する。EditLayer はオブジェクト ID ごとの上書きを保持し、ソース記録を変更しない。

22.2 実装手順

フィルタはオブジェクトを複製せず表示行インデックスを生成する。ビューポートピッキングは候補 ID と深度を返し、選択モデルが修飾キー規則を適用し、各オブザーバは一つの版付き選択イベントから更新する。

22.3 障害と回復

削除またはフィルタされたオブジェクトも有効な選択として残り得るが非表示になる。不正編集は元に戻す履歴へ入る前に拒否する。シーン再構築時はソース識別で選択復元を試み、曖昧さを報告する。

22.4 検証ゲート

テストでは一万行の仮想化、ピック順、多重選択、フィルタ変更、元に戻す／やり直す、ソース移動、再ロード後の選択復元を扱う。

22.5 累積 C++実装

C++クライアントマイルストーン。安定 ID、強調表示、可逆上書き、ソース移動を備え、ツリー、ビューポート、プロパティ検査を同期させる。

```
// Cumulative implementation listing
void SelectionModel::replace(std::span<const SceneObjectId> ids,
                             SelectionOrigin origin) {
    std::vector<SceneObjectId> next(ids.begin(), ids.end());
    std::sort(next.begin(), next.end());
    next.erase(std::unique(next.begin(), next.end()), next.end());
    if (next == selected_) return;
    selected_ = std::move(next);
    ++version_;
    changed_.publish({version_, origin, selected_});
}

bool EditLayer::setTransform(SceneObjectId id, const Transform& value) {
    if (!isFinite(value) || value.scale.minComponent() <= 0.0f) return false;
    history_.push(TransformCommand{id, overrides_[id].transform, value});
    overrides_[id].transform = value;
    return true;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 23 章 コリジョンジオメトリと移動診断

コリジョンデータは元レンダラが描画しなくても検査価値を持つ。エクスプローラはコリジョンメッシュ、歩行可能性仮説、レイテスト、接地、差分オーバーレイを公開し、コリジョン記録と可視地形を混同しない。

実装成果。

明示的な信頼度ラベルを備えたコリジョン検査、加速、レイクエリ、任意接地、視覚差分オーバーレイを実装する。

23.1 責務と所有

CollisionDocument はデコード済み三角形、ソース範囲、加速構造、分類ラベルを保持する。移動状態はコリジョンスナップショット版を参照し、再ロード後に古い結果が黙って残らないようにする。

23.2 実装手順

デコードはインデックスと有限座標を検証し、三角形を黙って削除せず、境界ボリューム階層を構築し、任意の診断分類を別の派生層として生成する。

23.3 障害と回復

退化または非有限三角形はソース参照付きで隔離する。対応コリジョンがない場合、カメラ移動は自由飛行へ切り替え、接地不可を表示する。

23.4 検証ゲート

フィクスチャでは三角形保持、レイヒット順、BVH と総当たりの同値性、接地許容差、版無効化、オーバーレイ状態分離を検証する。

23.5 累積 C++実装

C++クライアントマイルストーン。明示的な信頼度ラベルを備えたコリジョン検査、加速、レイクエリ、任意接地、視覚差分オーバーレイを実装する。

```
// Cumulative implementation listing
std::optional<RayHit> CollisionScene::cast(const Ray& ray) const {
    std::optional<RayHit> nearest;
    bvh_.visitRay(ray, [&](TriangleId id) {
        const Triangle& t = triangles_[id.value];
        if (auto hit = intersect(ray, t.positions)) {
            if (!nearest || hit->distance < nearest->distance)
                nearest = RayHit{id, hit->distance, hit->barycentric, t.source};
        }
    });
    return nearest;
}

std::optional<float> CollisionScene::groundHeight(Vec3 p, float above) const {
    Ray ray{{p.x, p.y + above, p.z}, {0.0f, -1.0f, 0.0f}};
    if (auto hit = cast(ray)) return ray.origin.y - hit->distance;
    return std::nullopt;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 24 章 可視性：視錐台カリング、距離、オクルージョン

visibility は、特定 view に対してどの candidate work を render する価値があるかを決める過程である。一つの test に還元してはならない。renderer は camera frustum の外の object を除き、distance rule の先の content を省き、より単純な representation を選び、他の geometry の背後に隠れた geometry を reject し、authored visibility boundary を使いうる。同じ object がスクリーンショットから消えることは、これらの複数 mechanism と整合する。

範囲。本章は visibility analysis framework である。ファイナルファンタジー XI がどの visibility system を実装するかは確立しない。

C++クライアント・マイルストーン。 supported visibility field を decode し、unknown flag を保持して frustum、distance、optional occlusion test を実装する。

```
// Cumulative implementation excerpt: layered visibility without discarding unknown flags.
bool visible(const VisibilityRecord& v, const Camera& camera,
             const OcclusionBuffer* occlusion) {
    if (!camera.frustum().intersects(v.bounds)) return false;
    const float distance = length(v.bounds.center() - camera.eye);
    if (distance > v.drawDistance) return false;
    if (occlusion && v.occlusionTestSupported && occlusion->hidden(v.bounds)) return false;
    return true; // v.unknownFlags remain stored for diagnostics.
}

std::vector<const Instance*> gatherVisible(const Scene& s, const Camera& c) {
    std::vector<const Instance*> out;
    for (const Instance& i : s.instances)
        if (visible(i.visibility, c, s.occlusion.get())) out.push_back(&i);
    return out;
}
```

24.1 viewing frustum

view transform と projection transform は screen へ到達できる space の volume を定める。frustum culling は candidate object または bounding volume をその volume に対して test し、完全に外にある work を reject できる。結果は camera position、orientation、projection、candidate に選んだ bound に依存する。大きな bound は detailed geometry が off-screen に見える後も visible のままかもしれない、小さな bound はより selective になりうる。

24.2 distance と detail selection

distance-based rule は content が画像にとって重要でなくなると remove、simplify、substitute できる。transition は abrupt または gradual であり、category、camera direction、quality setting、scene condition に依存しうる。遠方 object の detail が減ることは自動的に geometric LOD ではない。fog、mipmapping、texture filtering、alpha behavior、display scaling も関連した視覚変化を作れる。

24.3 occlusion と hidden work

occlusion は camera の広い view 内にあるが別 geometry の背後に隠れた geometry に関わる。system によっては work を submit する前に occlusion を test し、別の system は submit 後の depth test に主に頼り、さらに別の system は authored portal や room boundary を使う。pixel-level depth result は、より早い occlusion-culling stage を証明しない。candidate が draw 前に省かれたかが問いであり、それには draw または selection evidence が必要である。

24.4 visibility はしばしば hierarchical である

renderer は最初に zone region を選び、次に object group を test し、最後に depth で個々の hidden pixel を reject できる。content によってこの hierarchy の level は異なりうる。terrain、static structure、character、particle、interface element が同じ visibility policy を共有する必要はない。一つの scene class から universal rule を推論してはならない。

24.5 controlled visibility experiment

安定した object を選び、一度に一つの camera parameter を変える。frustum から出るまで rotate し、direction を保って遠ざかり、occluder の背後へ移動し、documented quality setting を変える。object の draw call、resource binding、または最終 pixel だけが変わるかを記録する。複数 position で test を繰り返すと continuous visual effect と discrete selection boundary を分けられる。

よい結果は『この記録された condition では、この distance の後に object の draw が止まる』と述べるかもしれない。trace、bound、implementation evidence がその名前を支えるまで、特定 culling algorithm へ飛躍してはならない。

証拠基準。absent draw は capture condition における pre-raster selection の証拠である。present draw にもかかわらず pixel がない場合は、depth、alpha、fog、clipping の結果である可能性がある。

第 25 章 地形、地面、大規模な環境ジオメトリ

ground rendering は世界の scale、navigation readability、atmosphere の多くを担う。terrain-like mesh、authored structural surface、repeated material、vertex color、fog、decal、distance rule を組み合わせうる。terrain という語は視覚的には有用だが、証拠なしに特定 data format や generation method についての主張として使ってはならない。

範囲。本章は大きな environment surface の分析枠組みを与える。ファイナルファンタジー XI の ground data structure や rendering path を確認するものではない。

C++クライアント・マイルストーン。 terrain sector を parse し、reusable ground geometry、material、bounds、大規模 environment submission を作る。

```
// Cumulative implementation excerpt: validated terrain sectors become reusable draws.
TerrainSector readTerrainSector(ByteReader& r, std::size_t end) {
    TerrainSector s;
    s.bounds = readAabb(r);
    s.material = ResourceId{r.u32()};
    const auto vertexCount = r.u32(), indexCount = r.u32();
    s.vertices = readTerrainVertices(r, vertexCount, end);
    s.indices = r.arrayWithin<std::uint16_t>(indexCount, end);
    validateIndices(s.indices, s.vertices.size());
    return s;
}

void submitTerrain(const Terrain& terrain, const Frustum& f, DrawList& draws) {
    for (const TerrainSector& sector : terrain.sectors)
        if (f.intersects(sector.bounds))
            draws.add({sector.gpuGeometry, sector.material, sector.bounds});
}
```

25.1 geometry と surface appearance は別である

low-density mesh でも texture detail、color variation、lighting、fog、layered material によって豊かに見える。逆に dense mesh も uniform material では単純に見える。分析では geometric evidence—vertex position、topology、draw range、transform—と surface evidence—texture、stage、vertex color、state、distance behavior—を分けるべきである。

25.2 repetition と variation

large surface は限られた image 集合を再利用することが多い。variation は tiling、coordinate transform、vertex color、mask、overlay、separate object から生じうる。スクリーンショットの

repeated pattern は tiled material を示唆するが、coordinate scale、texture dimension、stage 数を証明しない。その主張には resource と stage evidence が必要である。

25.3 boundary と seam

visible boundary は geometry、material、texture-coordinate behavior、fog、lighting、zone data、authored decal-like overlay の変化を示すかもしれない。seam は複数 angle と distance から test できるため特に有用である。seam が camera と共に動くなれば presentation または projection に関係するかもしれない、geometry に従うなら material または spatial boundary を反映するかもしれない。

25.4 ground、collision、rendering

player movement や collision に使う surface が screen へ render される surface と同じである必要はない。render geometry は simplify、displace、material ごとに split、または collision が残る場所で absent になりうる。collision にも invisible boundary が含まれる。両方の data set を結び付けられるまで、movement observation と graphics claim を分ける。

25.5 large-scale distance treatment

fog、mipmap、reduced geometric detail、visibility selection、sky composition はすべて convincing horizon に寄与しうる。効果は相互作用する。遠方 feature がないのは cull された、fog に隠れた、lower-detail representation に統合された、または test condition で submit されなかった可能性がある。draw record と controlled camera route で分ける。

25.6 investigation checklist

選んだ ground region について camera route、transform、geometry buffer と range、material と texture-stage state、利用可能な vertex color、distance change、neighboring draw group、visible seam を記録する。目的は map-like image から construction を推測することではなく、evidence から surface を再構成することである。

証拠基準。terrain label は geometry、resource、client evidence が実際の rendering unit を定めるまで記述的なものに留まる。

第 26 章 静的オブジェクト、小物、建築ディテール

static object は zone に読み取りやすい landmark と material character を与える。wall、door、bridge、furniture、vegetation-like form、decorative structure などである。geometry や texture を共有しても、placement、transform、material state、visibility rule、animation state が異なりうる。視覚的な repetition は調査すべき関係の証拠であって、一つの resource や draw strategy の証明ではない。

範囲。本章は static scene content の分析方法を与える。ファイナルファンタジー XI の object database、instancing system、placement format を特定しない。

C++クライアント・マイルストーン。 model と placement record を parse し、shared geometry と per-instance state で prop と architecture を instantiate する。

```
// Cumulative implementation excerpt: shared model geometry, per-placement state.
struct ModelInstance {
    std::shared_ptr<const Model> model;
    Mat4 world;
    InstanceFlags flags;
    ResourceId environment;
};

ModelInstance instantiate(const PlacementRecord& p, ModelCache& cache) {
    auto model = cache.getOrLoad(p.modelId);
    Mat4 world = translation(p.position) * rotationZ(p.rotation.z)
        * rotationY(p.rotation.y) * rotationX(p.rotation.x)
        * scale(p.scale);
    return {std::move(model), world, decodeInstanceFlags(p.rawFlags), p.environmentId};
}
```

26.1 placement は model と別である

model は reusable geometry と associated material possibility を記述する。placement は location、orientation、scale、ときに additional flag や variant を与える。複数 placement は同じ resource を指しながら別の screen result を生みうる。逆に視覚的に似た prop が matching material を持つ別 model の場合もある。resource identity と world placement を別々の evidence field として扱う。

26.2 reuse と variation

reuse は memory と authoring cost を節約し、variation は world が機械的に見えることを避ける。transform difference、mirrored orientation、texture variation、vertex color、decal、nearby lighting、partial occlusion は shared asset を違って見せられる。比較ではまず screen 上で何が repeats するかを確立し、次に buffer identity、texture、draw range も repeats するかを test する。

26.3 architectural composition

architecture はしばしば large structural surface と、distinct material や draw order を使いうる smaller detail を組み合わせる。window、trim、sign、transparent ornament は、視覚的には一つの building に属しても separate draw になることがある。このため object-level label は trace analysis に粗すぎることもある。player が一つと認識する object が複数 geometry と material submission から成りうる。

26.4 static は unchanging を意味しない

static placement でも animated texture、changing vertex color、condition-dependent material、attached effect を持ちうる。camera が動くと selection や culling も異なりうる。static は resource lifetime や render behavior についてより強い主張を証拠が示すまで、stable placement の意味で使う。

26.5 investigation checklist

各 prop class について visual reference、placement transform、geometry と index range、material と texture binding、state group、nearby repeated instance、distance または time による change を記録する。shared resource と名付ける前に、複数 placement にわたる correlation を探す。最も有用な結果は、visual repetition、data sharing、draw sharing を区別する。

証拠基準。matching appearance は reuse についての仮説である。matching resource identifier と compatible draw evidence は、記録された client と scene に対する reuse を確認できる。

第 27 章 キャラクター：組み立て、スキニング、装備

visible character は一つの mesh ではなく assembly であることが多い。body part、head、hair、equipment、weapon、accessory、effect は別々の geometry、material、transform、draw order を持ちうる。character analysis では player-facing の全体から、documented な renderable component の集合へ進む必要がある。

範囲。本章は一般的な character-rendering concept を説明する。特定のファイナルファンタジー XI character format、skeleton layout、equipment assembly rule を確立しない。

C++クライアント・マイルストーン。 検証済み character component を decode し、equipment、skeleton binding、skinned geometry を assemble する。

```
// Cumulative implementation excerpt: assemble parts and CPU-skin validated influences.
Character assembleCharacter(const CharacterSelection& choice, AssetRepository& assets) {
    Character c;
    c.skeleton = assets.loadSkeleton(choice.skeleton);
    for (ResourceId partId : choice.visibleParts) {
        CharacterPart part = assets.loadCharacterPart(partId);
        validateBonePalette(part, c.skeleton);
        c.parts.push_back(std::move(part));
    }
    return c;
}

Vec3 skinPosition(const SkinnedVertex& v, std::span<const Mat4> bones) {
    Vec3 p{};
    for (const Influence& i : v.influences)
        p += transformPoint(bones.at(i.bone), i.localPosition) * i.weight;
    return p;
}
```

27.1 assembly と attachment

assembly はどの component が character instance に属し、互いにどう配置されるかを定める。一部は skeleton を共有し、別の部分は attachment node に従い、さらに別の部分は parallel に update される independent object になりうる。screen position が一致するだけでは attachment relation を特定できない。transform、update behavior、draw timing を比較する。

27.2 skinning

skinning は skeleton pose に従って mesh を deform する。vertex は一つ以上の bone または node の influence を受け、weight が寄与を定める。final pose は CPU、fixed-function matrix blending、programmable vertex processing、その他の path で evaluate されうる。visible joint motion は deformation または articulated placement の証拠であって、一意の implementation choice ではない。

skinning を調べるには、vertex layout、存在する場合は blend-related input、active transform または constant、shader または fixed-function state、controlled pose sequence を記録する。rigid attachment と deforming surface を分けることで、動く equipment part をすべて skinned と呼ぶ誤りを防ぐ。

27.3 equipment と material boundary

equipment は silhouette、texture binding、material state、draw 数を変えうる。visual slot は必ずしも一つの mesh または texture に対応しない。逆に複数 visual part が一つの compatible submission に結合されることもある。menu や item name だけに頼らず、draw range と resource binding で render boundary を特定する。

27.4 layering と transparency

character には cutout hair、translucent ornament、additive effect、opaque body surface が含まれる。一つの conceptual outfit に由来しても、異なる depth と blending treatment を必要とする。character capture は neighboring draw とともに分析し、attachment、transparency、ordering artifact を geometry fault と取り違えないようにする。

27.5 character evidence record

character state、camera condition、visible equipment set、component draw、buffer と range、texture、material state、transform、animation time、overlap behavior を記録する。一つの equipment または pose だけを変えて capture を繰り返すと、どの draw が stable で、どれが変化した component に属するかを特定できる。

証拠基準。confirmed assembly relation には shared identifier、transform、animation correlation、trace evidence が必要である。similar appearance や familiar equipment category は仮説に過ぎない。

第 28 章 キャラクター組み立てワークスペース

キャラクター描画は組み立てを可視データモデルにすると扱いやすくなる。このワークスペースは種族と体型、装備スロット、構成リソース、スケルトン結合、マテリアル、描画グループ、未解決置換を公開する。

実装成果。

スロット解決、互換性検査、非同期置換、由来情報、プリセットを備えたキャラクター組み立てワークスペースを実装する。

28.1 責務と所有

CharacterSelection は利用者の意図、CharacterRecipe は解決済みリソース識別、CharacterDocument は不変のデコード内容、CharacterInstance はポーズと実行時可視性である。層を分けることでプリセットと診断が信頼できる。

28.2 実装手順

選択変更は影響スロットだけを解決し、共有リソースを再利用し、スケルトン互換性を検証し、描画グループを再構築し、完成インスタンスを原子的にビューポートへ入れ替える。

28.3 障害と回復

部品欠落時はラベル付き空スロットまたは明示的な診断代替を使う。互換性のないパレットは拒否し、見かけ上それらしいポーズへ丸め込まない。

28.4 検証ゲート

テストでは各スロット単独、共有リソース再利用、非互換スケルトン、高速選択変更のキャンセル、プリセット往復、安定スクリーンショット構図を扱う。

28.5 累積 C++実装

C++クライアントマイルストーン。スロット解決、互換性検査、非同期置換、由来情報、プリセットを備えたキャラクター組み立てワークスペースを実装する。

```
// Cumulative implementation listing
CharacterBuildResult CharacterAssembler::build(const CharacterSelection& s,
                                              StopToken stop) {
    CharacterBuildResult out;
    out.recipe = catalog_.resolve(s, out.issues);
    out.skeleton = assets_.loadSkeleton(out.recipe.skeleton, stop);
    for (const ResolvedPart& part : out.recipe.parts) {
        if (stop.stopRequested()) return out;
        auto model = assets_.loadCharacterPart(part.resource, stop);
        if (!paletteFits(model, out.skeleton)) {
            out.issues.push_back({Severity::Error, part.slot,
                                "bone palette is incompatible"});
            continue;
        }
        out.parts.push_back({part.slot, std::move(model), part.source});
    }
    out.complete = !stop.stopRequested() && out.skeleton.valid();
    return out;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 29 章 アニメーション、ポーズ評価、モーションデータ

このクライアント系統におけるアニメーションは、スケルトンのポーズ再生だけではない。キャラクターのクリップ、エフェクト曲線、テクスチャ座標の移動、ジェネレーターの時計、子エフェクトのスケジュールはいずれも時間とともに可視状態を変える。したがって、フィールドを解釈する前に、どの時計で何を動かすのかを明示する必要がある。

証拠範囲。スケルタル形式とジェネレーター曲線は別のリソース系統である。以下の曲線・ジェネレーターに関する記述は DAT 構造、コーパス比較、制御されたパラメータ変更実験に基づく。未解明フィールドは未解明のまま示す。

29.1 クリップ、トラック、複数の時間領域

スケルタルクリップはポーズチャンネルを進める。スケジューラーはエフェクトを放出できる期間を開く。ジェネレーターは放出間隔を持つ。各粒子は独自の年齢と寿命を持ち、曲線は通常 0~1 の正規化年齢を評価する。放出を止めても、すでに生成された粒子が即座に消えるとは限らない。

29.2 ポーズ評価と付着

ポーズ評価はモーション、階層、バインド変換を組み合わせる。結果の行列は剛体部品の配置、スキニング、そしてアクター付着型エフェクトの基準変換に使える。付着エフェクトはバインド姿勢ではなく、現在フレームで評価済みのボーン変換を参照すべきである。

29.3 更新順序

一つのアクター上でキャラクターモーション、装備配置、顔制御、付着エフェクトが共存できる。クリーンルーム実装では、ポーズ評価、付着解決、依存エフェクトのシミュレーション／描画の順に更新する。これは実装上の要件であり、リテール内部のタスク順序そのものを断定するものではない。

29.4 正規化エフェクト曲線

タイプ 0x19 のリソースには、第一成分が正規化時間、第二成分が値となるスカラー対が格納される。ジェネレーター命令は位置、回転、拡大縮小、RGBA、UV、モーフ関連制御に曲線を結び付けられる。隣接点の線形補間は有力な作業モデルだが、参照命令と可視結果が一致した用途だけを確認済みとする。

```

struct CurveKey { float t; float value; };

float sampleCurve(std::span<const CurveKey> keys, float normalizedAge) {
    if (keys.empty()) return 0.0f;
    const float t = std::clamp(normalizedAge, 0.0f, 1.0f);
    auto right = std::lower_bound(keys.begin(), keys.end(), t,
        [](const CurveKey& k, float x) { return k.t < x; });
    if (right == keys.begin()) return right->value;
    if (right == keys.end()) return keys.back().value;
    const CurveKey& b = *right;
    const CurveKey& a = *(right - 1);
    const float u = (b.t == a.t) ? 0.0f : (t - a.t) / (b.t - a.t);
    return std::lerp(a.value, b.value, u);
}

```

29.5 定速変化と曲線変化

形式はフレームごとの定速変化と、参照曲線の両方を表現できる。速度、角速度、拡縮速度、UV スクロールは前者、正規化された位置・回転・拡縮・色・透明度・UV・モーフは後者に属する。すべてをサンプル列へ焼き込まず、両方の表現を保持する。

29.6 実験記録

リソース、命令セクション、オペコード、ペイロード、開始条件、放出期間、粒子寿命、曲線 ID、可視時刻を記録する。一つのフィールド変更で放出頻度だけが変わり、既存粒子の寿命が変わらなければ、それは粒子寿命ではなくジェネレーター時計の証拠となる。

C++クライアント到達点。SkeletonTime、SchedulerTime、GeneratorTime、ParticleAge を別々に保持し、一つのフレームカウンターを流用しない。

```

// Cumulative implementation excerpt: four clocks with explicit ownership.
struct AnimationClocks {
    double skeletonSeconds = 0.0;
    double schedulerSeconds = 0.0;
    std::uint64_t generatorFrame = 0;
};
struct ParticleClock { std::uint32_t ageFrames = 0, lifetimeFrames = 1; };

void advance(AnimationClocks& c, std::span<ParticleClock> particles, double dt) {
    c.skeletonSeconds += dt;
    c.schedulerSeconds += dt;
    ++c.generatorFrame;
    for (ParticleClock& p : particles) ++p.ageFrames;
}

bool alive(const ParticleClock& p) { return p.ageFrames < p.lifetimeFrames; }

```

証拠基準。単調な対の表とそれを参照する命令は曲線関係を示す。補間規則、芸術的用途、リテールの更新順序には追加の挙動試験または実行証拠が必要である。

29.7 minimal skeletal-motion evaluator

supported motion slice は各 transform component の base value と、shared frame-value array への optional channel offset を持つ。Listing 19.1 は 30 frames per second で channel を sample し、quaternion を normalize し、local transform を compose し、parent を child より先に evaluate する。疑わしい sample の repair や incompatible clip の resample は行わない。

Listing 19.1 base-plus-channel pose evaluation と hierarchical composition.

```
struct BoneTrack {
    Vec3 baseTranslation;
    Quaternion baseRotation;
    std::array<std::optional<std::size_t>, 3> translationChannel;
    std::array<std::optional<std::size_t>, 4> rotationChannel;
};

LocalPose sampleTrack(const BoneTrack& track, std::span<const float> samples,
                      std::size_t frame, std::size_t frameCount) {
    LocalPose pose{track.baseTranslation, track.baseRotation};
    for (int axis = 0; axis < 3; ++axis)
        if (track.translationChannel[axis])
            pose.translation[axis] = samples.at(*track.translationChannel[axis] + frame);
    for (int component = 0; component < 4; ++component)
        if (track.rotationChannel[component])
            pose.rotation[component] = samples.at(*track.rotationChannel[component] + frame);
    pose.rotation = normalize(pose.rotation);
    return pose;
}

std::vector<Mat4> evaluateSkeleton(const Skeleton& skeleton,
                                  const Motion& motion, float seconds) {
    if (motion.frameCount == 0 || motion.tracks.size() != skeleton.bones.size())
        throw ParseError("motion and skeleton are incompatible");
    const std::size_t frame = std::min<std::size_t>(
        static_cast<std::size_t>(seconds * 30.0f), motion.frameCount - 1);
    std::vector<Mat4> world(skeleton.bones.size());
    for (std::size_t i = 0; i < skeleton.bones.size(); ++i) {
        const LocalPose local = sampleTrack(motion.tracks[i], motion.samples,
                                             frame, motion.frameCount);
        const Mat4 localMatrix = compose(local.translation, local.rotation);
        const int parent = skeleton.bones[i].parent;
        if (parent >= static_cast<int>(i))
            throw ParseError("bone hierarchy is not parent-first");
        world[i] = parent < 0 ? localMatrix : world.at(parent) * localMatrix;
    }
    return world;
}
```

reconstruction boundary. quaternion hemisphere repair、gap interpolation、head/body compatibility fallback、root-motion removal、clip resampling は有用な viewer policy である。retail-client behavior とせず、untouched decoded sample と比較できる policy として log する。

第 30 章 アニメーションブラウザと再生コントローラ

アニメーション対応にはポーズ評価だけでは足りない。アプリケーションはクリップをカタログ化し、決定論的にプレビューし、時間をスクラブし、ループを選択し、トラックを比較し、欠落または修復されたチャンネルを説明しなければならない。

実装成果。

本章のスケルタル評価器を駆動し、すべての代替処理を公開するアニメーションカタログとトランスポートコントローラを実装する。

30.1 責務と所有

AnimationCatalog は識別とメタデータ、PlaybackController は再生状態、PoseEvaluator はクリップ・スケルトン・時刻に対する純粋評価、AnimationReport は代替処理とソース範囲を保持する。

30.2 実装手順

ブラウザはチャンネル本体より先にメタデータを走査する。再生は計測時間で進み、スクラブは要求された正確な時刻を評価し、比較モードは同じスケルトンとカメラで二つのクリップを評価する。

30.3 障害と回復

長さゼロ、非互換トラック、不正な親順、非有限キーは診断付きで評価を停止する。ループと欠落チャンネルの挙動は隠れた推測ではなく UI に表示する方針である。

30.4 検証ゲート

テストでは境界時刻、ループ折返し、一時停止安定性、スクラブ決定性、階層検証、比較整列、ポーズハッシュを検証する。

30.5 累積 C++実装

C++クライアントマイルストーン。本章のスケルタル評価器を駆動し、すべての代替処理を公開するアニメーションカタログとトランスポートコントローラを実装する。

```
// Cumulative implementation listing
void PlaybackController::update(double elapsedSeconds) {
    if (!state_.playing || state_.duration <= 0.0) return;
    state_.time += elapsedSeconds * state_.rate;
    if (state_.looping)
```

```

        state_.time = positiveModulo(state_.time, state_.duration);
    else if (state_.time >= state_.duration) {
        state_.time = state_.duration;
        state_.playing = false;
    }
    changed_.publish(state_);
}

void PlaybackController::seek(double seconds) {
    state_.time = std::clamp(seconds, 0.0, state_.duration);
    changed_.publish(state_);
}

```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 31 章 ライティングモデル、頂点カラー、マテリアル応答

lighting は scene input と surface に書き込まれる最終 color の関係である。Direct3D 時代の renderer では、この関係に normal、light、material property、vertex color、texture、fog、texture-stage operation が関わりうる。明るくまたは暗い region は観測であり、その背後の lighting model は input と state evidence を必要とする主張である。

範囲。本章は surface shading の分析分類を説明する。ファイナルファンタジー XI の light setup、material equation、特定 draw における vertex color の使用を特定しない。

C++クライアント・マイルストーン。 decoded material、light、vertex color、texture-stage input を検査可能な shading configuration へ変換する。

```

// Cumulative implementation excerpt: one inspectable shading configuration.
struct ShadingConfig {
    Color4 ambient, diffuse;
    Vec3 lightDirection;
    Color4 vertexColor;
    std::array<TextureStageDesc, 2> stages;
    std::uint32_t stageCount;
};

ShadingConfig makeShading(const MaterialRecord& m, const LightState& l,
                          Color4 vertexColor) {
    ShadingConfig s{l.ambient, l.diffuse, normalize(l.direction), vertexColor, {}, 0};
    for (const auto& stage : m.validatedStages) {
        if (s.stageCount == s.stages.size()) break;
        s.stages[s.stageCount++] = stage;
    }
    return s;
}

```

31.1 normal と directional response

normal は多くの lighting calculation で使う surface orientation を与える。normal と light direction の関係が変わると diffuse response が変化しうる。specular-like highlight、ambient term、emissive

contribution は別の behavior を加えうる。同じ visual gradient は texture または vertex color に bake されることもあるため、normal を持つ layout だけでは dynamic lighting を証明しない。

31.2 vertex color

vertex color は vertex に color value を格納し、primitive 全体で interpolate される。precomputed shading、local tint、ambient variation、material mask、lighting input、その他の information を表せる。interpolate が安価なため、追加 texture なしに大きな variation を加えられる。その semantic role は data、state、変化する scene condition との correlation から推論する必要がある。

31.3 material response

material は available input が output color になる方法を制御する。texture を select し、diffuse または vertex color で modulate し、別 contribution を add し、alpha を別に route できる。visible response は lighting state、texture-stage operation、blend state、fog、target content に依存しうる。material name は equation ではない。active input と operation を記録する。

31.4 baked、dynamic、mixed lighting

scene は precomputed color と dynamic light または time-dependent change を組み合わせられる。baked information は texture、vertex color、separate overlay に存在しうる。dynamic information は per vertex で compute され、changing constant で与えられ、moving material で表現されうる。mixed result は real-time graphics で一般的だが、image から presume せず split を示さなければならぬ。

31.5 investigation checklist

調査 surface について、利用可能な normal と color attribute、material と texture-stage operation、light-related state または constant、fog、target と blend state、camera、time、nearby condition を controlled に変えた capture を記録する。仮説 input が変わると何がかわるかを問う。stable texture pattern と camera-independent vertex gradient は、documented light parameter に従う response とは異なる結論を支持する。

証拠基準。data 内の vertex color または normal は確認済み attribute である。それらを baked lighting、dynamic lighting、ambient occlusion、その他 semantic role と呼ぶには tested correlation が必要である。

第Ⅳ部 大気、エフェクト、コンテンツツール

第32章 空、天体、時刻表現

空は環境レコードと、通常のゾーン配置表に現れないことがあるジオメトリを組み合わせる。環境レコードはクリア色、フォグ、描画距離、光色、スカイドームのリングを供給し、天体や雲に似たジオメトリはエフェクト系から起動され得る。

証拠範囲。環境レコード配置とカメラ追従ドームのモデルは強く裏付けられる。すべての天体・雲リソースについて、起動条件と最終変換が完全に判明したわけではない。

32.1 環境レコードとスカイドーム

タイプ0x2Fは屋内フラグ、モデル／地形用の光設定、クリア色、描画距離、空半径、8個のリング色、8個の高度位置を持つ。屋外では放射状のグラデーションドームを構成する。クリア直後に深度書き込みを切り、ビュー行列から平行移動を除いて描き、ワールド描画前に状態を戻す。

32.2 時刻と天候の補間

環境ID、天候タグ、時刻から有効レコードを選び、深夜をまたいで前後の時刻レコードを補間する。fogFarが0のときはフォグ無効という離散状態として扱い、小さな正值を経由して補間してはならない。

32.3 ジェネレーター起動型の空レイヤー

ゾーンDATには、配置表に存在しない空・雲・星・太陽・月らしいメッシュが含まれる。初期のパラメータ変更実験と後年のコーパス調査は、ジェネレーターが空や水に関係する環境アセットを起動できる点で一致する。配置表だけを描くレンダラーは、作者が用意した内容を欠落させる。

32.4 天体の向きと深度

天体カードは完全なビルボード、軸固定、カメラ追従など複数の向きを取り得る。名前から推測せず、エフェクトの向きデータから解決する。大きな深度バイアスと深度書き込みなしが報告されているが、資産ごとの正確な状態は命令とパス単位で確認する。

32.5 実装順序

最初に、証拠が強いドームを実装する。次にジェネレーターの参照を解決し、対応済みの天体カードを描く。雲の移動と天候連動は命令と依存関係を追跡できてから追加する。

C++クライアント到達点。環境補間、スカイドーム描画、ジェネレーター実行を別モジュールとし、復号済みリソース ID で接続する。

```
// Cumulative implementation excerpt: environment, dome, and generators stay separate.
struct ZoneAtmosphere {
    EnvironmentInterpolator environments;
    SkyDomeRenderer sky;
    EffectRuntime effects;

    void draw(RenderContext& rc, WeatherTag weather, GameMinute minute) {
        const EnvironmentState state = environments.sample(weather, minute);
        rc.clear(state.clearColor);
        sky.draw(rc, state.sky, removeTranslation(rc.camera().view()));
        rc.applyFog(state.fog);
        effects.activate(state.environmentEffects);
    }
};
```

証拠基準。配置表に無くジェネレーター依存グラフに存在することは起動経路を支持するが、最終変換、向き制約、描画パスまでは証明しない。

32.6 reconstruction laboratory : camera-attached sky

decoded retail sky mesh が未対応の場合、vertex-colored dome は original sky geometry を再現すると主張せずに、environment color、depth order、fog isolation、camera attachment を test できる。dome は camera を追従し、far plane 内に置き、depth write を無効にし、通常 world geometry より前に draw する。

Listing 21.1 environment pipeline test 用と明示した fallback sky。

```
void drawReconstructionSky(IDirect3DDevice8& d, const SkyMesh& dome,
                           const Camera& camera, float farPlane) {
    const Mat4 world = translation(camera.eye) * scale(farPlane * 0.9f);
    setWorld(d, world);
    d.SetRenderState(D3DRS_LIGHTING, FALSE);
    d.SetRenderState(D3DRS_FOGENABLE, FALSE);
    d.SetRenderState(D3DRS_ZWRITEENABLE, FALSE);
    d.SetRenderState(D3DRS_ALPHABLENDENABLE, FALSE);
    drawVertexColoredMesh(d, dome);
    d.SetRenderState(D3DRS_ZWRITEENABLE, TRUE);
}
```

speculation label。generated dome topology と radius は reconstruction choice である。decoded environment color と独立に observed した ordering だけを game-specific evidence へ入れる。

第 33 章 フォグ、遠景ヘイズ、色調整

distance effect は renderer が近くの detail から horizon へ移行する方法を形作る。fog は depth-related または distance-related value に従って fragment を変更できる。haze は fog、atmosphere、display behavior を合わせた visual result を表せる。color grading は material、target、presentation を通じて適用される systematic color change のより広い語である。これらの label は視覚的に重なるが、異なる技術的問いを表す。

範囲。本章は distance と color phenomenon を分ける方法である。ファイナルファンタジー XI の fog equation、parameter、weather control、presentation pipeline を確認しない。

C++クライアント・マイルストーン。 supported fog parameter を parse し、distance haze と scene-color control を runtime で検査可能にする。

```
// Cumulative implementation excerpt: parse and expose linear fog state.
FogState readFog(ByteReader& r) {
    FogState f;
    f.color = decodePackedRgb(r.u32());
    f.farDistance = r.f32();
    f.nearDistance = r.f32();
    f.enabled = f.farDistance != 0.0f;
    return f;
}

void applyFog(IDirect3DDevice8& d, const FogState& f) {
    d.SetRenderState(D3DRS_FOGENABLE, f.enabled);
    if (!f.enabled) return;
    d.SetRenderState(D3DRS_FOGTABLEMODE, D3DFOG_LINEAR);
    d.SetRenderState(D3DRS_FOGCOLOR, toD3dColor(f.color));
    d.SetRenderState(D3DRS_FOGSTART, std::bit_cast<DWORD>(f.nearDistance));
    d.SetRenderState(D3DRS_FOGEND, std::bit_cast<DWORD>(f.farDistance));
}
```

33.1 draw-state question としての fog

fixed-function context では fog setting を draw に対して active にし、fog color を draw の computed fragment result と combine できる。mode と parameter は transition が linear、exponential-like、その他の形に見えるかへ影響する。fog は一つの draw group で enabled、別 group で disabled になりうるため、world image には fogged element と unfogged element の両方が含まれうる。

33.2 haze は観測である

haze は distance での reduced contrast、shifted color、detail loss を表す。API fog、blended overlay、texture choice、sky coordination、light-like change、display scaling、複数 mechanism か

ら生じうる。distant fade を『fog』と呼ぶのは有用な short hand であることが多いが、技術的主張にはそれを支える state または composition evidence が必要である。

33.3 scene 全体の color change

scene-wide color shift は changing fog color、clear color、vertex color、material parameter、texture selection、render-target pass、final display conversion から生じうる。change の scope は有用な clue である。interface element が world と共に変わるなら late presentation effect がより plausible になり、選ばれた world draw だけが変わるなら material または fog state がより plausible になる。trace evidence なしにどちらも conclusive ではない。

33.4 controlled distance study

測定した複数 distance の landmark を選び、fixed camera direction、time、weather、display condition で capture する。color、contrast、alpha-like fade、draw presence、depth behavior を比較する。次に一度に一つの condition を変える。measured series は effect の equation または source を明らかにする前に distance relationship を確立できる。

33.5 evidence record

利用可能なら fog enablement と mode、fog color と parameter、depth state、material と texture stage、target sequence、background color、affected draw class、controlled image series を記録する。effect が world distance、camera rotation、time、weather、platform で変わるかを注記する。これにより confirmed state change と inferred atmospheric role を分けられる。

証拠基準。documented fog state はその draw について確認済みである。
'haze'、'grading'、'atmosphere' は measured visual と state evidence に結び付けるべき解釈である。

第 34 章 水、反射、表面の動き

FFXI のゾーン内の水は、単一の専用水システムではなく、通常の描画機構を使う作者定義コンテンツとして扱うのが最も妥当である。配置ジオメトリとして存在する面もあれば、環境ジェネレーターから起動される面もあり、UV 変化、曲線、重ねたジオメトリ、粒子的要素によって動く。

証拠範囲。配置された水面、通常の透明パス状態、ジェネレーターによる UV 移動は裏付けられる。画面空間反射、屈折、専用水シェーダーは確認されていない。

34.1 二つのコンテンツ経路

第一は配置データから参照され、透明マテリアル群で描かれる通常のゾーンジオメトリである。第二はジェネレーター起動型のジオメトリまたは粒子で、未配置の海面、滝、噴水、しぶき、環境レイヤーを含む。両方を実装する前にゾーン欠落と断定してはならない。

34.2 表面の動き

ジェネレーター命令の U/V 定数増分は、更新ごとにテクスチャ座標をスクロールできる。別の曲線参照は粒子の正規化年齢に対して U/V を変化させる。UV 移動はジオメトリを変形せずサンプリング位置だけを変える。

34.3 透明度と深度

透明なゾーングループは送信元アルファ／送信元アルファ反転のブレンド、深度書き込み無効、LessEqual 比較、深度バイアスを用いて同一平面の重ねを保つ。岸辺のフェードは頂点アルファが担うことが多い。ローダーが 0x80 を 1.0 へ正規化済みなら、固定機能のアルファ補償を二重適用してはならない。

34.4 反射らしい外観

空色のテクスチャ、頂点色、アルファ層、時刻ライティングは反射画像を参照しなくても反射らしく見える。レンダーターゲットまたは環境マップ依存が確認されるまでは「反射らしい」と表現する。キャラクターのキューブマップ経路が水にも使われるとは限らない。

34.5 描画ではない水関連データ

衝突面の地形コードは浅瀬／深水を足音や水しぶき選択に使えるが、水面を描かない。河川や海岸に関連する経路データも、移動や位置音響に使われるだけで描画水面ではない場合がある。

34.6 C++統合

```
void drawWaterAndOverlays(RenderContext& rc, const Scene& scene) {  
    rc.setDepth(Compare::Less, true);  
    drawOpaqueAndCutout(rc, scene);  
}
```

```

rc.setBlend(Blend::SrcAlpha, Blend::InvSrcAlpha);
rc.setDepth(Compare::LessEqual, false);
rc.setDepthBias(scene.zoneOverlayBias);
drawTransparentZoneGroups(rc, scene);

rc.disableBlend();
rc.setDepth(Compare::Less, true);
rc.setDepthBias(0);
}

```

証拠基準。配置、ジェネレーター参照、UV 命令、描画状態は個別に確認できる。それらを根拠のない専用水レンダラーという一語にまとめてはならない。

34.7 reconstruction laboratory : water subsystem を仮定しない

practical client は、まず placed translucent geometry を通常 zone pipeline で描画する。その後 texture-coordinate motion を per-material reconstruction policy として有効にする。water に似た object name を distinct format の証明として扱ってはならない。Listing 10.1 の state packet を使い、blending on、depth write off、explicit culling、decoded material record から分離した scroll を test する。

acceptance test。camera を固定し、opaque/no motion、blended/no motion、blended/motion、blended/motion with reversed draw order の四 capture を比較する。各 visible difference を説明した変更を記録する。reflection、refraction、cubemap response、screen-space effect は独立 evidence が得られるまで unsupported である。

第 35 章 天候：雨、雪、風、環境パーティクル

天候は三層の協調である。有効な天候 ID、その天候と時刻に対応する環境外観、そして降水などの任意の視覚ジェネレーターである。三層を分けることで、通信フィールド、フォグレコード、粒子エミッターを一つの構造と誤認せずに済む。

証拠範囲。天候 ID と環境レコードは、完全な降水ランタイムより強く裏付けられる。降水は復号済みフィールド、制御実験、再構築が混在する。

35.1 天候 ID と遷移

有効 ID は環境レコード解決に使う 4 文字の天候タグを選ぶ。遷移時間は天候状態に属し、粒子寿命ではない。同じ天候 ID を受け取っても、必要なジェネレーター資産が無いゾーンでは雨雪が表示されない可能性がある。

35.2 環境外観

有効天候について、前後の時刻にあるタイプ 0x2F レコードを補間し、クリア色、ドーム、フォグ、描画距離、光入力を更新する。降水ジオメトリが無くても、これらは天候に伴う大気変化となる。

35.3 降水ジェネレーター

降水はタイプ 0x05 ジェネレーター系と特殊なバッチ経路に関連する。放出間隔、数、範囲または位置揺らぎ、寿命、移動、色、拡張、曲線参照などが復号されている。ただし、リテールの正確なバッチ処理と向き処理を完全再現できる段階ではない。

35.4 複数系統にまたがる風

風は降水方向と植生の第二位置ブレンドに影響し得るが、同じタイミングだから同じ保存フィールドとは限らない。粒子速度、植生のグローバル係数、UV 移動、音を別入力として保持する。

35.5 カメラ近傍の暫定実装

クリーンルームクライアントの初期段階では、天候 ID で駆動するカメラ近傍エミッターとして降水を実装できる。ただしこれは安定した視界内密度を得る再構築であり、リテールのバッチ変換経路そのものではないと明示する。

35.6 検証

一つのゾーンでカメラ、時刻、天候を固定し、ジェネレーター ID、放出間隔、生存粒子数、寿命分布、範囲、ブレンド／深度状態、環境値を記録する。次に同じ天候で資産構成が異なるゾーンを比較し、起動条件を切り分ける。

C++クライアント到達点。WeatherState、EnvironmentState、PrecipitationRuntime を別オブジェクトとし、天候タグと復号済み参照で接続する。

```
// Cumulative implementation excerpt: three weather layers connected by explicit tags.
struct WeatherState { WeatherTag active; float transition = 1.0f; };
struct PrecipitationRuntime { void update(WeatherTag, float dt); void draw(RenderContext&); };

class WeatherSystem {
    WeatherState weather_;
    EnvironmentDatabase environments_;
    PrecipitationRuntime precipitation_;
public:
    EnvironmentState update(GameMinute minute, float dt) {
        precipitation_.update(weather_.active, dt);
        return environments_.sample(weather_.active, minute, weather_.transition);
    }
}
```

```
void drawParticles(RenderContext& rc) { precipitation_.draw(rc); }
};
```

証拠基準。天候と相関する粒子は観察である。ジェネレーター ID と復号・挙動試験済みフィールドが揃えばエミッターモデルを支持できるが、正確なバッチ、ゲート、パス所有者は未解決である。

35.7 reconstruction laboratory : deterministic camera-local precipitation

generator format が未 decode の場合、camera-local precipitation は有用な fallback になる。ただし deterministic、bounded、substitute と明示する。Listing 24.1 は seed から repeatable position を生成し、camera-centered volume 内で particle を wrap する。この distribution を retail client が用いるとは主張しない。

Listing 24.1 pipeline test 用 deterministic fallback precipitation.

```
struct ReconstructionPrecipitation {
    std::uint32_t seed;
    std::vector<Vec3> particles;

    void rebuild(std::size_t count, const Camera& camera, float extent) {
        DeterministicRandom random(seed);
        particles.resize(count);
        for (Vec3& p : particles)
            p = camera.eye + Vec3{random.signedUnit() * extent,
                                   -60.0f + random.unit() * 85.0f,
                                   random.signedUnit() * extent};
    }
    void update(float dt, const Camera& camera, float speed) {
        for (Vec3& p : particles) {
            p.y += speed * dt;
            if (p.y > camera.eye.y + 25.0f) p.y -= 85.0f;
        }
    }
};
```

weather identity、environment color transition、wind、generator activation、fallback precipitation を別 record として保持する。五つを一つの weather enum へ置き換えると、後から client を改善するための evidence が失われる。

第 36 章 環境制御と比較ワークスペース

空、フォグ、水、天候は、制御が同期し再現可能になって初めて有用な研究対象になる。このワークスペースはデコード済み環境値、再構成方針、時刻・天候遷移、並列または差分比較を公開する。

実装成果。

デコード状態と再構成方針を分離したまま編集、比較、キャプチャ、報告できる環境ワークスペースを実装する。

36.1 責務と所有

EnvironmentState はデコード済み入力、EnvironmentPolicy は明示選択された再構成挙動、EnvironmentRuntime は補間値を持つ。ComparisonPreset はカメラ、時刻、天候、方針、表示設定を記録する。

36.2 実装手順

コントロールは保留状態を編集し、検証がランタイムスナップショットを生成し、比較する二つのペインは同じシーンスナップショットと別版の環境方針を使う。

36.3 障害と回復

未知ソース値は可視のまま変更しない。未対応天候や水表現は該当制御だけを無効化する。手動近似はキャプチャとレポートで再構成と明記する。

36.4 検証ゲート

テストではプリセット再現、補間端点比較、決定論的降水の固定、同一カメラ確認、方針ラベルの書き出しレポート反映を検証する。

36.5 累積 C++実装

C++クライアントマイルストーン。デコード状態と再構成方針を分離したまま編集、比較、キャプチャ、報告できる環境ワークスペースを実装する。

```
// Cumulative implementation listing
EnvironmentSnapshot EnvironmentController::commit() const {
    EnvironmentSnapshot out;
    out.source = pending_.source;
    out.policy = pending_.policy;
    out.time = wrapHours(pending_.time);
    out.weather = catalog_.contains(pending_.weather)
        ? pending_.weather : WeatherId::Clear;
    out.fog = interpolateFog(pending_.source, out.time, out.weather);
    out.sky = interpolateSky(pending_.source, out.time, out.weather);
    out.label = out.policy.isReconstruction
        ? "reconstruction" : "evidence-scoped";
    return out;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 37 章 パーティクルシステム、軌跡、魔法エフェクト

FFXI のエフェクトは小さなプログラムである。タイプ 0x05 ジェネレーターがリソースを選び、付着、放出、移動、向き、拡張、色、曲線、UV、描画分類、音／光らしい処理、子ジェネレーターの命令を適用する。タイプ 0x19 曲線と複数のジオメトリ系統が入力となるため、クライアントには境界検査付きパーサーと決定論的シミュレーションランタイムの両方が必要である。

証拠範囲。命令フレーミング、多数のペイロード長、リソース関係、曲線参照、UV スクロール、複数の挙動は DAT 調査と制御されたパラメータ変更で裏付けられる。オペコードの意味はセクションに依存し、未解明の命令・フラグ・付着意味・演算順序が残る。

37.1 リソース系統

作業モデルのエフェクトグラフには、タイプ 0x05 ジェネレーター、0x19 曲線、0x1F 展開三角形、0x21 カード／スプライト、0x25 モーフ、テクスチャ、任意の音・光、子ジェネレーター参照が含まれる。一つの魔法が一つのメッシュではなく、複数ジェネレーターをスケジュールすることもある。

37.2 ジェネレーターのフレーミング

共通ヘッダーはおよそ 0x80 バイトで、+0x70 付近から単調増加する四つのセクション境界を持つ。命令はオペコード、ペイロード長、予約 2 バイトからなる 4 バイト接頭辞とペイロードで構成される。破損後にそれらしいオペコードを検索せず、全境界を検証する。

```
struct EffectCommandView {
    std::uint8_t section;
    std::uint8_t opcode;
    std::span<const std::byte> payload;
    std::uint64_t sourceOffset;
};

std::vector<EffectCommandView> parseCommandSection(
    ByteReader& r, std::uint8_t section, std::size_t end) {
    std::vector<EffectCommandView> out;
    while (r.position() < end) {
        const auto commandOffset = r.position();
        const auto opcode = r.u8();
        const auto payloadSize = r.u8();
        const auto reserved = r.u16le();
        if (opcode == 0) break;
        if (reserved != 0) r.noteUnknown(commandOffset + 2, reserved);
        if (r.position() + payloadSize > end)
            throw ParseError("effect command crosses section boundary");
        out.push_back({section, opcode, r.bytes(payloadSize), commandOffset});
    }
    if (r.position() > end) throw ParseError("effect section overrun");
    return out;
}
```

37.3 セクション依存オペコード

数値だけでは命令を一意に識別できない。主要パラメータ部で拡張曲線を示す番号が、別セクションでは U/V 定速増分を示すことがある。{section, opcode}の組でディスパッチし、未対応の組は生ペイロードとソースオフセットを保存する。

```
void decode(const EffectCommandView& c, EffectProgram& out) {
    switch ((std::uint16_t(c.section) << 8) | c.opcode) {
        case 0x0202: decodeVelocity(c.payload, out); break;
        case 0x020B: decodeAngularVelocity(c.payload, out); break;
        case 0x022E: decodeUCurve(c.payload, out); break;
        case 0x0327: decodeConstantUScroll(c.payload, out); break;
        default: out.unknownCommands.push_back(copyRaw(c)); break;
    }
}
```

37.4 初期リソースと粒子状態

初期リソース命令は参照アセット、ローカル位置、リソース系統選択、寿命、向き関連値を含む。対象にはエフェクト三角形、カード、モーフ、音らしいレコード、ポイントライトらしいレコード、ゾーンジオメトリが観察される。参照を解決し、選択された型と実体の型が一致してから描画物を生成する。

37.5 付着と向き

制御実験では付着値が腰、胸、頭、手、足、手首、身体の前左右など再現可能な位置と対応した。アクター相対の付着表を強く支持し、ボーン接続の可能性も高いが、普遍的なボーン番号式は未確定である。生値を保持し、試験済みの項目だけを写像する。

向きには非拘束、完全ビルボード、軸固定、移動方向追従、画面貼り付きが観察される。付着位置と粒子移動の後に向きを解決し、最終行列を作る。全カードに同じビルボード行列を適用してはならない。

37.6 放出間隔と寿命

間隔フィールドは放出間の待ちを制御する。有力な解釈は「1回の放出に要するフレーム数-1」で、保存値0は毎フレーム、1は2フレームごととなる。放出数、粒子寿命、スケジューラーの放出期間はそれぞれ別である。

```
struct EmitterClock {
    std::uint32_t periodFrames = 1;
    std::uint32_t accumulator = 0;
    bool tick() {
```

```

        if (++accumulator < periodFrames) return false;
        accumulator = 0;
        return true;
    }
};

EmitterClock makeClock(std::uint16_t storedInterval) {
    return EmitterClock{std::uint32_t(storedInterval) + 1u, 0u};
}

```

37.7 発生形状と移動

確認された命令群には定速移動、速度揺らぎ、球面発生、体積または目標方向らしい発生、半径方向の内向き／外向き速度とその揺らぎがある。初期回転、回転揺らぎ、角速度、角速度揺らぎも並行した群を作る。乱数はインスタンスごとの決定論的シードで再現可能にする。

```

struct ParticleState {
    Vec3 position{};
    Vec3 velocity{};
    Vec3 euler{};
    Vec3 angularVelocity{};
    Vec3 scale{1, 1, 1};
    Vec3 scaleVelocity{};
    Color4 color{1, 1, 1, 1};
    Vec2 uvOffset{};
    float ageFrames = 0;
    float lifetimeFrames = 1;
};

void integrate(ParticleState& p) {
    p.position += p.velocity;
    p.euler += p.angularVelocity;
    p.scale += p.scaleVelocity;
    p.ageFrames += 1.0f;
}

```

37.8 拡縮、色、テクスチャ座標

初期拡縮、軸別／一様な拡縮揺らぎ、拡縮速度、初期色、色揺らぎ、曲線による拡縮・RGBA を表現できる。U/V 定速増分と U/V 曲線はジオメトリと独立してサンプリング位置を動かす。古い記録には赤／青ラベルの不整合があるため、生バイトと赤青を区別できる試験で色順を決める。

37.9 曲線とモーフ制御

位置、回転、拡縮、赤、緑、青、アルファ、U、V は個別に正規化曲線を参照できる。モーフジオメトリと相関する追加の曲線対もあるが、正確な意味と適用順序は未完である。適用できない参照も削除せず保持する。

37.10 描画分類、深度、順序

観察された分類には加算、通常アルファ、逆減算／暗化、ゼロ・送信元反転らしい方式、不透明がある。別フラグが深度試験、深度書き込み、カメラ／ワールド空間、固定挙動に影響する。古い実験は大分類を示すが最終ビット配置は不完全なので、新しいパス証拠を優先し未知ビットを診断に残す。

柔らかな勾配を持つ粒子へ一律にアルファテストを継承してはならない。ブレンド、アルファテスト、深度試験、深度書き込み、カリング、ソートは別々の状態である。すべての制約が一致するときだけバッチ化する。

37.11 子ジェネレーターとライフサイクル

複数の命令が別ジェネレーターを参照する。子は親の位置を継承し、親の拡張や回転の影響も受け得る。生成時、親終了時、粒子終了時など命令によってトリガーが違ふ可能性があるため、参照を即時再帰生成せずイベント辺として表現する。

```
enum class EffectEvent { OnStart, OnEmit, OnParticleDeath, OnDrain, OnStop };
struct ChildEdge { EffectEvent event; ResourceId child; TransformInheritance inheritance; };

void dispatchEvent(EffectRuntime& rt, EffectInstance& parent, EffectEvent e) {
    for (const ChildEdge& edge : parent.program.childEdges)
        if (edge.event == e) rt.spawnChild(parent, edge);
}
```

37.12 決定論的粒子プール

復号済みプログラムは不変、インスタンスは可変とする。各インスタンスは時計、決定論的乱数状態、生存粒子ハンドル、子イベント状態を持つ。中央プールで無制限な確保を防ぎ、最大生存数を観測可能にする。更新順はスケジューラー、放出、積分、曲線評価、死亡イベント、描画リスト構築とする。

37.13 回帰コーパス

多数の魔法リソースと使用命令番号を対応付けた履歴表は、意味の証明ではなく試験コーパスとして有用である。一つの候補命令だけを持つ最小例、命令の大半を共有する同系統、珍しい命令を使う例を選び、境界超過ゼロ、未知命令保存、決定論的粒子数、依存関係の有界性を検証する。

```
TEST(EffectRuntime, IsDeterministic) {
    auto a = runFixture("effect_minimal.dat", 600, 0x12345678);
    auto b = runFixture("effect_minimal.dat", 600, 0x12345678);
    EXPECT_EQ(a.frameHashes, b.frameHashes);
}
```

37.14 未解決事項

付着値とボーンの完全対応、ビルボードフィールド、全ブレンド／深度ビット、複数の発生形状、モーフ曲線、音・光パラメータ、子トリガー、カード群選択、降水バッチ、元クライアントの更新順序が未解決である。未対応命令はパース時に消してはならない。

C++クライアント到達点。任意フレームで停止し、ソースオフセット、復号命令、未知ペイロード、生存粒子、曲線値、子イベント、各描画の完全な状態記述を表示できるようにする。

```
// Cumulative implementation excerpt: pauseable, inspectable effect simulation.
class EffectInspector {
    EffectRuntime runtime_;
    bool paused_ = false;
public:
    void setPaused(bool value) { paused_ = value; }
    void step() { runtime_.simulateOneFrame(); }
    void update() { if (!paused_) step(); }
    InspectorSnapshot snapshot() const {
        return {runtime_.frame(), runtime_.decodedCommands(),
                runtime_.unknownCommands(), runtime_.particles(),
                runtime_.curveSamples(), runtime_.childEvents(),
                runtime_.drawDescriptors()};
    }
};
```

公開基準。生バイト、セクション、ペイロード、制御試験または実行トレースが一致するときだけ命令を確認済みとする。構造的一致の反復は「強く支持」、見た目だけの命名は「推測」とする。

第 38 章 エフェクトブラウザとシミュレーションワークスペース

エフェクトワークスペースはセクション化されたコマンド列を検査可能なシミュレーションへ変える。コマンド台帳、依存グラフ、決定論的シード、再生制御、生存パーティクル表、カーブ表示、描画記述子、未知命令を統合する。

実装成果。

一つのタイムラインで命令、シミュレーション、描画、証拠ビューを駆動する決定論的エフェクトワークスペースを実装する。

38.1 責務と所有

EffectDocument は不変の解析データ、EffectInstance はシミュレーション状態、EffectPlayback は時刻とシード、EffectTrace は生成・更新・子生成・描画イベントとソースコマンド ID を保持する。

38.2 実装手順

オープン時にはシミュレーション前に台帳と依存関係を構築する。シークはチェックポイントへ戻って決定論的に再生する。一時停止は生成と更新を止め、ステップは設定済みティックだけ進める。

38.3 障害と回復

未知命令は順序を保ち、担当する挙動だけを停止し得る。プール枯渇、不正カーブ、子生成循環、リソース欠落は黙って消えるのではなくトレースイベントを生成する。

38.4 検証ゲート

テストでは決定論的パーティクル状態、シーク再生同値、子循環制限、ソートキー、未知命令保持、時刻別スクリーンショットハッシュを検証する。

38.5 累積 C++実装

C++クライアントマイルストーン。一つのタイムラインで命令、シミュレーション、描画、証拠ビューを駆動する決定論的エフェクトワークスペースを実装する。

```
// Cumulative implementation listing
void EffectWorkspace::seek(double target) {
    const SimulationCheckpoint& cp = checkpoints_.nearestAtOrBefore(target);
    instance_.restore(cp.state);
    trace_.truncateAfter(cp.time);
    double t = cp.time;
    while (t + tick_ <= target) {
        instance_.step(tick_, trace_);
        t += tick_;
    }
    if (t < target) instance_.step(target - t, trace_);
    playback_.setTime(target);
    refreshViews(instance_, trace_);
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 39 章 透明表現、ソート、マルチパスエフェクト

transparency は blend setting だけではない。geometry、material、target、pass にまたがる ordering problem である。renderer はどの opaque result が destination を作るか、どの transparent draw を group できるか、どれを sort するか、later pass が earlier result を reuse または replace するかを決め

なければならない。multi-pass effect は layering、accumulation、order-sensitive artifact を通じてこれらの decision を visible にする。

範囲。本章は composition problem と evidence method を説明する。ファイナルファンタジー XI の transparent-pass ordering、sorting algorithm、render-target workflow を確認しない。

C++クライアント・マイルストーン。 supported pass record を transparent queue、target change、sorting policy、multi-pass composition へ変換する。

```
// Cumulative implementation excerpt: decoded pass records drive targets and queues.
void executePasses(RenderGraph& graph, RenderContext& rc) {
    for (const PassRecord& pass : graph.passes()) {
        rc.setRenderTarget(graph.target(pass.target));
        if (pass.clear) rc.clear(pass.clearColor);
        auto draws = graph.draws(pass.drawRange);
        if (pass.sort == SortPolicy::BackToFront)
            std::stable_sort(draws.begin(), draws.end(), fartherFirst);
        rc.setDepth(pass.depthCompare, pass.depthWrite);
        rc.setBlend(pass.sourceBlend, pass.destinationBlend);
        for (const Draw& draw : draws) rc.draw(draw);
    }
}
```

39.1 sorting が難しい理由

blended geometry には meaningful な destination color が必要だが、二つの translucent object は surface 全体で異なる depth relationship を持ちうる。object center、draw group、particle で sort するか、まったく sort しないかで異なる approximation を生む。一部 effect はこれを許容し、別の effect は special geometry、depth rule、multiple pass で避ける。visual artifact は ordering constraint を示せるが、選択した algorithm を単独では特定しない。

39.2 opaque foundation と transparent phase

一般的な conceptual structure は reliable な depth を書く opaque foundation の後に、異なる depth-write behavior を持つ cutout と blended phase が続く形である。actual frame には sky、water-like surface、particle、character component、interface layer、off-screen target がそれぞれの ordering need で複数 interrupt することがある。trace evidence が transition を示すまで phase name を仮説として扱う。

39.3 multi-pass composition

一つの visible effect は同じ geometry への複数 pass、または異なる layer で assemble されうる。base material の後に glow、mask、distortion-like layer、depth-aware overlay が続くことがある。pass は

main target または intermediate surface へ write できる。multi-pass effect を confirm するには repeated geometry または linked target flow の evidence が必要であり、complex-looking texture だけでは足りない。

39.4 artifact の diagnosis

missing translucent region、incorrect overlap、bright accumulation、halo edge、self-intersection は blend factor、depth write、culling、target content、sorting、capture timing から生じうる。camera と effect state を stable に保ち、adjacent draw と state を比較して diagnose する。まず visible symptom に名前を付け、次に説明できる最小 mechanism 集合を test する。

39.5 pass ledger

suspected pass ごとに target、clear state、draw order、geometry range、texture、stage または shader、blend と depth state、culling、後に変わる image contribution を記録する。resource flow、repeated geometry、controlled visual dependence が relation を支えるときだけ pass を link する。この ledger は descriptive effect name が fabricated render graph になることを防ぐ。

証拠基準。ordered trace event と target binding は pass sequence を confirm できる。その sequence を transparency sorter、post-process、named effect pipeline と呼ぶには additional implementation または behavioral evidence が必要である。

第 40 章 影、デカル、投影表現

shadow、surface 上の mark、projected pattern は単純に見えても非常に異なる technique から生じうる。dark region は textured quad、projected texture、separate mesh、vertex color、blended overlay、render-target result、lighting response になりうる。decal-like mark は geometry に従い、surface の少し上に浮き、base texture に bake され、presentation layer からのみ現れることがある。

範囲。本章は classification と testing guide である。ファイナルファンタジー XI の shadow method、decal system、projection implementation を確認しない。

C++クライアント・マイルストーン。 supported projected-visual record を交換可能な shadow、decal、projector interface の背後で decode する。


```
// Cumulative implementation excerpt: interchangeable projected-visual decoders.
class ProjectedVisual {
public:
    virtual ~ProjectedVisual() = default;
    virtual void submit(const Scene&, DrawList&) const = 0;
};
class BlobShadow final : public ProjectedVisual {
    ShadowRecord record_;
public:
    explicit BlobShadow(ShadowRecord r) : record_(r) {}
    void submit(const Scene& scene, DrawList& out) const override {
        out.add(projectBlob(record_, scene.receivers(), DepthWrite::Off));
    }
};

std::unique_ptr<ProjectedVisual> decodeProjected(const RecordView& r) {
    if (r.kind() == RecordKind::Shadow) return std::make_unique<BlobShadow>(readShadow(r));
    if (r.kind() == RecordKind::Decal) return std::make_unique<Decal>(readDecal(r));
    throw UnsupportedRecord(r.kind());
}
```

40.1 shadow-like result

shadow-like effect はまず behavior で特定する。どこに現れるか、object と共に動くか、nearby surface に conform するか、light または time で変わるか、他 content とどう overlap するかである。これらの observations は world-attached mark と screen-space darkening を区別できるが、geometry または calculation はまだ特定しない。

40.2 decal と offset geometry

decal は receiver への texture projection、surface 近くに置く small mesh、additional material pass、base texture へ baked された data として implementation できる。offset geometry は angle によって receiver から visible に離れる代わりに depth conflict を避けられる。parallax、clipping、draw order、depth bias-like behavior、mark が material または object boundary を cross するかを観測する。

40.3 projected coordinate

projection は projector、light-like source、camera に関係する transform を使って image を map する。changing projected pattern は source transform と receiver geometry に依存する。capture では ordinary texture transform に似ることがある。projection を確立するには coordinate-generation state または shader behavior、source transform、receiver draw、source と receiver の change に pattern がどう respond するかを記録する。

40.4 target-based と composited effect

一部の apparent mark または shadow は intermediate surface へ render し後で blend して生じうる。この場合 source scene data と final receiver の relation は一つの draw で見えないかもしれない。target change、linked texture binding、full-screen または receiver-like pass、opaque geometry と interface layer に対する ordering を探す。

40.5 investigation checklist

object position、receiver geometry、camera angle、time、nearby occluder を controlled に変えて effect を capture する。related draw、target、depth と blend state、culling、texture、coordinate transform、mark が receiver draw の前後どちらに present かを記録する。まず behavior を記述し、次に最小 confirmed chain で classify する。

証拠基準。persistent visible mark は観測である。projected texture、decals、blob shadow、target-based shadow は resource、geometry、state、ordering evidence を必要とする implementation claim である。

第 41 章 インターフェース描画と 2D 合成

interface は独自の geometry、texture、ordering、coordinate system、readability requirement を持つ rendered layer である。world rendering 後に draw され、presentation pass と interleave され、separate target を通じて compose されうる。その見かけの flatness は特定 implementation を証明しないが、UI element は frame 終盤を再構成する際の有用な marker になる。

範囲。本章は 2D composition の分析方法を与える。ファイナルファンタジー XI の UI renderer、target layout、coordinate system、update order を確認しない。

C++クライアント・マイルストーン。 supported interface record を parse し、evidence-scoped 2D layer を compose する orthographic renderer を作る。

```
// Cumulative implementation excerpt: decoded rectangles become orthographic draws.
void UiRenderer::draw(const UiDocument& ui, IDirect3DDevice8& device,
    std::uint32_t width, std::uint32_t height) {
    const Mat4 projection = orthoOffCenterLH(0, float(width),
        float(height), 0, 0, 1);
    const D3DMATRIX d3dProjection = projection.d3d();
    device.SetTransform(D3DTS_PROJECTION, &d3dProjection);
    device.SetRenderState(D3DRS_ZENABLE, FALSE);
    for (const UiLayer& layer : ui.layersInOrder()) {
        setScissor(device, layer.clip);
```

```
        for (const UiSprite& s : layer.sprites)
            drawTexturedQuad(device, s.destination, s.uv, textureCache_.get(s.texture));
    }
```

41.1 screen space と world space

world-space draw は scene transform と camera projection で position される。interface element は通常、pixel、normalized dimension、anchor、その他 layout rule に従う screen-oriented coordinate system に position される。object に attached して見える marker も world-to-screen calculation 後に screen-composite されることがある。space を割り当てる前に camera、resolution、object motion の dependency を test する。

41.2 quad、atlas、layer

多くの 2D interface は textured quad から assemble され、しばしば larger texture atlas の region を使う。panel、border、icon、cursor、meter、simple effect は異なる source rectangle と draw order を使いながら texture を share できる。matching artwork は shared atlas の証拠ではない。resource dimension、coordinate、binding が関連する evidence を与える。

41.3 ordering と depth

UI layer は depth testing を disable または alter し、alpha blending を使い、stacking を draw order に頼ることがある。background panel、text、highlight、cursor、modal overlay は distinct blend と texture state を持つ複数 draw になりうる。interface element には nameplate や targeting indicator のような world-aware part も含まれ、pure screen-space composition から注意深く分ける必要がある。

41.4 resolution と layout behavior

interface placement は resolution、aspect ratio、safe area、window mode、scaling、localization に respond する。一つの display configuration の capture は layout rule を定めない。documented configuration 間で同じ state を比較し、anchor movement、element dimension、cropping、texture sampling change を記録する。

41.5 investigation checklist

UI draw が起きる frame position、target と depth state、transform または coordinate setup、vertex layout、texture binding、blend factor、draw order、controlled resolution と camera change での behavior を記録する。interface element が world-space dependency を share するか confirm した後 にのみ world content を reference として使う。

証拠基準。screen-oriented geometry を持つ late ordered draw は UI-layer 仮説を支持する。specific layout engine、atlas system、update order には resource と implementation evidence が必要である。

第 42 章 文字、アイコン、フォント、ウィンドウレイヤー

text と icon は小さな scale で texture sampling、clipping、blending、layout、ordering を示すため、technical UI question を特に visible にする。window は background、border、text、icon、cursor または highlight、mask-like piece の composition であることが多い。したがって player-facing label としての一つの window を、一つの renderable object と混同してはならない。

範囲。本章は interface element の rendering-oriented vocabulary である。ファイナルファンタジー XI の font resource format、glyph cache、icon atlas、window manager を確認しない。

C++クライアント・マイルストーン。 supported font、glyph、icon、window record を decode し、text layout、atlas、clipping、layering を実装する。

```
// Cumulative implementation excerpt: glyph metrics, atlas lookup, clipping, layering.
std::vector<GlyphQuad> layoutText(const Font& font, std::u32string_view text,
                                   Vec2 origin, Rect clip) {
    std::vector<GlyphQuad> out;
    Vec2 pen = origin;
    for (char32_t ch : text) {
        if (ch == U'\n') { pen.x = origin.x; pen.y += font.lineHeight; continue; }
        const Glyph& g = font.glyph(ch);
        Rect dst{pen.x + g.bearing.x, pen.y - g.bearing.y,
                 float(g.size.x), float(g.size.y)};
        if (intersects(dst, clip)) out.push_back({dst, g.atlasUv, g.atlasPage});
        pen.x += g.advance;
    }
    return out;
}
```

42.1 geometry と texture としての glyph

raster text は通常 glyph または glyph group の textured quad を置いて render される。font resource は individual image、atlas、metric、spacing data、language-dependent mapping を格納できる。

final appearance は filtering、alpha treatment、color、clipping、draw order にも依存する。visible font だけでは resource または layout mechanism を特定しない。

42.2 icon と atlas region

icon は standalone texture または atlas 内の source rectangle を使える。atlas reuse は bind を減らし UI batching を簡単にし、異なる coordinate で異なる image を選ぶ。atlas relation を confirm するには dimension、binding、texture coordinate、neighboring icon draw を比較する。visual style だけでは不十分である。

42.3 window layering と clipping

window は content 前の background と border draw、さらに selection、focus、modal state の overlay を必要としうる。clipping は scissor-like rectangle、geometry mask、texture alpha、separate target で implementation できる。panel edge で text line が消えることは有用な evidence だが、mechanism には state と geometry context が必要である。

42.4 localization と layout

異なる language は glyph selection、string length、line break、window dimension を変えうる。同じ UI state を edition 間で比較できる場合、これらの change は layout behavior の controlled evidence になる。text が precomposed、dynamically laid out、cached のどれかを自動的に明らかにするものではない。resource と draw change も inspect する。

42.5 investigation checklist

UI state、language と display configuration、text または icon content、target と depth state、texture resource、coordinate range、vertex count、filtering、blend state、clipping evidence、draw order を記録する。短い stable string と repeated icon で比較を precise にする。目的は glyph data、layout decision、final composition を分けることである。

証拠基準。textured glyph または icon draw は resource と geometry から confirm できる。font format、cache policy、window hierarchy には additional data または implementation evidence が必要である。

第 43 章 テキスト、ダイアログ、メッセージリソースエクスプローラ

有用なリソースエクスプローラは非視覚的文脈も扱う必要がある。本章では複数レイアウト、バイト保持デコード、制御トークン表示、検索、レコード相互参照、往復安全な書き出しに対応する境界検査付きテキストリソース基盤を構築する。

実装成果。

レイアウト差し替え可能で、トークン対応デコード、検索、由来情報、損失なし診断書き出しを備えたテキストエクスプローラを実装する。

43.1 責務と所有

TextRecord は識別、ソース範囲、生バイト、デコード済みラン、制御、デコード問題を保持する。表示文字列は派生ビューであり、元バイト列を置き換えない。

43.2 実装手順

認識がレイアウトデコーダを選び、文字デコード前にレコード境界を決め、コードページサービスが通常部分を変換し、ゲーム固有制御を型付きトークンにし、ブラウザは正規化表示文字列とトークン名を索引化する。

43.3 障害と回復

不正バイト列は問題範囲を正確に持つ置換ランを生成する。未知制御は 16 進トークンとして残す。書き出しには識別、生バイト、デコード文字列、トークン、問題を含め、証拠を失わない。

43.4 検証ゲート

フィクスチャでは空レコード、不正オフセット、文字と制御の混在、私用字形代替、重複文字列、検索正規化、書き出し再読込同一性を扱う。

43.5 累積 C++実装

C++クライアントマイルストーン。レイアウト差し替え可能で、トークン対応デコード、検索、由来情報、損失なし診断書き出しを備えたテキストエクスプローラを実装する。

```
// Cumulative implementation listing
TextRecord decodeTextRecord(ByteView bytes, SourceRef source,
                             const TextCodec& codec) {
    TextRecord out{source};
    std::size_t begin = 0;
    for (std::size_t i = 0; i < bytes.size(); i) {
        if (!isControlLead(bytes[i])) { ++i; continue; }
        if (begin < i) out.runs.push_back(codec.decode(bytes.subview(begin, i-begin),
                                                         source.subrange(begin, i-begin)));

        ControlParse c = parseControl(bytes.subview(i), source.offset + i);
        out.runs.push_back(c.run);
        i += c.consumed;
        begin = i;
    }
    if (begin < bytes.size())
        out.runs.push_back(codec.decode(bytes.subview(begin), source.subrange(begin)));
    return out;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 44 章 オーディオカタログ、デコード、再生

オーディオリソースにもグラフィックスと同じ証拠規律が必要である。アプリケーションは対応コンテナをカタログ化し、ヘッダを解析し、PCM または対応 ADPCM を所有サンプルヘデコードし、UI を止めずに再生し、未対応コーデックをデコードしたように見せず公開する。

実装成果。

安全なヘッダ解析、非同期デコード、再生制御、ループ、未対応コーデック報告を備えた検索可能なオーディオワークスペースを実装する。

44.1 責務と所有

AudioDocument はコンテナメタデータと由来情報、DecodedAudio は正規化サンプルフレーム、PlaybackVoice はデバイス状態、AudioCatalog は全ペイロードを事前デコードせず検索可能な識別を保持する。

44.2 実装手順

カタログはヘッダだけを読む。オープンするとデコードを予約し、境界付きリングバッファを満たし、進捗を通知する。再生は不変ブロックを消費し、シークは保留ブロックを取消してデコードチェックポイントから再開する。

44.3 障害と回復

未対応コーデックはメタデータを表示し再生を無効にする。不正なレート、チャンネル数、ブロックサイズ、切り詰めペイロードはデバイス送信前に停止する。一時デコードデータは明示的な寿命とサイズ制限を持つ。

44.4 検証ゲート

テストではヘッダ差異、PCM 同一性、ADPCM 基準ベクトル、シーク、ループ境界、キャンセル、デバイス変更、一時キャッシュ退避を扱う。

44.5 累積 C++実装

C++クライアントマイルストーン。安全なヘッダ解析、非同期デコード、再生制御、ループ、未対応コーデック報告を備えた検索可能なオーディオワークスペースを実装する。

```
// Cumulative implementation listing
void AudioWorkspace::play(ResourceId id) {
    stop();
    document_ = audio_.readHeader(id);
    if (!audio_.supports(document_>codec)) {
        status_ = "preview unavailable for this codec";
        return;
    }
    decodeJob_ = jobs_.start([this, doc = *document_](StopToken stop) {
        auto decoder = audio_.createDecoder(doc);
        while (!stop.stopRequested()) {
            SampleBlock block = decoder->next(4096);
            if (block.empty()) break;
            if (!queue_.push(std::move(block), stop)) break;
        }
        queue_.finish();
    });
    voice_.start(queue_, document_>sampleRate, document_>channels);
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 V 部 デスクトップアプリケーション、表示、性能

第 45 章 解像度、アスペクト比、表示スケーリング

renderer が生成した image は最終的に特定の size と aspect relationship を持つ display surface へ map される。resolution、aspect ratio、viewport、letterboxing または cropping、scaling、filtering、color conversion は world geometry または material を変えずに final appearance を変えられる。technical comparison では scene と同じ注意で presentation path を記述しなければならない。

範囲。本章は display-analysis terminology を与える。ファイナルファンタジー XI の internal resolution、viewport policy、scaling algorithm、aspect-ratio behavior を確認しない。

C++クライアント・マイルストーン。 decoded presentation setting、internal rendering dimension、output resolution、viewport、aspect ratio、UI scaling を分離する。

```
// Cumulative implementation excerpt: render size, output size, viewport, and UI scale.
struct PresentationState {
    Extent2D renderSize, outputSize;
    Rect viewport;
    float uiScale = 1.0f;
};

PresentationState makePresentation(Extent2D internal, Extent2D output) {
    const float scale = std::min(float(output.w) / internal.w,
                                   float(output.h) / internal.h);
    const Extent2D fitted{std::uint32_t(internal.w * scale),
                          std::uint32_t(internal.h * scale)};
    Rect vp{(output.w - fitted.w) / 2.0f, (output.h - fitted.h) / 2.0f,
            float(fitted.w), float(fitted.h)};
    return {internal, output, vp, scale};
}
```

45.1 render size と presentation size

internal render target は present される window または display size と異なりうる。scaling は border を使って source image を enlarge、reduce、stretch、preserve できる。final image は crop されるか viewport に置かれることもある。captured pixel dimension だけでは internal render size を特定しない。target descriptor と presentation parameter がより強い evidence である。

45.2 aspect ratio と projection

aspect ratio は presentation surface の shape と、world rendering に使う projection の両方へ影響しうる。wider display はより多い horizontal view を見せ、existing view を stretch し、crop し、別

policy を使いうる。UI anchoring は independent に振る舞いうる。difference を camera projection または layout へ帰属する前に、documented configuration 間で stable landmark と interface element を比較する。

45.3 scaling と filtering

scaling method は sharpness、texture detail、thin line、text、aliasing に影響する。point-like scaling は discrete source pixel を保ち、filtered scaling は neighboring sample を mix し、additional display processing はさらに change を加えうる。これらは texture-resolution difference を imitate し、alpha-tested edge の apparent threshold を変えうる。可能なら unscaled capture を保存し、conversion path を記録する。

45.4 safe area、border、cropping

一部 presentation は expected display で critical UI が visible なままになるよう margin または border を reserve する。window decoration、capture crop、overlay software も独自 boundary を加えうる。missing UI element または shifted edge は、different interface layout の evidence にする前に viewport と crop condition に対して test する。

45.5 comparison protocol

すべての image comparison について、判明している target size、output size、aspect ratio、window/full-screen state、viewport または border behavior、scaling mode、capture method、subsequent resampling を記録する。可能なら identical camera と UI state を使う。この protocol により display difference は renderer research の hidden noise ではなく explicit variable になる。

証拠基準。documented target または presentation parameter は確認済みである。visible stretch、crop、sharpness change の internal reason は、関連する target と presentation path が観測されるまで推論に留まる。

第 46 章 デスクトップシェル、ドッキング、ワークスペース寿命

デスクトップシェルは独立ビューアを一つの整合したツールへまとめる。メインウィンドウ、メニューとコマンド面、ドッキングされたブラウザ・診断ペイン、文書タブ、フォーカス経路、永続化境界、順序付きワークスペース終了を所有する。

実装成果。

すべてのエクスプローラとビューアが使う複数文書シェル、およびワークスペースファクトリ／寿命契約を実装する。

46.1 責務と所有

IWorkspace は識別、タイトル、変更状態、コマンド対象、描画・更新フック、終了交渉を公開する。シェルがワークスペースを所有し、ペインは弱参照またはトークン化接続で購読する。

46.2 実装手順

リソースを開くと、ファクトリ登録から最適ワークスペースを求め、方針が許す場合は既存互換文書を再利用し、そのタブを有効化し、共有インスペクタペインを選択コンテキストへ結び付ける。

46.3 障害と回復

ワークスペースは書き出し完了または破棄のため終了を拒否できる。一つのワークスペース例外は障害パネルへ変換する。破棄時は文書と GPU ビュー解放前に購読を失効させる。

46.4 検証ゲート

テストでは重複オープン方針、タブ有効化、コマンドフォーカス、ペイン再結合、終了取消、例外分離、レイアウト復元、終了順を検証する。

46.5 累積 C++実装

C++クライアントマイルストーン。すべてのエクスプローラとビューアが使う複数文書シェル、およびワークスペースファクトリ／寿命契約を実装する。

```
// Cumulative implementation listing
WorkspaceId WorkspaceManager::open(const OpenRequest& request) {
```

```

    if (auto existing = findReusable(request)) {
        activate(*existing);
        return *existing;
    }
    const WorkspaceFactory& factory = factories_.bestMatch(request);
    std::unique_ptr<IWorkspace> workspace = factory.create(services_, request);
    const WorkspaceId id = ids_.next();
    workspaces_.emplace(id, std::move(workspace));
    activate(id);
    return id;
}

bool WorkspaceManager::close(WorkspaceId id) {
    IWorkspace& w = *workspaces_.at(id);
    if (!w.prepareToClose()) return false;
    subscriptions_.revokeOwner(id);
    workspaces_.erase(id);
    activate(nextWorkspace());
    return true;
}

```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 47 章 コマンド、メニュー、設定、ツールウィンドウ

コマンドはメニュー ID、ショートカット、ボタンから独立して意図を表すべきである。本章では型付きコマンド経路、有効・チェック状態、設定検証、ツールウィンドウ所有、リソースまたはデバイス再作成が必要な変更の遅延適用を構築する。

実装成果。

一つの方針でメニュー、ショートカット、ダイアログ、ツールウィンドウを駆動する型付きコマンドと版付き設定を実装する。

47.1 責務と所有

CommandId は安定したアプリ語彙、CommandContext は有効ワークスペースと選択、CommandRouter は対象の照会と実行、Settings は型付き区分と版付き保存を持つ。

47.2 実装手順

UI 面は表示直前にコマンド状態を照会する。実行はフォーカス制御からワークスペース、シェルへ流れる。設定編集は下書きへ入り、全体検証後に即時、再ロード必要、再起動必要の差分を別々に適用する。

47.3 障害と回復

未知の永続設定は可能な限り読み書きで保持する。不正値は診断付きで代替する。数値メニュー ID をアプリケーションロジックへ渡さない。

47.4 検証ゲート

テストではコマンド優先順位、無効実行、チェック状態、ショートカット同値、設定移行、不正値代替、ツールウィンドウ単一化方針、デバイスリセット集約を検証する。

47.5 累積 C++実装

C++クライアントマイルストーン。一つの方針でメニュー、ショートカット、ダイアログ、ツールウィンドウを駆動する型付きコマンドと版付き設定を実装する。

```
// Cumulative implementation listing
bool CommandRouter::invoke(CommandId id, const CommandContext& c) {
    for (ICommandTarget* target : targetsFor(c)) {
        if (target->query(id, c).enabled) {
            target->execute(id, c);
            return true;
        }
    }
    return false;
}

ApplyReport SettingsController::commit(const SettingsDraft& draft) {
    ValidationReport valid = validate(draft.value);
    if (!valid.ok()) return {false, valid.issues, {}};
    SettingsDiff diff = compare(current_, draft.value);
    applyImmediate(diff);
    scheduleReloads(diff);
    current_ = draft.value;
    store_.saveVersioned(current_);
    return {true, {}, diff.restartRequired};
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 48 章 入力、カメラ、ピッキング、ナビゲーションモード

対話的検査には予測可能な入力が必要である。本章では生イベントとアクション、フォーカス・キャプチャと移動、回転・自由飛行・接地・対象注視カメラモードを観察対象のシーンデータから分離する。

実装成果。

設定可能な入力、四つのナビゲーションモード、信頼できるポインタキャプチャ、描画ビューポートと一致する選択レイを実装する。

48.1 責務と所有

InputState は物理状態、ActionMap は割当解決、NavigationController はアクションと経過時間、CameraRig はモード別状態、Picker はビューポート座標からレイを生成する。

48.2 実装手順

ウィンドウイベントはまず入力状態を更新する。文字や数値制御編集中はフォーカス規則が移動を抑止する。キャプチャされたポインタ差分はカメラアクションになる。選択クリックはそのフレーム描画に使った同じカメラ行列を使う。

48.3 障害と回復

フォーカス喪失時はキャプチャを解放し一時状態をクリアする。コリジョン欠落時は接地モードを明示的に無効化する。極端な経過時間はカメラ跳躍を起こさず、制限して報告する。

48.4 検証ゲート

テストでは割当、修飾キー組合せ、フォーカス抑止、キャプチャ解放、フレームレート非依存、カメラモード遷移、ピックレイ、接地代替を検証する。

48.5 累積 C++実装

C++クライアントマイルストーン。設定可能な入力、四つのナビゲーションモード、信頼できるポインタキャプチャ、描画ビューポートと一致する選択レイを実装する。

```
// Cumulative implementation listing
void NavigationController::update(const InputState& input, double elapsed) {
    const double dt = std::clamp(elapsed, 0.0, 0.1);
    ActionState a = bindings_.resolve(input);
    if (!focus_.allowsNavigation()) a.clearMotion();
    switch (camera_.mode()) {
    case CameraMode::Orbit:    updateOrbit(camera_, a, dt); break;
    case CameraMode::Fly:      updateFly(camera_, a, dt); break;
    case CameraMode::Grounded: updateGrounded(camera_, a, dt, collision_); break;
    case CameraMode::Focus:    updateFocus(camera_, a, dt, selection_); break;
    }
}

Ray Picker::ray(Vec2 pixel, Viewport vp, const CameraMatrices& m) const {
    const Vec2 ndc{2.0f * (pixel.x - vp.x) / vp.width - 1.0f,
                  1.0f - 2.0f * (pixel.y - vp.y) / vp.height};
    return unprojectRay(ndc, inverse(m.viewProjection));
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 49 章 状態変更、バッチ処理、リソース再利用

renderer は correct composition と state change および work submission の cost を balance する。texture、buffer、transform、shader、texture stage、blend rule、target、depth setting は draw 間で変わりうる。batching は compatible work を group して一部 change を減らし、resource reuse は data の unnecessary な recreate または reload を避ける。どちらも screenshot から直接 visible ではない。

範囲。本章は performance-analysis vocabulary である。ファイナルファンタジー XI の batching policy、cache design、exact cost model を確認しない。

C++クライアント・マイルストーン。 source identity と必要な order を失わず state cache、draw sort、batching、resource reuse を追加する。

```
// Cumulative implementation excerpt: cache state, but preserve required source order.
void Renderer::submit(std::span<const Draw> sourceOrdered) {
    std::vector<Draw> reorderable, fixed;
    for (const Draw& d : sourceOrdered)
        (d.requiresSourceOrder ? fixed : reorderable).push_back(d);
    std::stable_sort(reorderable.begin(), reorderable.end(),
        [](const Draw& a, const Draw& b) { return a.stateKey() < b.stateKey(); });
    for (const Draw& d : mergeWithoutCrossingBarriers(reorderable, fixed)) {
        stateCache_.applyOnlyChanges(d.state);
        resourceCache_.bind(d.resources);
        device_>DrawIndexedPrimitive(d.primitive, d.minIndex, d.vertexCount,
                                     d.startIndex, d.primitiveCount);
    }
}
```

49.1 state-change boundary

state change は draw が previous draw と異なる active setting または binding を必要とするときに起きる。一部 change は他より inexpensive であり、一部は correctness に必要であり、一部は compatible opaque work を reorder して避けられる。trace では call 数だけでなく、どの active value が実際に変わり、なぜ neighboring draw が share できたまたはできなかったかが重要である。

49.2 batching

batching は compatible object を together に submit し、geometry を fewer draw へ combine し、material で group し、rendering phase 前に work を collect することを意味しうる。これらの strategy

は visibility、transform、transparency、animation、memory に異なる tradeoff を持つ。同じ state の repeated draw は opportunity または existing grouping を示すが、geometry が combine されたか、どの batching algorithm が select したかを単独では明らかにしない。

49.3 resource reuse

texture、vertex buffer、index buffer、shader、target は多くの frame にわたり live し repeatedly bind されうる。reuse は recurring identifier と stable descriptor として観測されることが多く、update は changing content または parameter を示す。resource lifetime と draw frequency を分ける。resource は long-lived でも rarely used であり、frequently updated でも many draw で reused されうる。

49.4 correctness constraint

transparency、depth、effect、interface layering は、異なる order が state change を減らせても reordering を制限しうる。renderer は visible result を保つため extra bind または draw を deliberately accept できる。performance analysis では fewer call が常に better または more authentic path を意味するとせず、proposed optimization を composition constraint と pair にする。

49.5 trace analysis

trace を target と scene boundary で group し、texture、stream、declaration、shader、render state、draw type の change を記録する。各 group を visible scene と resource lifetime に比較する。最も強い結論は、未観測の cache または batch system を invent せず、『compatible draw の run にわたる一 texture の repeated binding』のような concrete pattern を特定する。

証拠基準。repeated binding と ordered state change は confirmed trace fact である。'batch'、'cache'、'sort key' は geometry、scheduling、code evidence を必要とする implementation label である。

第 50 章 CPU と GPU の処理分担

frame は異なる responsibility を持つ cooperating processor によって作られる。CPU-side work には scene update、animation evaluation、visibility selection、resource management、state setup、

draw submission が含まれる。GPU-side work には vertex processing、primitive rasterization、texture sampling、blending、target write が含まれる。driver と presentation はさらに scheduling layer を加える。visible frame だけではこの division は明らかにならない。

範囲。本章は performance-analysis category を説明する。ファイナルファンタジー XI の CPU/GPU balance、threading、driver behavior を測定または特定しない。

C++クライアント・マイルストーン。 parse/decode/upload timing と CPU-GPU frame telemetry で asset-to-frame 全経路を計測する。

```
// Cumulative implementation excerpt: scoped CPU timing plus GPU completion markers.
class ScopedTimer {
    Telemetry& t_; Stage stage_; Clock::time_point begin_ = Clock::now();
public:
    ScopedTimer(Telemetry& t, Stage s) : t_(t), stage_(s) {}
    ~ScopedTimer() { t_.record(stage_, Clock::now() - begin_); }
};

FrameMetrics renderMeasured(ClientApp& app) {
    { ScopedTimer t(app.telemetry, Stage::Parse); app.assets.parseReady(); }
    { ScopedTimer t(app.telemetry, Stage::Upload); app.assets.uploadReady(); }
    app.gpuMarkers.beginFrame();
    { ScopedTimer t(app.telemetry, Stage::Submit); app.renderer.draw(app.scene); }
    app.gpuMarkers.endFrame();
    app.renderer.present();
    return app.telemetry.finishFrame(app.gpuMarkers.completedDuration());
}
```

50.1 preparation と submission

geometry が graphics pipeline へ届く前に、application は world state を update し、animation を resolve し、visible work を choose し、transform を construct し、resource を lock または update し、device state を set する。draw call は work を describe するが、CPU と driver time も consume する。pixel cost が modest な scene でも、多数の small submission または expensive preparation に制限される。

50.2 GPU-side cost category

vertex workload は submitted geometry と processing path に依存する。pixel workload は screen coverage、texture sampling、blending、depth rejection、target、resolution に依存する。large translucent effect は little geometry でも多くの overlapping pixel を shade するため expensive になりうる。distant complex mesh は few pixel しか cover せず early に cull されれば cheap になりうる。performance change を説明する前に category を分ける。

50.3 queue と synchronization

CPU submission と GPU execution は同時刻に起きる必要はない。work は queue され、synchronization point は一方を他方の wait にできる。presentation behavior、resource lock、readback、certain query はこの relationship へ影響できる。したがって frame-time observation は最後に visible だった draw または effect へ自動的に帰属しない。

50.4 overclaim せずに測定する

repeatable scene を使い、可能なら一度に一 workload class を変える。visibility には camera direction、pixel cost には resolution、blending には effect density、animation と submission には character count、binding には texture state diversity を変える。configuration、capture method、uncertainty とともに frame-time measurement を記録する。correlation は exact CPU または GPU function を特定する前に likely bottleneck class を示せる。

50.5 evidence record

performance claim について scene condition、display configuration、draw と primitive count、state-change count、resource update、target usage、timing method、repeated sample、changed variable を記録する。measured timing と cause の explanation を分ける。後者には competing workload class を exclude する profile、trace、controlled evidence が必要である。

証拠基準。documented condition での measured frame-time change は confirmed である。それを CPU-bound、GPU-bound、driver-bound、synchronization-bound と呼ぶことは measurement evidence が cause を isolate しない限り推論である。

50.6 renderer 非依存の parser probe

pixel より前に structural regression を検出する。Listing 32.1 は fixture を hash し、全 bounded chunk を walk し、type count と failure を記録し、checked-in expectation と比較する。visual client と同じ registry を使うが、window も graphics device も作成しない。

Listing 32.1 parser regression test 用 stable structural output.

```
struct ProbeSummary {
    std::string fileHash;
    std::size_t chunkCount = 0;
    std::map<std::uint8_t, std::size_t> typeCounts;
    std::vector<FailureRecord> failures;
```

```

};

ProbeSummary probeResource(const std::filesystem::path& path,
                           const ChunkRegistry& registry) {
    const auto bytes = readWholeFileBounded(path, 256_MiB);
    ProbeSummary out{sha256(bytes)};
    EvidenceLog evidence;
    for (const ChunkView& chunk : walkChunkViews(bytes)) {
        ++out.chunkCount;
        ++out.typeCounts[chunk.info.type];
        try {
            const Recognition r = registry.inspect(chunk, evidence);
            if (r == Recognition::Invalid)
                out.failures.push_back({chunk.offset, "invalid"});
        } catch (const std::exception& e) {
            out.failures.push_back({chunk.offset, e.what()});
        }
    }
    return out;
}

void requireFixture(const ProbeSummary& actual, const Fixture& expected) {
    require(actual.fileHash == expected.fileHash);
    require(actual.chunkCount == expected.chunkCount);
    require(actual.typeCounts == expected.typeCounts);
    require(actual.failures == expected.failures);
}

```

fixture expectation には stable fact だけを入れる。input hash、selected range、chunk offset と type、supported-record count、dimension、named failure である。pointer value、unordered iteration order、timing、image 由来の guess を structural truth へ入れない。

第 51 章 バックグラウンドジョブ、キャンセル、UI 受け渡し

カタログ走査、デコード、ハッシュ、書き出しで UI を停止させてはならない。本章では境界付きワーカ実行、協調キャンセル、進捗、世代検査、例外捕捉、UI スレッドへの安全な公開を実装する。

実装成果。

カタログ化、デコード、ストリーミング、オーディオ、スクリーンショット、書き出しが共有するジョブシステムを実装する。

51.1 責務と所有

Job は停止源、進捗、結果チャネル、世代を持つ。JobQueue はワーカと上限を所有する。UI デイスパッチャはコールバックを所有する。文書は要求世代付き不変結果を公開する。

51.2 実装手順

投入時は型付きキーに従って置換または待ち行列化する。ワーカはすべての例外を捕捉し、定期的にキャンセルを確認し、完了を通知する。UI は文書と世代が一致する結果だけを受理する。

51.3 障害と回復

キャンセルはスレッド強制終了ではない。長時間デコーダは確認点を公開する。キュー飽和時は逆圧を適用する。終了時は受付停止、キャンセル要求、UI 完了処理、ワーカ join の順に行う。

51.4 検証ゲート

テストでは置換競合、キャンセル遅延、キュー上限、例外、古い結果、進捗単調性、高負荷終了、決定論的単一ワーカモードを扱う。

51.5 累積 C++実装

C++クライアントマイルストーン。カタログ化、デコード、ストリーミング、オーディオ、スクリーンショット、書き出しが共有するジョブシステムを実装する。

```
// Cumulative implementation listing
template<class Work, class Publish>
JobHandle JobQueue::replace(JobKey key, Work work, Publish publish) {
    cancel(key);
    const std::uint64_t generation = ++generations_[key];
    return enqueue(key, [=, work = std::move(work),
                        publish = std::move(publish)](StopToken stop) mutable {
        try {
            auto result = work(stop);
            ui_.post([=, result = std::move(result),
                    publish = std::move(publish)]() mutable {
                if (generations_[key] == generation) publish(std::move(result));
            });
        } catch (...) {
            ui_.post([=, error = std::current_exception()] {
                if (generations_[key] == generation) report(error);
            });
        }
    });
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 52 章 メモリ予算、ストリーミング、ロード挙動

persistent world は finite memory と transfer bandwidth を尊重しながら必要なとき resource を available にしなければならない。texture、geometry、animation、sound、UI resource、target、

temporary buffer が space を競う。loading behavior は player が観測するものであり、streaming、caching、eviction、prefetching、package boundary は resource と timing evidence を必要とする possible explanation である。

範囲。本章は resource-lifetime framework である。ファイナルファンタジー XI の memory budget、streaming system、package format、cache policy を確認しない。

C++クライアント・マイルストーン。 dependency-aware streaming、cache、eviction、missing/corrupt/unsupported record の recoverable handling を作る。

```
// Cumulative implementation excerpt: dependency-aware cache with recoverable failure.
std::shared_future<AssetResult> Streamer::request(ResourceId id) {
    if (auto cached = cache_.find(id)) return makeReadyFuture(*cached);
    return jobs_.submit([this, id] -> AssetResult {
        try {
            auto bytes = readResource(id);
            auto asset = decodeAsset(bytes);
            for (ResourceId dependency : asset.dependencies) request(dependency);
            cache_.insert(id, asset, asset.memoryBytes());
            cache_.evictLeastRecentlyUsedUntilWithin(budgetBytes_);
            return asset;
        } catch (const UnsupportedRecord& e) { return Unsupported{e.what()}; }
        catch (const std::exception& e)      { return Failed{e.what()}; }
    });
}
```

52.1 resource lifetime

resource は create、populate、bind、use、update、release、reuse のため retain されうる。これらの event は異なる time と thread または subsystem で起きうる。transition 後に trace で resource が現れても、その時 load された、already available data から create された、captured scene で初めて bind された可能性がある。first sighting をすべて load と呼ばず event type を記録する。

52.2 loading と streaming observation

loading screen、pause、object pop-in、changed texture detail、delayed animation は availability または presentation の観測である。bulk loading、incremental streaming、visibility selection、decompression、target creation、network state、その他 process に対応しうる。controlled route と repeated timing sample は possibility を狭められるが、pipeline を確立するには resource creation と update record が必要である。

52.3 budget と tradeoff

memory limit は texture resolution、 mip level、 geometry detail、 animation data、 temporary surface、 reuse についての choice を促す。 transfer bandwidth と CPU time は resource が ready になる時を左右する。 renderer は coarse representation を選び、 detail layer を delay し、 frequently used resource を resident に保てる。 これらは general tradeoff であり、 game の一 policy の evidence ではない。

52.4 transition study

repeatable transition を選ぶ。 area への entry、 camera route の change、 equipment の change、 visible effect の trigger である。 time、 draw presence、 resource descriptor、 creation/update event、 bind、 前後の visible sequence を記録する。 possible なら cold state と warmed state から repeat し、 test におけるそれらの意味を document する。 これにより one-time initialization と recurring availability behavior を分ける。

52.5 evidence record

各 resource claim について identifier、 type、 descriptor、 first observed creation、 update、 binding、 last observed use、 scene context、 timing sample、 release または reset evidence を記録する。 event order が support するときだけ record を visual change へ link する。 missing texture または delayed model だけでは eviction または streaming algorithm に名前を付けられない。

証拠基準。 recorded resource event と measured transition は confirmed である。 streaming、 cache hit、 eviction、 prefetch は demonstrated policy または code path を必要とする implementation claim である。

第 53 章 リソースキャッシュと明示的寿命

長時間動作するエクスプローラは関連アセットを繰り返し開き、所有を隠さず再利用する必要がある。本章ではソースバイト、デコード済み CPU 文書、デバイス依存 GPU オブジェクトを、明示キー、予算、固定、世代、退避を持つキャッシュへ分ける。

実装成果。

安定ハンドル、ロード統合、予算、診断、デバイス消失回復を備えた階層 CPU/GPU キャッシュを実装する。

53.1 責務と所有

CacheKey はインストール識別、リソース ID、ソースハッシュ、デコーダ版、デコードオプションを含む。CPU と GPU キャッシュ項目は独立したサイズと寿命を持つ。文書はキャッシュ内部ポインタではなくハンドルを保持する。

53.2 実装手順

照会は準備済みハンドルを返すか、進行中デコードへ参加するか、一つのジョブを開始する。予算適用は未固定の最終使用時刻が古い項目から退避する。デバイスリセットは GPU 世代だけを破棄し、CPU 文書は再利用する。

53.3 障害と回復

失敗結果には再試行嵐を防ぐ短い負キャッシュ寿命を与える。ソースハッシュ変更は新しいキーを作る。固定ワークスペースはソフト予算を超え得るが超過を報告する。

53.4 検証ゲート

テストではロード統合、キー分離、決定論的退避、固定、ソース変更、負キャッシュ期限、デバイス世代消失、終了後 GPU オブジェクトゼロを検証する。

53.5 累積 C++実装

C++クライアントマイルストーン。安定ハンドル、ロード統合、予算、診断、デバイス消失回復を備えた階層 CPU/GPU キャッシュを実装する。

```
// Cumulative implementation listing
void ResourceCache::trim(std::size_t budgetBytes) {
    while (residentBytes_ > budgetBytes) {
        auto victim = std::min_element(entries_.begin(), entries_.end(),
            [](const auto& a, const auto& b) {
                if (a.second.pins != b.second.pins) return a.second.pins < b.second.pins;
                return a.second.lastUse < b.second.lastUse;
            });
        if (victim == entries_.end() || victim->second.pins != 0) break;
        residentBytes_ -= victim->second.bytes;
        entries_.erase(victim);
    }
}

void GpuCache::onDeviceLost() {
```

```

++generation_;
entries_.clear();
residentBytes_ = 0;
}

```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 54 章 パフォーマンス上のトレードオフと旧世代ハードウェア設計

renderer は isolated feature の list ではなく tradeoff の集合である。より多い geometry はより多い animation work と競合し、rich texture は memory と bandwidth と競合し、transparency は fill rate と ordering と競合し、higher resolution はすべての pixel-cost operation と競合する。legacy-hardware design は modern technique の単純な absence ではなく、particular platform と content goal の下での prioritization として理解するのが最善である。

範囲。本章は tradeoff を解釈する framework である。primary evidence なしに、ファイナルファンタジー XI の original design decision、hardware budget、performance target を確立しない。

C++クライアント・マイルストーン。 file decoding、resource preparation、rendering、presentation を別々に benchmark し configurable budget を適用する。

```

// Cumulative implementation excerpt: independent budgets and repeatable benchmarks.
struct Budgets { double decodeMs, prepareMs, renderMs, presentMs; std::size_t memory; };

BenchmarkResult benchmarkScene(ClientApp& app, const RegressionScene& scene,
                               const Budgets& b) {
    BenchmarkResult r;
    r.decode = measure([&] { app.assets.decode(scene.resources); });
    r.prepare = measure([&] { app.scene.prepare(scene); });
    r.render = measure([&] { app.renderer.draw(app.scene); });
    r.present = measure([&] { app.renderer.present(); });
    r.memory = app.assets.residentBytes();
    r.withinBudget = r.decode.ms <= b.decodeMs && r.prepare.ms <= b.prepareMs
        && r.render.ms <= b.renderMs && r.present.ms <= b.presentMs
        && r.memory <= b.memory;
    return r;
}

```

54.1 frame budget は shared である

CPU preparation、submission、vertex processing、rasterization、texture sampling、blending、target、presentation、memory movement はすべて frame budget の一部を consume する。一 category を improve すると別の category を worsen することがある。geometry を combine すると

draw submission を reduce できるが visibility が less selective になり、extra texture stage は material detail を improve できるが sampling と state complexity を increase する。

54.2 content choice と renderer choice

low-detail distant object、repeated texture、compact effect、fogged horizon は art-direction choice、resource-format constraint、performance tradeoff、または複数の組み合わせを反映する。measurement、design record、implementation evidence が cost へ結び付けるまで visible simplification を optimization と label してはならない。同じ visual technique は aesthetics と performance の両方に役立つ。

54.3 platform-aware interpretation

platform ごとに異なる API、memory structure、display target、driver behavior を expose する。portability decision は shared content を preserve しながら different path または limit を必要にできる。version を compare するときは observed difference を先に特定し、configuration、platform capability、asset variation、implementation divergence のどれが evidence に support されるかを問う。

54.4 authenticity と modernization

technical account は historical behavior と modern reconstruction または enhancement を分けるべきである。limitation を more capable technique に replace すると image は improve しても original behavior を記述しなくなる。逆に artifact を preserve しても original cause を reproduce しない場合がある。comparison の goal—observed behavior、inferred mechanism、visual approximation—を explicit に述べる。

54.5 tradeoff argument

strong tradeoff claim は observed choice、measured または documented constraint、plausible alternative、それらを結ぶ evidence を名付ける。たとえば documented state change が measured condition で work class を reduce すると述べられるが、source なしに developer が particular budget

のために選んだとは主張できない。この standard は engineering reality と historical uncertainty の両方を尊重する。

証拠基準。measured cost と documented platform limit は tradeoff analysis を support できる。intent、priority、original design rationale には primary documentation または direct development evidence が必要である。

第 55 章 スクリーンショット、レポート、データ書き出し

書き出しは単なる保存ボタンではなく証拠境界である。本章ではスクリーンショット、デコード表、ジオメトリ交換データ、テクスチャ、調査レポートを、ソース識別、座標規約、信頼度、省略機能警告とともに出力する。

実装成果。

マニフェストと明示変換方針を持つトランザクション型スクリーンショット、表、テクスチャ、ジオメトリ、レポート書き出しを実装する。

55.1 責務と所有

ExportRequest は文書スナップショット、選択、オプション、出力先、由来情報を固定する。Exporter 実装は一時兄弟ファイルへ書き、出力を検証してから出力先を原子的に置換する。

55.2 実装手順

UI は意図を検証し、上書き承認を得て、キャンセル可能ジョブを開始し、マニフェストを記録する。ジオメトリ書き出しは軸、手系、単位、巻き順、頂点属性、マテリアル、未対応内容を宣言する。

55.3 障害と回復

キャンセル時は一時出力を削除する。既存出力先はコミットまで変更しない。書き出しはソースゲームファイル全体を複製せず、その内容の所有を示唆しない。

55.4 検証ゲート

テストでは上書き方針、キャンセル、ディスク満杯模擬、安定マニフェスト、座標変換、画像行方向、選択のみ書き出し、診断出力の再読込を扱う。

55.5 累積 C++実装

C++クライアントマイルストーン。マニフェストと明示変換方針を持つトランザクション型スクリーンショット、表、テクスチャ、ジオメトリ、レポート書き出しを実装する。

```
// Cumulative implementation listing
ExportResult ExportService::run(const ExportRequest& r, StopToken stop) {
    const auto temporary = siblingTemporaryPath(r.destination);
    removeIfExists(temporary);
    try {
        ExportManifest manifest = exporters_.at(r.format).write(r, temporary, stop);
        if (stop.stopRequested()) { removeIfExists(temporary); return {false, "cancelled"}; }
        validateExport(temporary, manifest);
        atomicReplace(temporary, r.destination, r.allowOverwrite);
        writeManifest(r.destination, manifest);
        return {true, {}};
    } catch (...) {
        removeIfExists(temporary);
        throw;
    }
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 56 章 セッション、プリセット、再現可能なワークスペース

調査ツールはウィンドウ座標だけでなく調査そのものを再開できるべきである。セッションはインストール識別、開いていたリソース、ワークスペース種別、選択、カメラ、時刻、天候、描画制御、証拠リンクを保存し、欠落リソースを回復可能として扱う。

実装成果。

実行時専用状態を保存せず調査を再現できる版付きセッションと用途別プリセットを実装する。

56.1 責務と所有

SessionDocument は版付きで安定した論理識別だけを含み、プロセスポインタや GPU ハンドルを含まない。プリセットはカメラ、キャラクター、環境、書き出し、比較条件向けの小さな名前付き部分集合である。

56.2 実装手順

保存はUIスレッドですべてのワークスペースをスナップショット化し、別スレッドで直列化し、トラッキング出力し、アプリ版とスキーマ版を記録する。復元は先にインストールを検証し、非同期でワークスペースを開き、成功した依存へ状態を適用する。

56.3 障害と回復

未知フィールドは移行後も保持する。欠落リソースは再リンク操作付きプレースホルダタブを作る。不正カメラや時刻値は代替し復元レポートへ記録する。

56.4 検証ゲート

テストでは往復、スキーマ移行、未知保持、部分復元、移動したインストール、競合オープン要求、決定論的比較プリセットを扱う。

56.5 累積 C++実装

C++クライアントマイルストーン。実行時専用状態を保存せず調査を再現できる版付きセッションと用途別プリセットを実装する。

```
// Cumulative implementation listing
RestoreReport SessionService::restore(const SessionDocument& saved) {
    SessionDocument current = migrations_.upgrade(saved);
    RestoreReport report;
    for (const WorkspaceSnapshot& ws : current.workspaces) {
        try {
            WorkspaceId id = workspaces_.open({ws.resource, ws.kind});
            workspaces_.whenReady(id, [state = ws.state, &report](IWorkspace& w) {
                report.merge(w.restore(state));
            });
        } catch (const std::exception& ex) {
            report.missing.push_back({ws.resource, ex.what()});
            workspaces_.openPlaceholder(ws, ex.what());
        }
    }
    return report;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 VI 部 ケーススタディ、検証、提供

第 57 章 不透明な環境描画を読み解く

opaque environment draw は scene selection、geometry、material state、depth behavior、visible image contribution を結ぶため有用な case-study unit である。目的は appearance から draw に label を付けることではない。observed render event から、それが何をするかについて最小限の supported description へ至る chain を作ることである。

範囲。これは case-study procedure であり、特定のファイナルファンタジー XI environment draw についての主張ではない。

C++クライアント・マイルストーン。 source byte、parsed record、resource、state、depth behavior、draw order を verify する opaque environment case study を作る。

```
// Cumulative implementation excerpt: opaque draw dossier from bytes to submission.
EnvironmentDrawReport inspectEnvironmentDraw(ResourceId id, const Camera& camera) {
    EvidenceLog evidence;
    const DatFile dat = openDat(id, evidence);
    const Scene scene = buildZone(dat, evidence);
    const auto visible = gatherVisible(scene, camera);
    const Draw draw = firstOpaqueEnvironmentDraw(visible);
    return {id, dat.hash(), draw.sourceChunkOffset, draw.geometry.header,
            draw.material, draw.state, draw.depthKey, draw.sourceOrder,
            evidence.entries()};
}

void verify(const EnvironmentDrawReport& r) {
    require(r.state.depthWrite && !r.state.blendEnabled);
    require(r.geometry.indexCount % 3 == 0 || r.geometry.isStrip);
    require(r.sourceChunkOffset < r.sourceFileSize);
}
```

57.1 capture context を確立する

platform、client build、zone と landmark、camera position と direction、time と weather、display configuration、capture method から始める。target geometry が visible な stable frame を特定し、nearby occluder、effect、interface element を記録する。この context なしには、後の trace を image へ reliably に match できない。

57.2 candidate draw を見つける

ordered draw evidence と controlled visual change を使い candidate draw を screen contribution に associate する。small camera movement、visibility change、target condition だけが異なる frame を

比較する。draw の target、primitive type と count、buffer と range、transform、texture、stage または shader、depth/culling/fog state を記録する。correlation が strong になるまで object に名前を付けない。

57.3 confirmed state を記述する

factual description を書く。たとえば vertex と index buffer のこの range を使う triangle draw、これらの texture binding、depth testing と writing、back-face culling、specified texture-stage operation である。この description は technical だが narrow でよい。evidence に trace できない broad material label より価値がある。

57.4 alternative を test する

同じ draw が different surface、second pass、shared material group、partially hidden object でありうるかを問う。camera、distance、neighboring condition を一度に一つ変える。suspected screen region が変わっても draw が persist するなら correlation は incomplete かもしれない。documented condition で region とともに disappear するなら confidence は increase する。

57.5 case-study result を作る

final entry には capture context、trace excerpt または record、confirmed resource/state fact、observed image contribution、considered alternative、confidence、next test を含める。よい結論は identified resource を使う opaque で depth-writing な environment contribution と述べられる。engine の internal object name や batching policy を主張する必要はない。

証拠基準。case study は confirmed draw fact から始め、その fact を visible scene へ結ぶために必要な inference だけを述べるべきである。

第 58 章 キャラクター描画を読み解く

character draw は難しい case-study target である。player が認識する character は多くの component から assemble され、pose、equipment、distance、effect、interface marker で change できる。discipline ある分析では一つの render event または small component group を isolate し、controlled state change を通じて visible character へ connect する。

範囲。これは character-draw case-study method であり、特定のファイナルファンタジー XI character path の classification ではない。

C++クライアント・マイルストーン。 decoded component、binding、animation、material、equipment、draw decision を公開する character case study を作る。

```
// Cumulative implementation excerpt: character component and decision dossier.
CharacterReport inspectCharacter(const CharacterSelection& choice, float time) {
    EvidenceLog evidence;
    Character c = assembleCharacter(choice, repository());
    Pose pose = evaluatePose(c.skeleton, c.animation, time, evidence);
    std::vector<PartReport> parts;
    for (const CharacterPart& p : c.parts) {
        parts.push_back({p.source, p.equipmentSlot, p.bonePalette,
                        p.material, p.alphaClass,
                        buildCharacterDraw(p, pose).state});
    }
    return {choice, time, pose.hash(), std::move(parts), evidence.entries()};
}
```

58.1 character state を freeze する

character identity は test label としてのみ記録し、relevant なら race/body variation、equipment state、action または pose、animation time、distance、camera、zone、lighting-like condition、effect、UI overlay を document する。一 component を change する前に stable pose を選ぶ。目的は animation、occlusion、equipment variation を draw association と混同しないことである。

58.2 component boundary を見つける

一 equipment、pose、visibility condition だけを change した capture または trace を compare する。stable な draw、appear または disappear する draw、resource または transform が change する draw を探す。buffer、range、vertex layout、texture、material state、transform、blend/depth behavior、neighboring draw を記録する。component boundary は menu slot または visual silhouette だけではなく evidence で confirm される。

58.3 rigid work と deforming work を分ける

moving draw は transform に従う rigid attachment、skinned surface、separately animated object、camera-related effect になりうる。vertex attribute、transform change、constant、shader/fixed-function path、pose response を compare する。proof なしに skinning method を名付けず、draw が pose で change し given layout を使うという最小 supported result を述べる。

58.4 layered material を説明する

hair、equipment trim、cutout、transparent ornament、weapon effect、shadow は otherwise opaque な character の周囲に draw を加えうる。target、order、blend と depth state、culling、texture、overlap behavior を特定する。missing visual piece は later transparent draw、visibility rule、resource condition かみしれず、base mesh が change した evidence ではない。

58.5 character case-study result

character case を component ledger として publish する。capture context、controlled variation、candidate draw、confirmed resource/state/transform fact、observed response、alternative、confidence、next test である。player-facing label と technical label を分ける。これにより later evidence が additional component または different assembly relation を明らかにしたとき result を extend できる。

証拠基準。controlled equipment または pose change に対する draw response は component association を strongly support できる。internal slot name、skeleton relation、animation system には direct data または implementation evidence が必要である。

第 59 章 透明エフェクトを読み解く

透明エフェクトを、生バイトから最終画素まで追跡する完全な事例研究とする。ジェネレーターのセクション依存命令からリソースとシミュレーション状態を辿り、描画順を持つ各透明寄与を説明できなければならない。

範囲。この手順は復号済みジェネレーターモデルを使うが、全オペコードとスケジュール規則が完成済みとは主張しない。

59.1 証拠条件を固定する

DAT 集合、リソース ID、ファイルハッシュ、チャンク位置、ヘッダー、セクション境界、依存関係、トリガー、カメラ、アクター姿勢、乱数シード、時間範囲を記録する。決定論的シードで同じ粒子配置を再生できる。

59.2 命令台帳を作る

すべての{section, opcode}、ペイロード長、生バイト、解釈、確度、参照リソースを列挙する。未知命令は元の順序で保持し、未対応機能と黙って捨てたデータを区別する。

59.3 時間を再構築する

スケジュール期間、放出間隔、放出数、各粒子の誕生フレーム、年齢、寿命、曲線時間、死亡フレーム、子ジェネレーターイベントを追う。外観調整より先に、生存数と可視遷移をキャプチャと比較する。

59.4 変換を再構築する

各粒子について付着変換、ローカル発生位置、速度積分、回転、拡縮、ビルボード制約、最終ワールド行列を記録する。位置ずれの原因を付着、座標変換、演算順、ビルボードに分離できる。

59.5 描画を再構築する

各描画のジオメトリ系統、カード／群選択、テクスチャ、頂点色、UV、アルファテスト、ブレンド式、深度試験、深度書き込み、バイアス、カリング、ソートキーを記録し、可視レイヤーへ結び付ける。

59.6 結果を公開する

選択フレームごとに、ソース命令、ランタイムイベント、粒子状態、描画状態、観察画素を同期した表として公開する。省略命令と代替解釈も示し、修正可能な説明にする。

```
struct EffectFrameRecord {
    std::uint32_t frame;
    std::vector<EmissionRecord> emissions;
    std::vector<ParticleSnapshot> particles;
    std::vector<DrawRecord> draws;
    std::vector<UnknownCommandRecord> unresolved;
};
```

C++クライアント到達点。フレーム記録を JSON と人間向け報告の両方へ出力し、パーサー改訂間で差分比較できるようにする。

```
// Cumulative implementation excerpt: export one effect frame in two reviewable forms.
void exportEffectFrame(const EffectFrameRecord& frame,
                      const std::filesystem::path& base) {
    JsonWriter json(base.string() + ".json");
    json.object("frame", frame.frame)
        .array("emissions", frame.emissions)
        .array("particles", frame.particles)
        .array("draws", frame.draws)
```

```

        .array("unresolved", frame.unresolved);

    std::ofstream text(base.string() + ".txt");
    text << "Frame " << frame.frame << '\n';
    for (const auto& e : frame.emissions) text << describe(e) << '\n';
    for (const auto& d : frame.draws) text << describe(d) << '\n';
    for (const auto& u : frame.unresolved) text << "OPEN: " << describe(u) << '\n';
}

```

証拠基準。成功した再構築は時間、変換、リソース、状態を同時に説明する。似た画素を複数の仕組みで作れるため、見た目の一致だけでは内部モデルを検証できない。

第 60 章 証拠からフレームを再構成する

frame reconstruction は final image がどう assemble されたかについての evidence-backed model である。もっともらしく見える decorative diagram ではない。reconstruction は ordered fact を保ち、observed contribution と inferred pass role を分け、missing link を特定し、新しい trace、capture、resource record が picture を変えるとき revisable でなければならない。

範囲。本章は reconstruction workflow を定義する。confirmed なファイナルファンタジー XI frame graph を示すものではない。

C++クライアント・マイルストーン。 file evidence を parsed record、resource、state transition、intermediate target へ結ぶ frame-inspection mode を作る。

```

// Cumulative implementation excerpt: select a draw and walk back to source evidence.
FrameInspection inspectFrame(const CapturedFrame& frame, DrawId selected) {
    const DrawRecord& draw = frame.draw(selected);
    FrameInspection out;
    out.draw = draw;
    out.stateChanges = frame.stateHistory.leadingTo(selected);
    out.resources = frame.resources.boundAt(selected);
    out.targets = frame.renderGraph.ancestry(draw.target);
    for (ResourceId id : out.resources)
        out.sourceEvidence.push_back(frame.evidence.forResource(id));
    return out;
}

```

60.1 frame dossier から始める

一つの stable frame context を collect する。platform と client build、scene location、camera、time と weather、display path、visible effect、UI state、capture method、trace range である。original evidence を stable identifier とともに store する。dossier は unrelated observation を一つの imagined frame へ combine することを防ぐ boundary である。

60.2 ordered event list を作る

clear operation、target change、scene boundary、resource update、state transition、draw call、presentation を observed order で list する。interpretation ではなく descriptor を attach する。target identifier、primitive count、texture binding、transform、depth と blend state、visible correlation である。event の order が unknown なら conventional pipeline が期待する場所に置かず unknown と mark する。

60.3 contribution を conservatively group する

state、resource、geometry、timing、visual behavior が relation を support する場合だけ draw を group する。group は opaque depth-writing environment work、character component set、blended layer、UI layer、unresolved contribution と describe できる。複数 draw が adjacent であるだけで named engine subsystem を imply してはならない。

60.4 evidence を final image へ map する

各 proposed group について、どの region または behavior を explain し、何を explain しないかを述べる。controlled frame difference で map を test する。candidate draw が change すると expected image region は change するか。この process は frame のすべて pixel を一度に solve せず local contribution を establish できる。

60.5 uncertainty と alternative を報告する

useful reconstruction には confirmed node、strong-inference link、speculative link、open question を含める。各 inference について competing explanation とそれを distinguish する next observation を名付ける。blank または unresolved region は future research を direct するため false certainty の pass label より価値がある。

60.6 revisionable frame graph を publish する

result を capture と record への link を持つ dated evidence graph または ledger として present する。API event、inferred role、player-facing label を separate field に保つ。later evidence が一 link を

change するとき、history 全体を rewrite せず affected claim と confidence だけを revise する。これにより textbook は reconstruction が complete になる間も technically honest であり続ける。

証拠基準。event order と state/resource record は frame の part を confirm できる。claimed link すべてが direct support を持つまで complete graph は model に留まる。

第 61 章 確認済みの構造、未解決事項、競合するモデル

technical history はすべての gap を隠す mystery として、またはすべての plausible explanation を discovery として present してはならない。architecture map は directly established なこと、strongly inferred なこと、speculative なこと、同じ evidence を説明する複数 model がある場所を示す必要がある。本章はその publication discipline を定義する。

範囲。本章は reporting framework を定める。ファイナルファンタジー XI の complete confirmed architecture を宣言するものではない。

C++クライアント・マイルストーン。 provenance、confidence、contradiction、reproducible next test を持つ architecture/header/flag register を維持する。

```
// Cumulative implementation excerpt: versioned claims with contradictions and next tests.
struct Finding {
    std::string key, statement;
    Confidence confidence;
    std::vector<EvidenceRef> evidence;
    std::vector<std::string> contradicts;
    std::string nextTest;
};

class FindingsRegistry {
    std::map<std::string, std::vector<Finding>> history_;
public:
    void publish(Finding f) { history_[f.key].push_back(std::move(f)); }
    const Finding& current(std::string_view key) const {
        const auto& versions = history_.at(std::string(key));
        return versions.back();
    }
    bool hasUnresolvedConflict(std::string_view key) const {
        return !current(key).contradicts.empty();
    }
};
```

61.1 confirmed architecture

confirmed architecture は direct evidence に結び付く claim から成る。observed API event、resource descriptor、reproducible state change、documented platform behavior、validated meaning を持つ recovered structure、case で execution が established された implementation path である。各 claim

には stable identifier、evidence link、scope、version または client context を持たせる。evidence が broader applicability を示すまで confirmation は local である。

61.2 open question

open question は weakly written な answer ではない。precisely stated な missing link である。たとえば coordinate transform の unresolved source、scene role が unknown な draw group、content flow が incomplete な target である。gap の周囲で known なこと、absent な evidence、答える observation を述べる。これにより uncertainty は research plan になる。

61.3 competing model

different mechanism が evidence と compatible なままのとき competing model は有用である。moving surface は coordinate scrolling、animated image、geometry deformation を含みうる。distant fade は fog、LOD、blending を含みうる。各 model を prediction、supporting fact、contradictory fact、discriminating test とともに present する。一つが familiar に聞こえるから winner を選ばない。

61.4 confidence は変化できる

new evidence は claim を strengthen、weaken、split、retire できる。original date と scope を持つ earlier record を preserve し、confidence がなぜ change したかを説明する revision を add する。これにより speculative history をあたかも always confirmed だったかのように rewrite せず、reader が evidence trail の quality を judge できる。

61.5 architecture register

claim ID、subsystem または question、statement、confidence、evidence type と link、scope、alternative、contradiction、next test、revision history を持つ architecture register を maintain する。mapping が confirmed されるまで descriptive label と internal name を separate に保つ。register は narrative chapter を support しながら technical review に十分 precise であり続ける。

publication standard。本書は 'unknown'、'strong inference'、'competing model' と comfortable に述べるべきである。credibility は completeness の illusion ではなく evidence boundary から来る。

第 62 章 ヴァナ・ディールの映像技術から得られる教訓

long-lived virtual world は一つの renderer feature では説明できない。その visual identity は geometry、material、animation、atmosphere、effect、interface composition、platform constraint、それらを time に沿って arrange する pipeline の interaction から生じる。したがって本書の central lesson は technical であると同時に methodological である。convincing image は result の evidence であって、自動的に cause の evidence ではない。

範囲。この conclusion は本書の approach を summarize する。unresolved model をファイナルファンタジー XI についての confirmed fact へ変えるものではない。

C++クライアント・マイルストーン。 検証済み DAT-reading/rendering system を testable visual client へ統合し、regression scene と extension point を文書化する。

```
// Cumulative implementation excerpt: integrated visual client and regression scenes.
int VisualClient::run(const ClientConfig& config) {
    graphics_.create(config);
    for (const RegressionScene& fixture : regressionCatalog()) {
        Scene scene = assets_.loadScene(fixture.resource, evidence_);
        renderer_.draw(scene, fixture.camera);
        const Image image = graphics_.captureBackBuffer();
        const ImageDiff diff = compareImages(image, fixture.reference,
                                              fixture.tolerance);
        regression_.record(fixture.name, diff, evidence_.entries());
    }
    if (!regression_.allPassed()) return EXIT_FAILURE;
    return interactiveLoop();
}
```

62.1 frame を system として読む

frame は scene state、selected work、geometry、material と state rule、ordered composition、presentation として理解できる。各 category は互いに affect する。fog は horizon を shape しながら distance の interpretation を変え、alpha state は effect を visible にしながら order を important にし、texture は material role を suggest しながら stage operation が actual contribution を決める。system thinking は isolated visual clue が support できる以上の meaning を持つことを防ぐ。

62.2 constraint は style を作る

hardware と API constraint は option を remove するだけではない。texture、color、fog、geometry reuse、draw ordering、interface layering の particular な使い方を encourage する。analysis はこれらの pressure を respect すべきだが、visible choice のすべてを cost の direct consequence として

扱ってはならない。visual style は aesthetic decision、technical adaptation、または両方でありうる。historical explanation が obvious に感じられても evidence boundary は重要である。

62.3 observation と explanation の区別を保つ

最も reusable な research record は observed なことから始める。state value、resource binding、draw order、transform、timing change、controlled pixel result である。explanation は次に来て confidence で label される。この practice は new information が来たときにも text を useful に保つ。corrected model は original observation を erase しないからである。

62.4 uncertainty を productive にする

unknown は empty caveat ではなく discriminating test を持つ question になるべきである。competing model は prediction を述べる。negative result は record する。provisional frame graph は unresolved link を mark する。これにより future investigation は cumulative になり、reader は unsupported claim を inherit せず participate できる。

62.5 living technical history

completed textbook は revisionable であり続けるべきである。chapter は direct evidence を gain し、case study は expand され、appendix は API reference と terminology を collect し、architecture register は confidence の change を document できる。English edition と Japanese edition は prose が clarity のため adapt されても同じ claim identifier、evidence label、technical referent を preserve すべきである。この形で work は graphics technology の guide であると同時に conclusion に至った方法の durable record になる。

final standard。precision、reproducibility、candid uncertainty は本書の ambition への limit ではない。
detailed account を trust する価値あるものにする理由である。

62.6 最初の resource-to-triangle slice の completion test

最初の vertical slice は、resource identifier が installation table から解決され、file hash と chunk walk が記録され、一つの supported palette texture と一つの supported unskinned geometry record が unchecked read なしで decode され、resource が upload され、explicit material packet で indexed

triangle を draw し、selected draw から source field を表示でき、headless probe が expected structural summary を再現したとき complete である。

broader compatibility はこの completion claim に含めない。weighted geometry、追加 texture type、transformed map geometry、animation、transparent effect、environment reconstruction は、それぞれの fixture と evidence gate によって進める。小さな verified client は、黙って guess する broad parser より強い foundation である。

第 63 章 テスト、ファジング、回帰ハードニング

デモファイルだけで動くパーサとレンダラは信頼できるアプリケーションではない。本章では単体フィクスチャ、不正入力コーパス、性質テスト、デコーダファジング、構造プローブ、画像比較、寿命検査、端から端のワークスペーステストを統合する。

実装成果。

対話操作なしで全デコーダとアプリ境界を検査する階層テストランナとファズ入口を実装する。

63.1 責務と所有

テストアセットはハッシュと想定利用条件で識別する。構造期待値と画像期待値は分離する。すべての失敗はシード、リソース識別、デコーダ版、オプション、最小再現入力を報告する。

63.2 実装手順

単体テストは純粋リーダと数学、プローブはグラフィックスなしのファイル、ヘッドレス描画はフィクスチャキャプチャ、UI ハーネスはワークスペースとコマンド、ファザは境界付きチャンク変異と終了・不変条件を検査する。

63.3 障害と回復

クラッシュ、タイムアウト、無検査割当、範囲外読み取り、古いコールバック、リーク、未対応データの黙認は失敗である。画像許容差はフィクスチャごとに文書化し、構造アサーションの代替にしない。

63.4 検証ゲート

リリースゲートは複数ワーカ構成の決定論的テスト、不正コーパス、デバイス再作成、キャンセル集中、セッション移行、書き出し、完全終了時の資源集計を実行する。

63.5 累積 C++実装

C++クライアントマイルストーン。対話操作なしで全デコーダとアプリ境界を検査する階層テストランナとファズ入口を実装する。

```
// Cumulative implementation listing
int fuzzOneInput(std::span<const std::byte> bytes) noexcept {
    try {
        Budget budget{.maxBytes = 64u << 20, .maxNodes = 100000,
                      .maxDepth = 64, .deadline = Clock::now() + 100ms};
        ByteView view(bytes, budget);
        Recognition r = recognizers().classify(view.prefix(4096));
        for (const Candidate& c : r.candidates) {
            DecoderResult result = decoders().at(c.type).decode(view, budget);
            require(result.bytesConsumed <= bytes.size());
            require(result.nodeCount <= budget.maxNodes);
        }
    } catch (const BoundedParseError&) {
        return 0;
    } catch (...) {
        recordUnexpectedFailure(std::current_exception(), bytes);
        return 1;
    }
    return 0;
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

第 64 章 パッケージング、文書化、安全な拡張

最終章では動作する開発ビルドを保守可能なアプリケーションへ変える。IDE を指定しない実行要件、初回起動、移動可能設定、診断バンドル、デコーダ拡張点、互換方針、ライセンス境界、リリースチェックリストを定義する。

実装成果。

ツールチェーン非依存のリリース配置、マニフェスト、初回起動、診断バンドル、拡張登録、最終受け入れチェックリストを作成する。

64.1 責務と所有

リリースマニフェストはアプリ版、スキーマ版、デコーダ版、対応能力、プラットフォーム要件、ファイルハッシュを記録する。拡張は狭いインターフェースを通じて認識器、デコーダ、ワークスペースファクトリ、書き出し、テストを登録する。

64.2 実装手順

初回起動はデータなしで開き、適法なローカルインストールの選択方法を説明し、検証し、カタログ走査を提案する。診断バンドルはログと設定を含むが、既定ではソースアセットバイトを除外する。

64.3 障害と回復

非互換拡張は登録前に拒否する。未知設定とセッションフィールドは保持する。更新処理はソースデータを黙って書き換えたり再配布したりしない。

64.4 検証ゲート

完成ゲートはパッケージ出力を空の利用者ディレクトリへ配置し、初回設定、フィクスチャインストールのカタログ化、全ワークスペース種別のオープン、診断書き出し、セッション復元、アプリ所有データの安全な削除を実行する。

64.5 累積 C++実装

C++クライアントマイルストーン。ツールチェーン非依存のリリース配置、マニフェスト、初回起動、診断バンドル、拡張登録、最終受け入れチェックリストを作成する。

```
// Cumulative implementation listing
bool ExtensionRegistry::registerExtension(const Extension& e,
                                         const ReleaseManifest& host) {
    if (!e.requiredHost.contains(host.applicationVersion)) return false;
    if (e.abiVersion != host.extensionAbi) return false;
    for (auto& recognizer : e.recognizers) recognizers_.add(recognizer);
    for (auto& decoder : e.decoders) decoders_.add(decoder);
    for (auto& workspace : e.workspaces) workspaces_.add(workspace);
    for (auto& exporter : e.exporters) exporters_.add(exporter);
    return true;
}

DiagnosticBundle makeDiagnosticBundle(const Application& app) {
    DiagnosticBundle b;
    b.addText("manifest.json", app.releaseManifest().toJson());
    b.addText("settings-redacted.json", app.settings().redactedJson());
    b.addText("diagnostics.json", app.diagnostics().toJson());
    return b; // Source resource bytes are excluded by default.
}
```

証拠境界。コードは本書の契約に対する独立実装である。ファイル固有の意味は、対応する証拠注記が確認した範囲に限る。

付録 A グラフィックス API リファレンス

本付録は本書で用いる Direct3D 8 vocabulary の compact reference である。API capability と common interface role を説明する。ファイナルファンタジー XI で confirmed された call list として読んではならない。game-specific claim には第 4 章と第 61 章で定めた evidence standard が必要である。

reference boundary。API terminology は platform documentation で confirm される。client usage は別の問いである。

A.1 core interface

IDirect3D8 は adapter を enumerate し rendering device を create するための Direct3D object を表す。IDirect3DDevice8 は primary rendering interface である。resource を create し、state を set し、stream と texture を bind し、draw call を issue し、scene を manage し、image を present する。resource interface には IDirect3DTexture8、IDirect3DVertexBuffer8、IDirect3DIndexBuffer8、surface-oriented resource が含まれる。

A.2 device creation と presentation

IDirect3D8::CreateDevice は selected presentation と behavior parameter の下で IDirect3DDevice8 を create する。conventional scene sequence では IDirect3DDevice8::BeginScene、draw と state method、IDirect3DDevice8::EndScene、IDirect3DDevice8::Present を使う。Present は next back buffer を presentation path へ transfer する。target と depth surface は device method で query または change できる。

A.3 resource creation

representative creation method には CreateTexture、CreateVertexBuffer、CreateIndexBuffer、CreateRenderTarget、CreateDepthStencilSurface、CreateImageSurface がある。resource

descriptor は dimension、format、usage、pool を express する。application code が eligible resource を read または update するとき Lock と Unlock operation が relevant である。

A.4 transform、stream、drawing

SetTransform は standard transform category を establish する。SetStreamSource は vertex data と stride を bind し、SetIndices は index data を bind する。SetVertexShader は FVF または programmable vertex-shader path を select し、SetVertexShaderConstant は programmable processing に constant を supply する。DrawPrimitive と DrawIndexedPrimitive は primitive type と active state に従って interpreted される geometry を submit する。

A.5 state と texture-stage method

SetRenderState は depth、culling、fog、alpha testing、blending など device-level rendering condition を control する。SetTexture は texture を stage へ bind する。SetTextureStageState は fixed-function color、alpha、coordinate、その他 stage operation を control する。SetRenderTarget は rendering destination を change する。state-block method は state group を capture して apply できる。

A.6 API activity の cite 方法

trace にこれらの method が含まれる場合、exact interface/method、parameter または resulting descriptor、nearby call に対する order、client/build context、visual correlation を記録する。method を artistic label へ直接 translate してはならない。たとえば SetTexture は binding を record するが、それだけでは diffuse map、light map、effect pass を establish しない。

research rule。original API identifier は両 language edition で preserve する。identifier ではなく explanation を translate する。

A.7 toolchain 非依存の compile checklist

language mode : exception を有効にした C++20 以上。platform 宣言 : Windows および Direct3D 8。
linkage : window function が必要とする operating-system library、および Direct3D 8 entry point を

提供する import library。entry : Listing 6.4 は通常の main function を使用し、process instance を runtime で取得するため、特定の IDE project template を必要としない。

各 complete checkpoint listing は別々の実行ファイルとして扱う。累積 client では、その宣言と定義を Section 1.7 の責務 module へ移し、entry point を一つだけ残す。複数の checkpoint entry point を同じ実行ファイルへ compile してはならない。

付録 B Render State リファレンス

本 reference はよく議論される Direct3D 時代の render-state concept を visual role で group する。trace と chapter の reading aid であり、特定 client または material で confirmed された value の list ではない。

reference boundary。state category は API vocabulary である。observed value は recorded draw と context についてのみ evidence である。

B.1 depth と rasterization

depth testing は candidate fragment を stored depth と compare する。depth writing は accept 後に stored depth を update する。depth function は comparison rule を定める。depth bias-like setting は coplanar behavior を alter できる。fill mode は point、wireframe、filled primitive presentation を select する。clipping と related rasterization choice はどの geometry が later stage へ reach できるかに influence する。

B.2 culling と orientation

cull mode は front-facing、back-facing、または neither の triangle class を reject するかを定める。interpretation は vertex winding と transform convention に depend する。culling は object-level visibility selection と rasterization 後の depth rejection と distinct である。

B.3 fog

fog enablement、mode、color、start/end または density-like parameter は supported な場合 fog contribution を govern する。fog は path により conceptual に vertex-または pixel-related になり、一

部 draw で enabled、別 draw で disabled になりうる。visible fade を fog と呼ぶ前に active state と associated condition を confirm する。

B.4 alpha test

alpha testing は alpha value と alpha reference の comparison を enable する。alpha function は comparison を specify する。failed fragment は partially blend されず discard され、他 state が permit すると cutout behavior を produce する。

B.5 blending

alpha blending は source と destination color の combination を enable する。source と destination blend factor が weighting を determine し、separate alpha behavior は API path により own rule を持ちうる。destination は accumulated image であるため blend state は target、ordering、depth state とともに読む必要がある。

B.6 shading と color behavior

shading mode、lighting enablement、color-vertex behavior、material source choice、ambient-like value、related setting は vertex と material input が current color になる方法に affect する。texture-stage operation はその result を extend または replace できる。material を analyze するときこれら state category を together に record する。

B.7 trace notation

各 state record で symbolic state name、observed value、draw identifier、target、order、client context、confidence を使う。unknown value を explicit に preserve する。compact state diff は related draw 間で何が change したかを show するため full unannotated state dump より information が多いことがある。

research rule。state を directly record できる場合、visual effect だけから render state を infer してはならない。

付録 C Texture Stage ▪ Combiner リファレンス

fixed-function texture stage は texture sample、diffuse または vertex color、current result、その他 configured argument を combine して material を assemble する。本付録は operation を record するための compact notation を与える。API reference であり、特定 material または effect が particular sequence を使った evidence ではない。

reference boundary。exact stage state を先に record し、scene と resource correlation 後にのみ material role を assign する。

C.1 stage input

common conceptual input には TEXTURE、DIFFUSE、CURRENT、supported な場合 TEMP、それらの complement または alpha replica がある。TEXTURE は stage に bound された texture から sample される。DIFFUSE は common に vertex-derived diffuse color を refer する。CURRENT は prior stage processing から accumulated した result である。exact valid argument set は operation と API path に depend する。

C.2 color と alpha operation

color operation と alpha operation は separately configure される。useful conceptual operation には SELECTARG1、SELECTARG2、MODULATE、ADD、SUBTRACT-like variant、interpolation-like operation がある。stage は color を modulate しながら alpha を one source から choose でき、processing direction を disable できる。trace record では operation と every argument を preserve する。

C.3 coordinate selection

stage は selected texture-coordinate set と coordinate transform を use できる。coordinate は pass through、transform、another source から generate、その他 stage state に従って configure できる。pattern を tiled、projected、scrolling と呼ぶ前に coordinate source、transform flag、stage で available な mesh attribute を record する。

C.4 sampler-related state

addressing、minification と magnification filtering、mip filtering、related setting は stage へ supplied される sample に affect する。repetition、seam、blur、sharpness、distance behavior を create できる。sample を consume する combiner operation とは distinct である。

C.5 symbolic trace notation

active stage ごとに $C1 = \text{MODULATE}(\text{TEXTURE0}, \text{DIFFUSE})$ 、 $A1 = \text{SELECTARG1}(\text{TEXTURE0})$ のような expression を書き、appropriate な場合 $C1/A1$ を next stage の CURRENT へ feed する。unknown input または unsupported operation を explicit に mark する。expression は recorded state の readable restatement であり、その substitute ではない。

C.6 evidence checklist

stage index、bound texture、coordinate source、coordinate transform、color operation と argument、alpha operation と argument、sampler state、active vertex attribute、draw identifier、resulting visual correlation を record する。later reader は prose material label に頼らず combiner expression を reproduce できるべきである。

research rule。chain 内の position だけから stage を light map、detail map、reflection map、mask と呼んではならない。

付録 D 行列と座標空間の入門

本付録は real-time graphics で使う transform の compact mathematical vocabulary を与える。convention は異なる。row/column vector、multiplication order、handedness、depth range は particular API path または client について verify する必要がある。ここでの symbol は explanatory notation として使い、assumed implementation convention として使わない。

convention boundary。matrix equation は relevant convention と value が observed されるまで model である。

D.1 point、vector、homogeneous coordinate

point は location を表し、vector は direction または displacement を表す。homogeneous coordinate は fourth component を add し、translation、rotation、scale、projection-like operation を matrix で表現できる。common notation では position は $w = 1$ 、direction は $w = 0$ を使うが、storage と multiplication convention は依然 important である。

D.2 local-to-world transform

local position p_{local} は world transform W で world space に置かれる。一 notation では $p_{\text{world}} = p_{\text{local}} \times W$ と書く。別 convention では $W \times p_{\text{local}}$ と書く。これらを mix してはならない。本質は変わらない。 W は reusable local geometry を scene position、orientation、scale へ place する。

D.3 view と projection transform

view transform V は world position を camera に relative に express する。projection transform P は camera-relative coordinate を clip space へ map する。one multiplication convention の conceptual sequence は $p_{\text{clip}} = p_{\text{local}} \times W \times V \times P$ である。perspective division は viewport mapping 前に normalized screen-related coordinate を yield する。

D.4 hierarchy

attachment または bone hierarchy では child の final transform は local transform と parent の accumulated transform を combine する。multiplication order は convention に depend するが、dependency が key fact である。parent を change するとすべての descendant を move できる。animation を analyze するとき local と parent relationship を separate に record する。

D.5 texture coordinate

texture coordinate は通常 two-dimensional value (u, v) であり、transform のため extend されることが多い。texture transform は object position と independent に coordinate を scale、rotate、translate、project できる。static mesh 上で image が scroll するとき coordinate transform は candidate explanation の一つであり、stage と transform evidence なしに conclusion ではない。

D.6 practical notation

各 recorded transform について source space、destination space、matrix または constant identifier、known な場合 multiplication convention、affected draw を identify する。ambiguous name より local→world、world→view のような label を prefer する。recovered matrix を visual expectation へ fit させるため silent に transpose せず unknown convention を explicit に mark する。

research rule。source/destination space のない matrix value は incomplete evidence である。

付録E ブレンド、深度、アルファ計算式

本付録は common depth、alpha-test、blending behavior を describe する compact formula を使う。formula は conceptual reading tool である。API detail、color representation、target format、active state が draw の actual result を determine する。

reference boundary。relevant state、target、ordering を record するまで formula を apply してはならない。

E.1 depth comparison

z_s を source depth、 z_d を stored destination depth とする。depth function は fragment が proceed するかを decide する。particular convention で $z_s < z_d$ または $z_s \leq z_d$ のような relation である。depth writing が enabled で fragment が accepted なら z_d は z_s に replace されうる。comparison direction と near/far の meaning は path ごとに verify する必要がある。

E.2 alpha testing

α_s を source alpha、 α_{ref} を configured reference とする。alpha test は comparison function $f(\alpha_s, \alpha_{ref})$ を evaluate する。false なら fragment は discard され、true なら later operation へ continue する。tested fragment では binary acceptance であるが、filtering は larger scale で visible edge を soft に見せうる。

E.3 general color blending

common conceptual form は $C_{out} = C_s \times F_s + C_d \times F_d$ である。 C_s は source color、 C_d は destination color、 F_s/F_d は configured source/destination factor である。factor は active blend

state に従い source alpha、destination alpha、color、constant に depend できる。equation は earlier test と operation が fragment を blending へ reach させた後にだけ apply される。

E.4 familiar special case

source-alpha-like translucency は conceptually $C_{out} = C_s \times \alpha_s + C_d \times (1 - \alpha_s)$ と表せる。

additive-like accumulation は chosen source factor に対して $C_{out} = C_s \times F_s + C_d \times 1$ と表せる。これらの name は convenient shorthand だが、exact factor、clamping、alpha-channel behavior は trace から読む必要がある。

E.5 ordering と destination

destination C_d は draw が blend される moment の target color である。したがって identical source draw でも different opaque draw、transparent draw、clear color、target pass の後では different result を produce できる。blend equation は target と order なしには complete ではない。このため transparent effect には texture image だけでなく pass ledger が必要である。

E.6 formula record

analysis で使う各 formula について source resource と stage result、alpha-test

enablement/function/reference、blend enablement/factor、depth test/function/write、target、order、color/format assumption を record する。どの term が observed でどれが inferred かを state する。これにより equation は decorative ではなく auditable になる。

research rule。familiar visual result は familiar blend equation を license しない。factor と destination chain を verify する。

付録 F 証拠台帳テンプレート

一つの bounded technical claim に一つの register entry を使う。entry は independently reviewable な result にするべきである。別 reader が何を observed し、どの condition で、何を support し、何が possible なままか、どう further test するかを understand できるようにする。

template rule。この form を uncertain claim を formal に見せるために使ってはならない。missing evidence は entry に属する。

F.1 identity

Claim ID : RV-YYYY-NNN のような stable identifier。 Title : short neutral description。 Status : confirmed、 strong inference、 speculation。 Revision : date、 author/editor、 superseded entry への link。

F.2 claim と scope

Claim : evidence が support する範囲に限定した one sentence。 Scope : platform、 client/build、 scene または asset context、 claim が apply する condition。 local observation を all zone、 model、 platform、 version へ silent に generalize してはならない。

F.3 observation record

Observation : direct に occurred または appeared したこと。 Context : location/landmark、 camera、 time/weather、 display configuration、 UI/effect state、 capture method。 Evidence type : direct capture、 controlled comparison、 API trace、 resource inspection、 implementation analysis、 documentation、 または stated source。 stable file、 image、 trace、 citation identifier を attach する。

F.4 technical record

claim に relevant な field だけを record する。 target、 draw order、 geometry/buffer range、 transform、 texture、 stage または shader、 render state、 blend/depth behavior、 resource update、 timing、 output change である。 original identifier と value を preserve する。 unavailable field は assumed label へ replace せず unknown と mark する。

F.5 alternative と confidence

Alternative : evidence と compatible なままの explanation を list する。 Contradiction : cleanly fit しない observation を list する。 Confidence rationale : status が confirmed、 strong inference、 speculation である理由を explain する。 confirmed な narrow fact はその purpose についての speculative explanation と coexist できる。

F.6 next test と publication text

Next test : alternative を discriminate する smallest observation または experiment。Publication

text : correct confidence label を使う reader-facing sentence。Source link : primary

documentation、capture、trace record、related claim ID。Revision note : 何がなぜ change したかを record する。

neutral form の example。'RV-2026-014 : recorded Windows client/build と camera condition で、draw D-184 はこれらの observed blend/depth state を使う。visible glow への contribution は strong inference であり、target sequence は unresolved である。'

付録 G 用語集

本用語集は recurrent term の working meaning を fix する。API identifier は standard English form のままにする。glossary definition は explanatory であり、named feature が particular client に occur する evidence ではない。

terminology rule。claim confidence を vocabulary から separate に preserve する。precise term でも speculative model を describe できる。

G.1 core term

Alpha test — alpha reference に対する binary fragment comparison。Alpha blending — blend factor による source/destination color の combination。Back buffer — presentation のために prepare される device-owned surface。Blend factor — source または destination contribution を weight する value。Culling — orientation または broader visibility rule による geometry rejection。どの kind かを specify する。

Draw call — active resource と state で primitive を submit する command。Fragment — final target write 前の candidate pixel-level result。Geometry — primitive として interpreted される vertex/index data。Material — draw の surface contribution を determine する active resource/state recipe。Pass — ordered rendering contribution。order と target context の evidence があるときだけ使う。

G.2 coordinate と scene term

Local space — object に relative な coordinate。World space — object を scene に place する coordinate。View space — camera に relative な coordinate。Projection — camera-related coordinate から screen space へ向かう mapping。Frustum — camera-defined viewing volume。Transform — space または position 間の mapping。common に matrix で represent される。

Zone — player-facing area label。render partition と equal だと assume しない。Chunk — spatial subset の analytical label。internal meaning を confirm してから fact として使う。Draw group — observed state/resource/order evidence で together に treated される submission set。

G.3 resource と pipeline term

Texture — stage または material に bound される sampled image data。Surface — texture level、target、depth surface になりうる two-dimensional device storage。Mipmap — reduced-resolution texture level の sequence。Vertex buffer — vertex record を含む resource。Index buffer — vertex reference を含む resource。Texture stage — configured input を combine する fixed-function unit。Render state — draw interpretation に affect する device setting。

G.4 evidence term

Confirmed — reproducible observation、primary documentation、implementation evidence、equivalent decisive evidence で directly support される。Strong inference — decisive gap が残る best-supported explanation。Speculation — plausible だが unproven な model。Controlled comparison — condition を record し—relevant variable を change する test。Evidence register — claim を context、source、alternative、confidence へ connect する structured record。

editorial rule。source 内の term meaning が uncertain なら source term を quote または preserve し、interpretation を unresolved と mark する。

付録 H 技術的発見の年表

本付録は dated research ledger であり、guess の chronology ではない。各 entry は claim が textbook へ entered した時、support する evidence、その時点の confidence、later revision が何を change し

たかを record する。corresponding evidence-register entry が存在するまで client-specific finding を含めてはならない。

chronology rule。claim の date と original confidence を preserve する。history を rewrite せず revision を add する。

H.1 entry format

Date — Claim ID — Short finding — Scope — Evidence types — Confidence — Source/record link — Revision status。sorting には ISO date format (YYYY-MM-DD)を使う。observation の event date が claim entered date と different なら both を record する。

H.2 initial editorial milestone

2026-08-09 — RV-ED-001 — aligned chapter、appendix、evidence-label、terminology structure を持つ English/Japanese edition を establish した。Scope : textbook editorial framework。Evidence : document revision record。Confidence : confirmed。

2026-08-09 — RV-ED-002 — main text は confirmed finding、strong inference、speculation の separation を establish する。Scope : editorial policy。Evidence : Chapter 1–5 と Appendix F。Confidence : confirmed。

2026-08-09 — RV-ED-003 — Direct3D 8 API vocabulary は reference context として document され、client-specific usage は evidence pending のため unentered のままである。Scope : API reference policy。Evidence : Appendix A と primary API documentation。Confidence : confirmed。

H.3 client-specific finding policy

client-specific entry を add する前に evidence-register record を create する。chronology entry は duplicate せずその record を summarize する。platform、client/build、scene または asset scope、confidence を include する。later test が finding に contradict する場合、both record へ link する new dated revision entry を add する。

H.4 future entry の example

YYYY-MM-DD — RV-YYYY-NNN — Observed draw/state/resource fact。 Scope : platform、 client/build、 scene。 Evidence : trace と controlled capture。 Confidence : confirmed。 Revision : none。

YYYY-MM-DD — RV-YYYY-NNN — Observed behavior の best-supported explanation。 Scope : stated condition。 Evidence : controlled comparison と trace。 Confidence : strong inference。 Next test : stated discriminating observation。

current status。 game-client-specific technical finding はまだこの chronology へ entered していない。 これにより本書の stated evidence standard を preserve する。

付録 I 参考文献と一次資料

本 bibliography は platform context と API vocabulary に使う primary/official technical source を record する。 現段階では intentionally narrow である。 new client-specific claim には support する exact evidence record、 capture、 trace、 document、 primary source を add する必要がある。

citation rule。 related topic を discuss する page ではなく claim を support する source を cite する。

I.1 official Final Fantasy XI source

Square Enix. FINAL FANTASY XI — WE ARE VANA'DIEL 20th Anniversary Commemorative Website. Adventurers' History. <https://we-are-vanadiel.finalfantasyxi.com/history/?lang=en> Accessed 2026-08-09. high-level platform/service-history context に使用。

Square Enix. Switching from the PlayStation 2 or Xbox 360 Version to the Windows Version. PlayOnline. <https://www.playonline.com/ff11us/envi/win/winshift.html> Accessed 2026-08-09. official 2016 platform-service transition notice に使用。

I.2 official Direct3D 8 reference

Microsoft. IDirect3DDevice8. Microsoft Learn, archived documentation. <https://learn.microsoft.com/en-us/previous-versions/windows/embedded/>

ms889277%28v%3Dmsdn.10%29 Accessed 2026-08-09. device-interface/method-group reference に使用。

Microsoft. IDirect3DDevice8::BeginScene. Microsoft Learn, archived documentation.
<https://learn.microsoft.com/sr-latn-rs/previous-versions/ms889279%28v%3Dmsdn.10%29>
Accessed 2026-08-09. scene-boundary reference に使用。

Microsoft. IDirect3DDevice8::Present. Microsoft Learn, archived documentation.
<https://learn.microsoft.com/en-us/previous-versions/ms889707%28v%3Dmsdn.10%29> Accessed 2026-08-09. back-buffer presentation reference に使用。

Microsoft. IDirect3DDevice8::CreateTexture. Microsoft Learn, archived documentation.
<https://learn.microsoft.com/en-us/previous-versions/ms889290%28v%3Dmsdn.10%29> Accessed 2026-08-09. texture-resource reference に使用。

Microsoft. IDirect3DDevice8::CreateVertexBuffer. Microsoft Learn, archived documentation.
<https://learn.microsoft.com/en-us/previous-versions/ms889291%28v%3Dmsdn.10%29> Accessed 2026-08-09. vertex-buffer reference に使用。

I.3 evidence-register source

future technical claim では evidence register を first に cite し、その後 underlying primary material を cite する。original capture、trace、resource descriptor、controlled comparison、executable-analysis note、official documentation、explicitly identified archival source である。chapter 自身が evaluate する claim を support する唯一の source として explanatory chapter を cite してはならない。

current status。bibliography は book の platform/API reference context を support する。client-specific rendering finding の source list ではまだない。

付録J 索引

本 subject index は working manuscript が change しても valid なままになるよう page number ではなく chapter/appendix reference を使う。entry は discussion location を identify するものであり

confirmation status ではない。term を game-specific fact として扱う前に referenced section の evidence label を consult する。

index rule。term が merely mentioned されるだけでなく substantively explained されるとき entry を add する。

J.1 A–D

Alpha blending — 17、37、39、59、Appendix E。Alpha test — 16、17、37、Appendix B、Appendix E。Animation — 27、29、58。API reference — 7、18、19、Appendix A。Aspect ratio — 2、45。Back buffer — 7、Appendix A、Appendix G。Batching — 49。Blend factor — 17、37、39、Appendix E。Camera — 3、11、24、32、45。Character assembly — 27、58。Chunk — 20、24、Appendix G。Coordinate space — 11、Appendix D。Culling — 16、24、Appendix B。Decal — 40。

J.2 E–M

Depth testing/writing — 16、17、34、39、59、Appendix B、Appendix E。Device — 7、Appendix A。Draw call — 3、7、12、Appendix G。Evidence register — 4、60、61、Appendix F。Fog — 16、32、33、Appendix B。Frame reconstruction — 3、60。Frustum — 11、24、Appendix D。Interface rendering — 41、42。Material — 13–18、31。Matrix — 11、Appendix D。Mipmap — 13。Multi-pass effect — 37、39、59。

J.3 N–S

Normal — 12、31。Occlusion — 24。Opaque draw — 16、17、57。Particle system — 35、37、59。Presentation — 3、7、45。Projection — 11、40、Appendix D。Render state — 16、49、Appendix B。Render target — 7、13、34、39、40。Resource lifetime — 7、49、52。Sky — 32。Skinning — 27。Sorting — 17、37、39、59。Streaming — 52。Surface — 13、25、34、Appendix G。

J.4 T–Z

Terrain — 20–25。Texture — 13、18、Appendix C。Texture coordinate — 11、18、34、Appendix D。Texture stage — 18、Appendix C。Transform — 11、27、29、Appendix D。Transparency —

17、34、37、39、59。UI — 41-45。Vertex buffer — 12、Appendix A。Vertex color — 31。Vertex shader — 19。Visibility — 20、24。Water — 34。Weather — 35。World scene — 20-26。

maintenance note。future chapter が confirmed client-specific finding を add するとき、reader が evidence record を quickly locate できるよう chapter reference とともに claim ID を add する。